



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
SEMESTER 2 2025/2026

SECP37843-02 – SPECIAL TOPIC IN DATA ENGINEERING

PROJECT: RECOVERABLE ASSETS (RA) & INVENTORY RISK MANAGEMENT
(IRM)

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 6

NAME OF STUDENTS	MATRIC NO.
SAFIYA NURSYAHADAH BINTI MASNOOR	A23CS0176
TAN ZHI MING	A23CS0189
FARRA NURZAHIN BINTI ZAHARIL ANUAR	A23CS0079
AIN NURNABILA BINTI MOHD AZHAR	A23CS0207
WOO CHENG SHUAN	A23CS0283
WUCHENXI	X25EC3007

ABSTRACT

Inventory mismanagement in manufacturing environments remains a persistent operational and financial challenge, particularly when organisations rely on periodic, manual reviews to detect risks such as surplus stock, expired materials and critical shortages. This project aims to develop an automated, end-to-end data engineering pipeline integrated with an interactive business intelligence dashboard to support Recoverable Assets (RA) and Inventory Risk Management (IRM) for PPG, a paint and coatings manufacturer. The study is academically significant as it applies established data engineering frameworks, namely the Medallion Architecture and Star Schema dimensional modelling to a real-world industrial context, while its practical importance lies in replacing error-prone manual spreadsheet processes with a consistent, auditable, and scalable automated system. The pipeline was developed using Microsoft Azure cloud services, following a four-layer medallion-inspired architecture comprising Bronze, Silver, Gold and Curated layers, orchestrated through Azure Data Factory, processed via Azure Databricks, warehoused in Azure Synapse Analytics and visualised through Microsoft Power BI. The Bronze layer ingests six raw CSV datasets covering materials, inventory snapshots, purchase orders, suppliers, production orders and sales orders; the Silver layer performs data cleaning and standardisation; the Gold layer applies PPG's business classification rules for Recoverable Assets, Magna Carta Dead Stock, expired materials, stockout risk, production fulfilment and customer order impact; and the Curated layer organises the output into a Star Schema for reporting. Key findings reveal that eleven materials were identified as Recoverable Assets with an excess stock value of RM92,770, three materials were classified as Magna Carta Expired with a total financial provision of RM29,720, nine materials fell below their reorder levels causing 183 unfulfilled production orders and 192 affected customer sales orders. The project successfully demonstrates that a cloud-based, automated data engineering pipeline can transform fragmented raw operational data into timely, accurate and actionable inventory risk insights. From a practical and managerial standpoint, the dashboard equips inventory managers, financial controllers and procurement teams with a centralised, self-service tool that enables early risk detection, supports compliance with IAS 2 inventory valuation standards and facilitates more informed and proactive decision-making across the organisation.

TABLE OF CONTENTS

	TITLE	PAGE
CHAPTER 1	INTRODUCTION	1
1.1	Introduction	1
1.2	Problem Background	2
1.3	Project Aim	3
1.4	Project Objectives	3
1.5	Project Scope	3
1.6	Project Importance	4
1.7	Report Organization	5
CHAPTER 2	LITERATURE REVIEW	6
2.1	Introduction	6
2.2	Supply Chain Management	6
2.3	Enterprise Resource Planning	8
2.4	Related Theory Used	8
2.5	Visualization	10
2.6	Comparison of Visualization Tools	11
2.7	Microsoft Azure	12
2.8	Microsoft Power BI	14
2.9	Chapter Summary	15
CHAPTER 3	METHODOLOGY	16
3.1	Introduction	16
3.2	Proposed Methodology	16
3.3	Phases of the Chosen Methodology	17
3.4	Project Planning Schedule	19
3.5	Technology Used Description	19
3.5	Chapter Summary	22

CHAPTER 4	ANALYSIS AND DESIGN	23
4.1	Introduction	23
4.2	System Architecture	23
4.3	Requirement Analysis	31
4.4	Project Design	34
4.5	Database Design	46
4.6	Interface Design	56
4.7	Chapter Summary	59
CHAPTER 5	IMPLEMENTATION AND TESTING	60
5.0	Introduction	60
5.1	System Development	61
5.2	Bronze Layer	61
5.3	Silver Layer	65
5.4	Gold Layer	71
5.5	Curated Layer	81
5.6	Power BI Dashboard Interface	90
5.7	Chapter Summary	100
REFERENCES		101

LIST OF TABLES

TABLE NO.	TITLE	PAGE
Table 4.1	Datasets	25
Table 4.2	Business Questions	29
Table 4.3	Technology Stack Summary	31
Table 4.4	Functional Requirement	32
Table 4.5	Non-functional Requirements	33
Table 4.6	Use Case Description for View Materials Below Reorder Level	36
Table 4.7	Use Case Description for View Recoverable Assets	37
Table 4.8	Use Case Description for View Magna Carta Dead Stock	38
Table 4.9	Use Case Description for View Magna Carta Expired Materials	39
Table 4.10	Use Case Description for View Total MC Provision Amount	40
Table 4.11	Use Case Description for View Unfulfilled Production Orders	41
Table 4.12	Use Case Description for View Affected Customer Sales Orders	42
Table 4.13	Source Dataset Description	47
Table 4.14	Fact Inventory Status	49
Table 4.15	Dim Material	51
Table 4.16	Dim Supplier	51
Table 4.17	Dim Time	52
Table 4.18	Dim Purchase Order	52
Table 4.19	Dim Production Order	52
Table 4.20	Dim Sales Order	53
Table 4.21	Database Relationship	53
Table 4.22	Business Rule Mapping	54
Table 4.23	Key Analytical Fields	55
Table 5.1	Business Questions Validation Result	88

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
Figure 2.1	Medallion Architecture Data Layer	9
Figure 2.2	Star Schema Dimensional Model	10
Figure 2.3	Azure End-to-End Data Pipeline Architecture	13
Figure 2.4	Microsoft Power BI — Inventory Risk Management Dashboard Layout	14
Figure 3.1	Gantt Chart	19
Figure 3.2	Azure-Based End-to-End Data Pipeline Architecture	20
Figure 4.1	End-to-End Data Pipeline Architecture Diagram	24
Figure 4.2	Use Case Diagram: Recoverable Assets & Inventory Risk Management System	35
Figure 4.3	Activity Diagram: Recoverable Assets & Inventory Risk Management System	44
Figure 4.4	Sequence Diagram: Recoverable Assets & Inventory Risk Management System	45
Figure 4.5	Proposed Star Schema Design	48
Figure 4.6	MC Overview	57
Figure 4.7	Stockout & Production Fulfillment Risk	58
Figure 4.8	Impacted Orders & Customers	58
Figure 5.1	Raw data uploaded	62
Figure 5.2	Create Linked Service	62
Figure 5.3	Data flow	63
Figure 5.4	Data pipeline ingestion	64
Figure 5.5	Data quality logic result outputs	65
Figure 5.6	Bronze Container Ingested Folder	65
Figure 5.7	Output Shows All 6 Files Loaded	66
Figure 5.8	Remove Duplicates from All Datasets	66
Figure 5.9	Output shows Duplicates Removed	66
Figure 5.10	Fill Nulls with Appropriate Defaults	67
Figure 5.11	Output Shows Null Values Handled	67
Figure 5.12	Fix Data Columns to Proper Date Format	68

Figure 5.13	Output Shows Data Types Fixed	68
Figure 5.14	Standardize String Casing	69
Figure 5.15	Output Shows The Formats Are Standardized	69
Figure 5.16	Final Validation	70
Figure 5.17	Output Shows Final Validation Row Counts and Null Counts	70
Figure 5.18	Silver Container Showing All 6 Folders	70
Figure 5.19	Inside One Folder Showing Parquet Files.	71
Figure 5.20	Loading Silver Parquet Files into Databricks	72
Figure 5.21	Gold Container Path Configuration	73
Figure 5.22	Material Consumption Calculation	73
Figure 5.23	Inventory Status Gold Transformation	74
Figure 5.24	Recoverable Asset and Reorder Risk Logic	75
Figure 5.25	Magna Carta Expired and Dead Stock Classification	76
Figure 5.26	Stockout Risk Gold Output	77
Figure 5.27	Magna Carta Provision Summary Output	78
Figure 5.28	Production Fulfilment Gold Output	79
Figure 5.29	Sales Order Impact Gold Output	80
Figure 5.30	Gold Container Output Folders	81
Figure 5.31	Loading Gold Parquet Files into Databricks	82
Figure 5.32	Dimension Tables Construction Code	83
Figure 5.33	Dimension Tables Output	83
Figure 5.34	Fact Table Construction and Output	84
Figure 5.35	Curated Container Showing All Nine Output Folders	85
Figure 5.36	SQL Views Registered in Azure Synapse Analytics curated_db	86
Figure 5.37	fact_inventory_status	87
Figure 5.38	Star Schema	89
Figure 5.39	Model View Relationship	91
Figure 5.40	Executive Summary Home Page — All 7 Business Questions Overview	92
Figure 5.41	Q1 Page – Materials Below Reorder Level Dashboard Page	93
Figure 5.42	Q2 Page – Recoverable Assets Dashboard Page	94

Figure 5.43	Q3 Page – MC Dead Stock Materials Dashboard	95
Figure 5.44	Q4 Page – MC Expired Materials Dashboard	96
Figure 5.45	Q5 Page – Total MC Provision Amount Dashboard	97
Figure 5.46	Q6 Page – Unfulfilled Production Orders Dashboard	98
Figure 5.47	Q7 Page – Affected Customer Sales Orders Dashboard	99

LIST OF APPENDICES

APPENDIX	TITLE	PAGE
Appendix A		40
Appendix B		41
Appendix C		42

CHAPTER 1

INTRODUCTION

1.1 Introduction

Manufacturing companies always deal with a surprisingly difficult and hard balancing act once it comes to the involved raw materials. Holding too many stocks will tie up capital at the same time risks materials unused until they finally expire, meanwhile holding too small amounts can cause production to a halt as well as leaving customers waiting. In industries, for instance paints and coatings where materials are ranged from pigments to packaging, this crucial challenge is stressed as each material behaves differently in terms of how long it lasts and so on. As for a company like PPG, managing twenty raw materials across different production lines without a reliable system causes risks to quietly build up before anyone notices them.

This project is meant to solve the problem mentioned by building an end-to-end data engineering pipeline which will continuously monitor inventory condition and flags materials which are running dangerously low. The pipeline follows a four-layer medallion-inspired architecture consisting of Bronze, Silver, Gold, and Curated layers. Bronze stores the raw uploaded source files, Silver performs cleaning and standardization, Gold applies business rules to produce inventory risk outputs and Curated organizes the final analytical data into a Star Schema exposed through Azure Synapse Analytics for power BI reporting.

What supports this project's relevance is that it replaces one process that a lot of organizations still handle manually through their prepared periodic spreadsheet reviews. By automating the classification rules as well as integrating the whole data sources into a one single pipeline, the systems manages to give inventory managers,

financial controllers, and procurement teams a disciplined picture of inventory risk which is reproducible with every run compared to dependent on the last person who opened the spreadsheet.

1.2 Problem Background

The main problem this project addresses stems from how most of the inventory risks actually tend to be managed in massive manufacturing organizations. Instead of being monitored continuously, the stock conditions are usually examined periodically, often on quarterly or year-end, using data that has already been manually taken from different operational systems. By the time a problematic material is caught on, it may have been sitting untouched for probably months, quietly accumulating causing economic value losses. The manual nature of this process also means that the business rules for grouping at-risk stock are used inconsistently, with zero reliable audit trail, and the connection between a specific material shortage and its impact on the customer orders is difficult to trace without a crucial amount of manual consolidation.

PPG is currently using two frameworks to classify their inventory risk. The Recoverable Assets (RA) framework flags some materials which stock on hand is more than twelve months of consumption, which is a signal that surplus has already built beyond what typical production can absorb. Where that surplus has gone further, the Magna Carta (MC) framework steps in which materials with no consumption exceeding nine months are grouped as Dead Stock. Meanwhile, materials which have already passed expiry date are grouped as Expired. Those groups carry very stressed financial consequences towards the company, as the International Accounting Standard 2 (IAS 2 - Inventories) needs organizations to list inventory with zero net of realizable value as quickly as possible. Once these materials go undetected, the balance sheet will overstate the company's assets which auditors, regulators, and lenders may raise audit and reporting concerns.

On the other hand, stockout events occur once the available stock drops below reorder point even before the arrival of the supplier's replenishment. In paint manufacturing, one missing material may delay a few production orders at a time. Without the visibility into which materials are near to the threshold, the commercial

team has no chance to act as early as possible, whether that means expediting a delivery or even managing customer expectations before the stock shortage turns into an operational disruption. This project mentions both sides of the inventory risk problem by building a pipeline on Microsoft Azure that automates classifications, detections, and end-to-end impact tracings.

1.3 Project Aim

The aim of this project is using Microsoft Azure and Power BI to create an interactive dashboard and an end-to-end data engineering pipeline that will automate inventory risk classification and track the effects of stockouts for PPG.

1.4 Project Objectives

- I. To design a four-layer medallion-inspired data pipeline consisting of Bronze, Silver, Gold, and Curated layers on Microsoft Azure to ingest, process, model, and serve inventory risk data.
- II. To implement data transformation processes that apply PPG's Recoverable Assets (RA) and Magna Carta (MC), stockout risk, production fulfilment, and customer order impact logic.
- III. To develop an interactive dashboard using Microsoft Power BI that visualizes the analytics-ready data from the Curated Star Schema to answer seven inventory and order impact business questions.

1.5 Project Scope

The project starts with the ingestion of data whereby six CSV datasets, namely materials, inventory snapshot, purchase orders, suppliers, production orders and sales orders are uploaded into Azure Data Lake. The Bronze layer stores the uploaded raw files without transformation. Data cleaning, duplicate removal, null handling, and data type standardization are performed in the Silver layer.

This will be followed by an emphasis of the project on data processing through the application of certain business rules. This will involve the identification of Recoverable Assets (materials that have a stock level of over 12 months of usage), identification of Magna Carta Dead Stock (materials that have not been used in more than nine months) and identification of expired materials using their expiry dates. The system will also be used to analyze stock levels to identify the materials that may be under threat of depletion, and how the shortages can impact on the sale of the products to the customers.

Once processed, the data will be in a clean form and transformed form, which will be arranged in a structured format (data warehouse) that can be analyzed. Lastly, the Power BI dashboard will be created to show the results in a clear and interactive manner that will assist the users in grasping the inventory status with ease and make more capable decisions.

Nevertheless, this project is limited in several ways. The partner will be an industry to provide the dataset, and it might be limited in terms of availability, completeness, and accessibility of the data. Some data can be sensitive and confidential hence adequate data handling and privacy should be observed during the project.

Moreover, the project will not use real-time data processing, but will emphasize on batch data processing, as time and resource limits do not allow real-time data processing. The deployment is confined to the usage of the Microsoft Azure and Power BI as primary tools. No advanced features (machine learning or predictive analytics) are present since the core task is to build a working and dependable data engineering pipeline to analyze inventory and report on it.

1.6 Project Importance

This project is valuable as it aims at solving major issues of inventory management, including overstocking of materials, expiration, and stock-outs, which may cause financial losses and inefficiencies in operation.

The project is set to address these challenges by building a data engineering pipeline where raw data about the company is converted to valuable insights. It assists

in determining the surplus stock (Recoverable Assets), unproductive or expired goods (Magna Carta) and goods in danger of shortage. The company will be able to take the early steps of minimizing wastes and eliminating production and customer order disruptions with this information.

Moreover, the project will offer a centralised dashboard that will simplify the task of the decision-makers to track the inventory and take immediate action in case of any issues.

As it is a work-based learning project based on industry data, it is also practical since the data engineering concepts can be applied in a practice-based environment. In general, the project helps in making better decisions, increasing efficiency and better management of inventory.

1.7 Report Organization

To provide the project in an understandable and orderly way, this report is divided into six chapters.

- (a) Chapter 1 Introduction: An overview of the project, including its background, problem statement, goal, objectives, scope, and significance.
- (b) Chapter 2 Literature Review: The connected studies, pertinent theories, and technologies utilized in this project, such as inventory management, data engineering concepts, and the tools used.
- (c) Chapter 3 Methodology: The technique utilized to complete the project, including the selected methodology, its stages, and the technologies involved.
- (d) Chapter 4 Analysis and Design: The system architecture, requirement analysis, and general system design, including database and interface design.
- (e) Chapter 5 Implementation and Testing: The system's development, including coding, system interfaces and testing to guarantee proper operation.
- (f) Chapter 6 Conclusion: Describes the project, assesses the accomplishment of goals, and offers recommendations for future enhancements.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter establishes the theoretical and technical framework for developing an automated, cloud-based data engineering pipeline designed for inventory risk management. It begins by examining supply chain principles regarding optimal stock levels to mitigate the financial risks of overstocking and material shortages. Furthermore, it analyzes the methodologies for extracting high-volume enterprise data and refining it through a structured multi-layer architecture. Finally, a comparative evaluation of cloud technologies is presented to justify the selection of the Microsoft Azure ecosystem and Power BI for transforming raw operational data into actionable, decision-support dashboards.

2.2 Supply Chain Management

Supply chain management (SCM) refers to the process of planning, coordinating, and controlling all activities involved in moving a product from its origin to the end of the customer. SCM covers the movement and storage of raw materials, work-in-progress inventory, and finished goods from the point of origin to the point of consumption including all the information systems needed to coordinate these activities (Mentzer et al., 2001). It connects functions such as sourcing, production, logistics, and delivery into one integrated system, enabling organizations to operate more efficiently and respond more effectively to changes in demand.

2.2.1 Inventory Management in Supply Chain Management

Inventory management is essential in supply chain management because it helps organizations handle uncertainty and maintain smooth operations. According to Nitsche (2018), supply chains today face high levels of volatility due to demand

fluctuations and disruptions, making it necessary for companies to adopt flexible inventory strategies. Inventory acts as a buffer that protects the supply chain from unexpected changes, ensuring that production and customer demand can still be met. However, poor inventory decisions can lead to either excess stock, which increases holding costs, or shortages, which disrupts operations and reduces customer satisfaction.

In addition, the use of proper inventory management techniques and technology significantly improves supply chain efficiency. Issah et. al (2025) explains that methods such as Economic Order Quantity (EOQ), Just-in-Time (JIT), and ABC analysis help organizations optimize stock levels and reduce waste. They also highlight that technology readiness plays an important role, as systems like real-time tracking and data analytics allow better monitoring and decision-making. As a result, organizations that combine effective inventory techniques with advanced technology are more efficient, responsive, and competitive in managing their supply chains.

2.2.2 End-to-End Data Pipeline Integration in Supply Chain Management

End-to-end data pipelines play an important role in modern supply chain management by enabling seamless data flow from raw sources to analytics-ready outputs. According to Behera and Chilukoori (2024), an end-to-end data pipeline integrates multiple automated stages including data ingestion, transformation, storage, and analytics delivery to ensure that raw operational data is converted into meaningful insights. These pipelines typically collect data from structured systems and real-time sources, process it using distributed computing tools, and store it in scalable environments such as data lakes and warehouses before delivering it to business intelligence platforms. This structured flow improves data consistency, accessibility, and decision-making efficiency in supply chain operations. In addition, Behera and Chilukoori (2024) emphasize that modularity is a key principle of pipeline design, where each stage operates independently, allowing organizations to update, troubleshoot, or reprocess data without disrupting the entire system. This modular structure aligns well with supply chain analytics environments, where flexibility and fast response to changes are essential for maintaining operational efficiency.

2.3 Enterprise Resource Planning (ERP)

Enterprise Resource Planning (ERP) refers to an integrated software system that organizations use to manage their core business processes through a single platform. ERP is critical to enhancing an organization's ability to control commercial activities and can result in a competitive advantage when combined with an organization's existing strengths (Harun & Dorasamy, 2023). By connecting key functions such as finance, procurement, inventory and human resources in real time, ERP systems enable organizations to make better decisions and improve overall operational efficiency.

2.3.1 ERP Data Engineering and ETL Pipeline

ERP systems generate large volumes of complex data, making data engineering and ETL processes challenging to manage. According to Kavala (2023), modern ERP environments require scalable data engineering frameworks that can handle high data volume, diverse data types, and continuous system changes. Traditional ETL pipelines are often rigid and rule-based, making it difficult to adapt when business requirements or data structures change. As a result, organizations need more flexible and intelligent ETL solutions, such as those supported by artificial intelligence, to improve data processing efficiency and ensure accurate transformation of ERP data into analytics-ready formats.

In addition, integrating big data technologies with ERP systems can significantly improve data processing and analytical performance. Bandara et al. (2023) found that technologies such as Hadoop and NoSQL databases enhance ERP responsiveness by enabling faster data processing, better data integration, and improved decision-making capabilities. These technologies support real-time analytics and provide greater visibility into business operations. Therefore, combining ERP systems with advanced data engineering pipelines and big data tools allows organizations to overcome ETL challenges and achieve more efficient and responsive supply chain analytics.

2.4 Related Theory Used

This section establishes the theoretical foundation for the proposed end-to-end data

pipeline. The architecture is built upon three core pillars which are the Medallion Architecture for structured data processing, the Star Schema for analytical modeling, and Data Quality frameworks to ensure the reliability of the Recoverable Assets (RA) and Magna Carta (MC) business logic. These theories provide the conceptual framework necessary to transform raw synthetic operational data into actionable inventory insights.

2.4.1 Medallion Architecture

The Medallion Architecture is a data design pattern used to logically organize data in a Lakehouse, aiming to incrementally improve the structure and quality of data as it flows through each layer (Armbrust et al., 2021). In this study, architecture is implemented through a three-layer approach starting with the Bronze layer. This layer acts as the landing zone for the source data, emphasizing “source of truth” preservation where data is ingested in its raw format. This ensures that data lineage is maintained, allowing for reprocessing if the business logic for inventory classification changes. The data then progresses to the Silver layer, where it is cleansed, integrated, and transformed according to defined business rules. It is within this tier that complex calculations, such as consumption averaged and stock status flagging, are applied. Finally. The data reaches the Gold layer, which hosts the curated, analytics-ready data. Theoretically, Gold tables are highly structured and aggregated to answer specific business questions regarding financial provisions and stockout risks. Figure 2.1 illustrates the flow of data through these progressive layers of refinement.



Figure 2.1 Medallion Architecture Data Layer

2.4.2 Star Schema

To support high-performance reporting and dashboarding requirements, the Star Schema is a central concept in the Kimball Dimensional Modelling methodology is employed. According to Kimball and Ross (2013), a Star Schema consists of a central fact table joined to several dimension tables, creating a star-like shape. In the context of this project, the central fact table stores quantitative metrics and key performance indicators (KPIs) related to inventory status. This is supported by dimension tables that provide descriptive context such as material types, supplier details, and temporal data. This theoretical model is chosen because it minimizes the complexity of join operations, which significantly enhances query performance in visualization tools when calculating financial totals or identifying material shortages. By separating numerical measurements from descriptive attributes, the star schema provides a flexible and intuitive structure for end-users to navigate complex datasets, as shown in the structural representation in Figure 2.2.

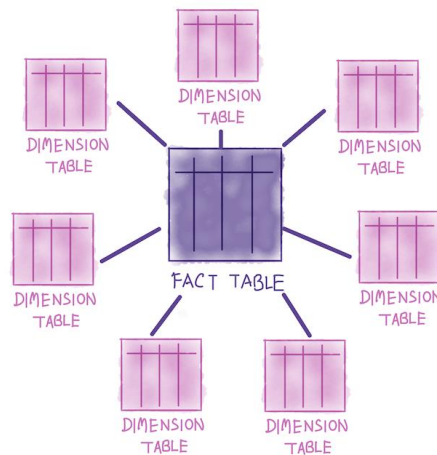


Figure 2.2 *Star Schema Dimensional Model*

2.5 Visualization

For this project, data visualization serves as the final output layer of the pipeline by translating the processed inventory risk results into a format that PPG's inventory managers, procurement teams, and financial controllers can directly act on without

having to query the underlying data themselves. Allen et al. (2021) found that high-dimensional, real-time visualization dashboards in manufacturing environments enable complex operational data to be interpreted at a fraction of the cost of full-scale digitalization, which is relevant here because PPG's inventory risk involves multiple simultaneous conditions which are overstocked materials, expired stock, and downstream customer order impact that cannot be communicated through a single static report.

Microsoft Power BI was selected as the visualization tool for this project as it connects naturally to Azure Synapse Analytics, which is the serving layer used in the pipeline. This natural connection means the dashboard retrieves its data directly from the curated SQL views registered in Synapse without needing any additional data movement or manual export steps, keeping the entire pipeline within the Microsoft Azure platform.

The dashboard includes eight pages covering one home overview and seven dedicated pages each answering one of the pre-defined business questions. Zimmermann and Brandtner (2024) stated that analytics-driven dashboards in supply chain settings improved decision quality among non-technical managers by presenting inventory metrics in self-service views, which directly applies to this project since PPG's intended users are business users who require clear visual outputs compared to the raw data. Each dashboard page maps to a specific Curated layer output, from the `mc_provision_summary` view for the total financial provision question to the `sales_order_impact` view for the affected customer orders question, ensuring that every business question defined in the project brief has a corresponding visual that draws from a validated pipeline output.

2.6 Comparison of Visualization Tools

This project uses four crucial technologies across the pipeline which are Azure Data Factory for data ingestion, Azure Data Lake Storage Gen2 for data storage, Azure Databricks for data transformation, and Azure Synapse Analytics for data warehouse. Each tool was picked over a commonly used alternative based on how well it fits within the Microsoft Azure environment that the project runs on. Monha et al. (2022) stated

in their systematic review of 106 pipeline studies that cloud-base tools like Azure Data Factory were the most commonly used for orchestration and ingestion tasks, highly due to their native integration with other cloud services, built-in monitoring, and reduced infrastructure overhead.

For ingestion, Azure Data Factory was chosen over Apache Airflow as Airflow is a code-driven tool that provides more flexibility but needs more manual server setup and infrastructure management, which leads to unnecessary complexity for a project with fixed ingestion requirements. For transformation, Azure Databricks was chosen over standalone Apache Spark. This is because both use the same Spark engine, but Databricks provides a well-managed workspace with notebook collaboration and direct integration with ADLS Gen2, removing the need to configure and manage Spark infrastructure separately. For the data warehouse, Azure Synapse Analytics was chosen over Google BigQuery and Snowflake as BigQuery is built for the Google Cloud ecosystem meanwhile Snowflake operates independently across multiple clouds, meaning both would require additional connectors to communicate with Azure services involved in this project. Synapse connects natively with ADF, ADLS Gen2, Databricks, and Power BI within the same Azure environment, keeping the entire pipeline on a single platform without extra configuration.

2.7 Microsoft Azure

This section reviews the five key technologies used to build the data pipeline in this project. Each technology handles a specific role in the process, from storing raw data to displaying the final results. Together, they form a connected system that automates the full inventory risk management workflow.

2.7.1 Microsoft Azure Data Lake Storage Gen2 (ADLS Gen2)

Azure Data Lake Storage Gen2 (ADLS Gen2) is a cloud storage service that can hold large amounts of data in an organised folder structure. In this project, ADLS Gen2 is used to store all three layers of data in the pipeline, which are the Bronze, Silver, and Gold containers. Each container holds a different version of the data,

starting from the original raw CSV files all the way to the cleaned and processed output. Microsoft (2024) notes that ADLS Gen2 is designed to work well with other Azure tools, which makes it a reliable foundation for the entire pipeline.

2.7.2 Azure Data Factory (ADF)

As shown in Figure 2.3, Azure Data Factory (ADF) is an orchestration service that automatically moves and manages data between different systems. In this project, ADF is responsible for picking up the six raw CSV files and loading them into the Bronze layer, as well as triggering the transformation steps that follow. It also monitors the pipeline to detect any failures during the run. As shown Keerthana and Nivetha (2025) found that using ADF in cloud pipelines reduces the need for manual data handling and improves the overall reliability of the process.

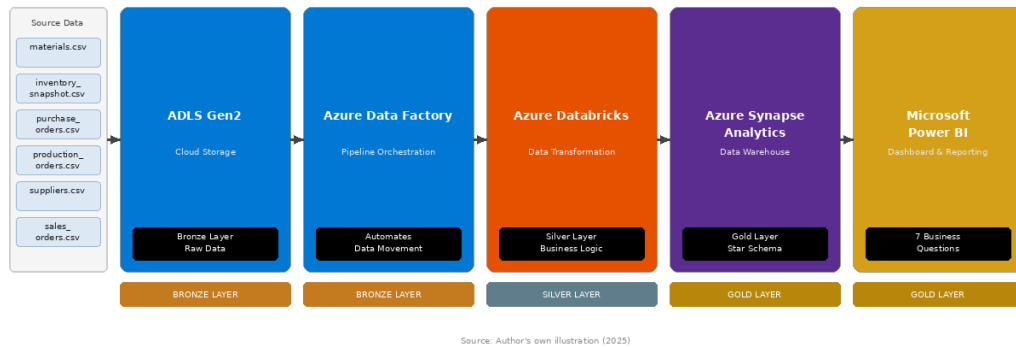


Figure 2.3 Azure End-to-End Data Pipeline Architecture

2.7.3 Azure Databricks

Azure Databricks is a data processing platform that allows engineers to write code and apply business logic to large datasets. In this project, Databricks is used in the Silver layer to carry out all the main transformations, such as calculating twelve-month consumption, flagging Recoverable Assets, identifying Magna Carta Dead Stock and Expired materials, computing the financial provision amount, and detecting stockout risks. According to Armbrust et al. (2021), Databricks is well suited for this kind of rule-based processing because it can handle the full dataset efficiently in a single run.

2.7.4 Azure Synapse Analytics

Azure Synapse Analytics is a data warehouse service that stores structured data and supports fast querying for reporting purposes. In this project, Synapse is used as the Gold layer where the cleaned and transformed data is loaded into a star schema, with a central fact table surrounded by dimension tables for materials, suppliers, time, and sales orders. This structure makes it easy for Power BI to retrieve the data quickly. Microsoft (2024) describes Synapse as a platform that brings together data storage and analysis in one place, which simplifies the connection between the pipeline and the dashboard.

2.8 Microsoft Power BI

Microsoft Power BI is a business intelligence tool that connects to data sources and displays the results as interactive visual dashboards. In this project, Power BI is the final output of the pipeline, connecting directly to Azure Synapse Analytics to read the Gold layer data and present the answers to the seven inventory business questions. These include which materials are overstocked, which qualify as Magna Carta, what the total financial provision is, and which customer orders are at risk. Tunpita and Kirimasthong (2024) showed that Power BI is effective for turning processed pipeline data into clear and easy-to-use dashboards for decision makers. Figure 2.4 shows an illustrative layout of the proposed dashboard.

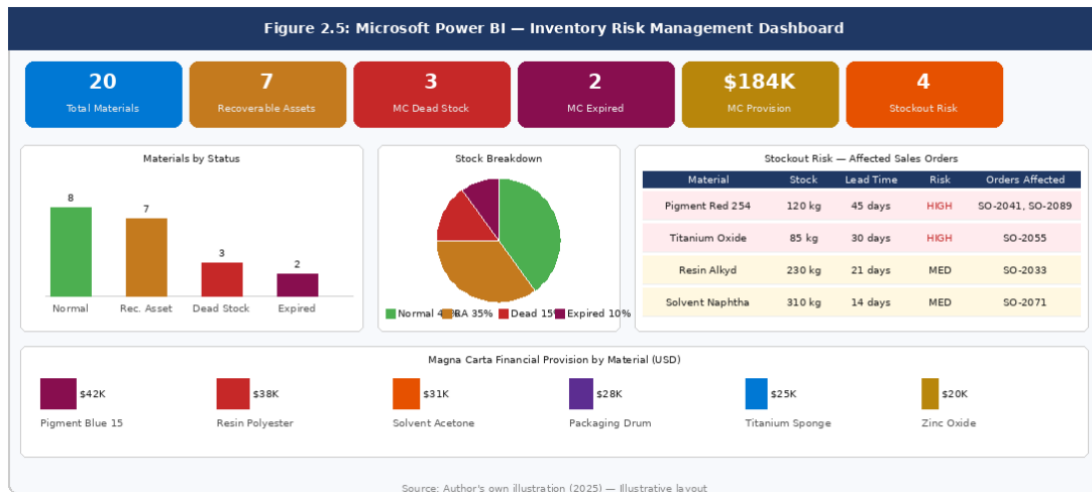


Figure 2.4 Microsoft Power BI — Inventory Risk Management Dashboard Layout

2.9 Chapter Summary

Chapter 2 has reviewed the literature and theoretical foundations that support this project. The first part examined related previous studies on cloud-based data pipelines, Power BI dashboard development, and inventory obsolescence classification. These studies show that cloud pipelines are now the standard approach for handling large volumes of operational data, and that classification frameworks such as Recoverable Assets and Magna Carta are well established in both industry and research. The second part covered three core theories used in this project. The Medallion Architecture provides the structure for the three-layer pipeline. The Star Schema organizes the Gold layer data for efficient reporting. The Data Quality framework ensures that the data is complete, valid, and consistent before it reaches the dashboard. The third part reviewed the five Azure technologies selected for this project, which are ADLS Gen2 for storage, Azure Data Factory for orchestration, Azure Databricks for transformation, Azure Synapse Analytics for data warehousing, and Microsoft Power BI for visualization. Together, the literature, theories, and technologies reviewed in this chapter provide a solid foundation for the system design and implementation described in the following chapters.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter outlines the methodology and technical framework adopted in the development of the inventory risk management system. It begins by justifying the selection of the Waterfall System Development Life Cycle (SDLC) as the project's development approach, followed by a detailed explanation of each phase within the methodology. Subsequently, the chapter describes the suite of cloud-based technologies employed throughout the pipeline, encompassing Microsoft Azure services and Microsoft Power BI, which collectively support the end-to-end data engineering solution.

The methodology and technology stack presented in this chapter were carefully selected to align with the structured and well-defined nature of the project's requirements. Given that the project operates within clearly established parameters, including predefined source data, fixed business logic, and a specified output schema, a disciplined and systematic development approach was deemed essential. The decisions outlined in this chapter serve as the foundation upon which the system's design, implementation, and validation are built in the subsequent chapters.

3.2 Proposed Methodology

The Waterfall System Development Life Cycle (SDLC) model was selected as the development approach for this project. The Waterfall methodology is suitable for this project as all requirements, business classification rules, source datasets, and expected outputs were all defined and agreed upon with PPG before the development work began. Mishra and Alzoubi (2023) found that Waterfall is more suitable than Agile for projects with clearly predefined requirements and stable scope, which every phase can be fully verified before the next begins, which directly matches the conditions of this project

since the six source files, the Recoverable Assets and Magna Carta business logic, and the seven dashboard business questions were all fixed from the very beginning. Saravanos and Curinga (2023) demonstrated through simulation that the Waterfall model's phase-by-phase structure supports accurate estimation and resource planning for development projects, which was crucial for this project given that six team members each had to coordinate specific pipeline layer responsibilities without overlap or rework.

The pipeline architecture of this project follows a strict sequential dependency that proves the Waterfall phases directly. The six source files need to be uploaded and confirmed in the Bronze layer before the Silver cleaning notebook can run. The Silver cleaned Parquet outputs must be validated for row counts and null values before the Gold business transformation notebook can apply the Recoverable Assets flag, Magna Carta classification, stockout risk detection, sales order impact tracing, and production fulfilment checking. The Gold outputs must all be confirmed present in the Gold container before the Curated notebook can build the star schema tables and write them to the Curated container. Finally, the Curated Parquet files must be verified in ADLS Gen2 before the SQL views are registered in Azure Synapse Analytics and connected to Power BI. This layer-by-layer dependency means that errors in an earlier stage would propagate to all subsequent stages, which is Waterfall's requirement to validate each phase before proceeding with the most appropriate choice for this pipeline. The methodology applied in this project follows five primary phases which are requirement analysis, system design, implementation, testing, and deployment.

3.3 Phases of Chosen Methodology

The Waterfall methodology used in this project is based on four major steps: requirement, design, development and testing. The steps are implemented one after another in order to have a clear and systematic development process.

3.3.1 Requirement

During the requirement stage, the general direction, objective and scope of the project are identified. This involves discussions with the industry partner to understand the inventory management challenges faced by the company and the expected outcomes.

The available datasets, such as inventory records, material information, sales orders and production data are examined to understand their structure and content. Business rules such as Recoverable Assets (RA), Magna Carta (MC), stockout risk and inventory provision calculations are also studied to ensure that the solution meets the actual business requirements.

3.3.2 Design

During this phase, a general system design is ready. The data pipeline architecture is created on the basis of the Bronze, Silver, Gold and Curated layers. The data warehouse design (Star schema and fact tables) is also to be designed. Moreover, the Power BI dashboard layout and its main elements are created in a way that will allow it to display the insights necessary.

3.3.3 Development

During this phase, a general system design is ready. The data pipeline architecture is created on the basis of the Bronze, Silver, Gold and Curated layers. The data warehouse design (Star schema and fact tables) is also to be designed. Moreover, the Power BI dashboard layout and its main elements are created in a way that will allow it to display the insights necessary. The Bronze layer serves as the raw data storage layer. This is where the source files provided by the industry partner are ingested into the system without modification. This preserves the original data for traceability and future reference. The Silver layer performs data cleaning, validation and standardization. To ensure that there is no missing values, inconsistent formats and data quality issues for further processing. The Gold layer contains business-ready datasets generated through transformation and calculations. Business logic are implemented in this layer. Finally, the Curated layer organizes the transformed data into fact and dimension tables optimized for reporting and visualization. These curated datasets are connected to Power BI later. Get the interactive dashboards and KPI reports from Power BI to answer the required business questions and support decision-making.

3.3.4 Testing

Testing is done to make sure that the system is operational and that the results are

correct. Data at each layer of the pipeline is verified to confirm that transformations and calculations are performed as intended. The implemented business rules are also validated as the expected output mentioned in the requirements. Finally, the Power BI dashboard is tested to ensure that all visualizations and KPI metrics display the correct information and provide meaningful insights to users.

3.4 Project Planning Schedule

Figure 3.1 illustrates a structured project schedule organized into six distinct phases: planning, requirements, analysis, design, development and testing. The timeline spans from March 16 to June 19, tracking individual and collaborative tasks assigned across six team members to manage workload distribution. Initial phases focus on group planning, literature reviews and an on-site visit which transition into a concurrent two-week system design phase. This is followed by a sequential week-by-week development of data pipeline layers (Bronze, Silver, Gold, and Curated) and a dashboard, before concluding with collective pre-demo and final project deployment activities.



Figure 3.1 [Gantt Chart](#)

3.5 Technology Used Description

In this project, a data engineering environment built with Microsoft Azure will be used to create an end-to-end inventory risk management pipeline. The technology stack was chosen to enable batch data processing, layered data storage, business rule transformation, dimensional modelling, and dashboard visualization. The main

technologies used in this project are source Excel/CSV files, Azure Data Lake Storage Gen2, Azure Databricks, Azure Synapse Analytics Serverless SQL Pool, and Microsoft Power BI.

The pipeline is structured in a four layers medallion style: Bronze, Silver, Gold and Curated. The Bronze layer contains the original files that have been uploaded but not modified. Silver layer is responsible for data cleaning and standardization. The Gold layer is used for business rules for Recoverable Assets, Magna Carta classification, stockout risk detection, production fulfilment checking, and customer order impact tracing. The Curated layer re-architects the Gold outputs into a Galaxy Schema and makes the final analytical tables available for reporting in Azure Synapse Analytics for Power BI.

This project is not an on-premises SQL database or Azure Data Factory ingestion. Rather, the source files are uploaded manually to Azure Data Lake Storage Gen2. This method is appropriate for the project's scope as the datasets are static, batch-oriented, and primarily for academic prototype development.

Figure 3.2 illustrates the overall Azure-based batch data pipeline, where manually uploaded Excel/CSV source files are stored in Azure Data Lake Storage Gen2, transformed using Azure Databricks, exposed through Azure Synapse Analytics, and visualized in Power BI.

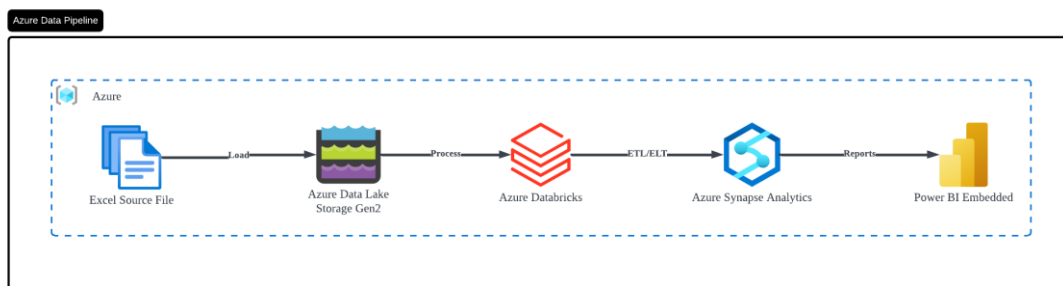


Figure 3.2 *Azure-Based End-to-End Data Pipeline Architecture*

3.5.1 Source Excel/CSV Files

This project uses the following source data, which is available in Excel/CSV format:

materials, inventory snapshot, purchase orders, production orders, suppliers, and sales orders. These datasets are used for the operational data required to analyze inventory risk, material usage, supplier replenishment, production fulfillment, and customer order impact. The files are manually loaded to the Bronze layer in Azure Data Lake Storage Gen2, which is the source of the data pipeline.

3.5.2 Azure Data Lake Storage Gen2

All layers of the pipeline are stored in Azure Data Lake Storage Gen2. The Bronze layer contains the original source files that are uploaded without any transformation, the Silver layer contains the cleaned and standardized datasets, the Gold layer contains the output datasets that are business ready (e.g. inventory status, stockout risk, production fulfillment, sales order impact), and the Curated layer contains the final set of tables in Galaxy Schema that are used for reporting. This stacked storage model helps traceability, makes the processing steps well known and enables data re-processing if necessary.

3.5.3 Azure Databricks

Azure Databricks is used as the main processing and transformation environment for the project. Databricks cleans the raw files in the Silver layer by deduplicating, filling in missing data, correcting data types, and normalizing string formats. Databricks leverages business rules for Recoverable Assets, Magna Carta classification, stockout risk detection, production fulfillment checking, and customer order impact tracing in the Gold layer. Databricks transforms the Gold outputs into a Galaxy Schema in the Curated layer, which consists of multiple fact tables and shared dimension tables.

3.5.4 Azure Synapse Analytics Serverless SQL Pool

The Curated data is served from Azure Synapse Analytics Serverless SQL Pool. The curated fact and dimension tables are stored as Parquet files in Azure Data Lake Storage Gen2 and SQL views are created over these files using Synapse. This means that Power BI can use the final analytics-ready data using structured SQL views without having to deploy a separate SQL warehouse.

3.5.5 Microsoft Power BI

The final dashboard is created using Microsoft Power BI. It is linked to the curated views that are exposed via Azure Synapse Analytics and loads the necessary fact and dimension tables into the Power BI model. The dashboard provides answers to 7 business questions regarding materials below reorder level, Recoverable Assets, Magna Carta Dead Stock, Magna Carta Expired materials, total provision amount, unfulfilled production orders and affected customer sales orders.

3.5.6 Integration of Technologies

The pipeline starts with six manually uploaded Excel/CSV source files in the Bronze layer of Azure Data Lake Storage Gen2. Azure Databricks then cleans the raw files, applies business rules and constructs the Galaxy Schema through the Silver, Gold, and Curated layers. Azure Synapse Analytics Serverless SQL Pool exposes the Curated tables as SQL views and Power BI imports these views to create the final interactive inventory risk management dashboard.

3.6 Summary

In conclusion, this chapter explained the step-by-step plan and the tools used to build this project. The project followed the Waterfall method which means working through fixed stages one after the other including from planning to final building. For the actual setup, the system uses Microsoft Azure cloud tools to handle raw Excel and CSV files that are uploaded by hand. These files are saved in Azure Data Lake Storage Gen2 (ADLS Gen2) and cleaned using Azure Databricks across four different steps: Bronze, Silver, Gold and Curated layers. Finally, this cleaned data is shared through Azure Synapse serverless SQL views and turned into easy-to-read charts in Power BI to help the company track inventory risks.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the analysis and design of the end-to-end data engineering pipeline for the Recoverable Assets (RA) and Inventory Risk Management (IRM) project. It covers the system architecture, including the full pipeline from data source to visualization, and explains each component's role within the Medallion Architecture. The chapter also addresses the requirement analysis, the star schema database design, and the Power BI dashboard interface design.

4.2 System Architecture

This section describes the full architecture of the end-to-end data pipeline. The architecture follows the Medallion Architecture pattern with three layers — Bronze, Silver, and Gold — and Use Microsoft Azure services to move, process, store, and visualize the data.

4.2.1 Architecture Overview

Figure 4.1 shows the complete pipeline architecture. Data flows from left to right, starting from the on-premises SQL database and ending at the Power BI dashboard. The pipeline contains six main stages and two supporting security components.

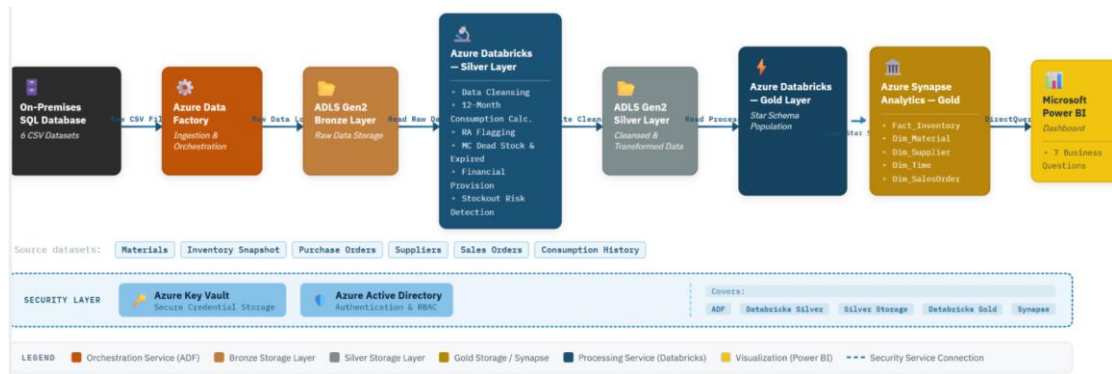


Figure 4.1 End-to-End Data Pipeline Architecture Diagram

The pipeline follows a left-to-right flow with clear separation between each stage. On the far left sits the On-Premises SQL Database, which holds the six raw CSV datasets. An arrow points right to the Azure Data Factory (ADF) box, which handles data ingestion and orchestration. ADF moves the raw data into the Bronze container inside Azure Data Lake Storage Gen2 (ADLS Gen2), shown as the first of three stacked containers on the right side of ADF.

From the Bronze layer, data flows down into the first Azure Databricks box, which represents the Silver layer processing. Inside this box, a vertical list shows the transformation operations: data cleansing, 12-month consumption calculation, Recoverable Assets flagging, Magna Carta Dead Stock classification, Expired material identification, financial provision computation, and stockout risk detection. After processing, Databricks writes the cleaned data back to the Silver container in ADLS Gen2.

The Silver data then flows into a second Azure Databricks box for Gold layer processing. This box shows the star schema population logic, where Databricks transforms the Silver data into fact and dimension tables. The output loads into Azure Synapse Analytics, labeled as the Gold layer, which contains the star schema with the Fact_Inventory table and four dimension tables (Dim_Material, Dim_Supplier, Dim_Time, Dim_SalesOrder).

An arrow connects Synapse Analytics to the Microsoft Power BI box on the far right. Power BI reads the Gold layer data and presents seven dashboard views that answer the key business questions.

Below the entire pipeline, a horizontal security bar spans the full width. This bar contains two boxes side by side: Azure Key Vault on the left (for credential management) and Azure Active Directory on the right (for authentication and role-based access control). Dotted lines connect the security bar upward to each major component in the pipeline, indicating that every stage uses these security services.

The overall layout uses a horizontal flow with a white or light-gray background. Each Azure service box uses a distinct color or icon to represent its role (for example: blue for storage, orange for processing, green for visualization). The three ADLS Gen2 containers (Bronze, Silver, Gold) use progressively darker shades to show the refinement stages. Arrows between components use solid lines for data flow and dotted lines for control or security flow.

The following subsections explain each component in detail.

4.2.2 On-Premises SQL Database (Data Source)

The pipeline starts with an on-premises SQL database that stores PPG's operational data shown in Table 4.1. This database contains six datasets in CSV format.

Table 4.1: Datasets

Dataset	Description
Materials	Master data for all raw materials used in production, including material code, description, material type, unit of measure, and shelf life.
Inventory Snapshot	Current stock on hand for each material, storage locations, batch numbers, and expiry dates at the time of extraction.
Purchase Orders	Orders placed to suppliers for material replenishment, including order dates, expected delivery dates, and order quantities.
Suppliers	Supplier details including supplier code, supplier name, average lead time, and delivery terms.
Sales Orders	Customer orders that depend on material availability, including order number, customer name, material requested, order quantity,

	and required delivery date.
Consumption History	Historical consumption records for each material, used to calculate 12-month average consumption and detect inactive materials.

These files represent the real operational state of PPG's inventory system. The raw data may contain missing values, duplicate records, date format inconsistencies, and other quality issues. The pipeline addresses these issues starting from the Silver layer.

4.2.3 Azure Data Factory (ADF)

Azure Data Factory handles the ingestion stage of the pipeline. ADF performs three main tasks:

I. Data Extraction. ADF connects to the on-premises SQL database using a self-hosted integration runtime and extracts the six CSV datasets.

II. Data Loading. ADF loads the extracted files into the Bronze container in ADLS Gen2 without applying any transformation. This preserves the original data as-is and ensures the Bronze layer acts as a faithful copy of the source system.

III. Pipeline Orchestration. ADF manages the scheduling, monitoring, and error handling of each pipeline run. It triggers the downstream Databricks notebooks after the Bronze ingestion completes successfully. If any step fails, ADF captures the error and stops the pipeline to prevent partial data from moving to the next layer.

By using ADF, the team removes the need for manual data transfers and ensures the pipeline runs consistently with every execution.

4.2.4 Azure Data Lake Storage Gen2 — Bronze Layer

ADLS Gen2 serves as the central storage system for the entire pipeline. The Bronze layer is the first of three containers within ADLS Gen2.

The Bronze layer stores the raw CSV files exactly as ADF delivers them from the source system. The team does not apply any cleaning, formatting, or transformation at this stage.

This approach serves two purposes:

I. Source of Truth. The Bronze layer preserves the original data so the team can always trace back to the raw state for auditing and debugging.

II. Reprocessing. If business rules change in the future, the team can re-run the Silver and Gold transformations starting from the Bronze layer without going back to the source system.

Each CSV file lands in the Bronze container as a separate file, organized by dataset name and ingestion date. The file format stays as CSV at this layer to match the source format.

4.2.5 Azure Databricks — Silver Layer

Azure Databricks reads the Bronze data and performs all cleansing and business logic transformations to produce the Silver layer. This is the most logic-intensive stage of the pipeline.

Data Cleansing Operations:

- Remove duplicate records across all datasets.
- Handle missing values using defined default rules (for example, fill missing numeric values with zero or flag them for review).
- Standardize data types (for example, convert string dates to proper date format).
- Validate that numerical values fall within logical bounds (for example, stock quantities must be non-negative).
- Ensure consistent naming conventions and field formats across all six datasets.

Business Rule Applications:

The Silver layer applies PPG's two main inventory classification frameworks:

Recoverable Assets (RA) Classification:

- Calculate the 12-month average consumption for each material using the

consumption history dataset.

- Compare current stock on hand against the 12-month consumption figure
- Flag materials where stock on hand exceeds 12 months of consumption as Recoverable Assets.

Magna Carta (MC) Classification:

- Identify materials with no consumption in the past 9 months and classify them as Dead Stock
- Identify materials past their expiry date and classify them as Expired
- Calculate the financial provision amount for each classified material based on IAS 2 inventory write-down requirements.

Stockout Risk Detection:

- Compare current stock levels against the defined reorder point for each material
- Flag materials where stock falls below the reorder threshold as stockout risk
- Link flagged materials to affected customer sales orders to show which orders face potential delays

Databricks writes the processed output back to ADLS Gen2 in the Silver container in Parquet format for better query performance. The team uses notebook-based transformations, which allow each business rule to be tested and validated independently.

4.2.6 Azure Synapse Analytics — Gold Layer

Azure Synapse Analytics hosts the Gold layer, which stores the final analytics-ready data in a star schema design. Databricks reads the Silver data, transforms it into the star schema structure, and loads the result into Synapse.

The star schema contains one central fact table and four dimension tables:

Fact Table:

- Fact Inventory: Stores quantitative measures including stock on hand quantity, 12-month consumption total, provision amount, stockout flag, RA classification code, MC classification code, and expiry status for each material at each snapshot date

Dimension Tables:

- Dim Material: Material attributes such as material code, description, material type, unit of measure, and shelf life.
- Dim Supplier: Supplier details including supplier code, supplier name, average lead time, and delivery performance rating.
- Dim Time: Time attributes such as year, quarter, month, and date for temporal filtering and trend analysis.
- Dim SalesOrder: Sales order details including order number, customer name, material code, order quantity, required delivery date, and order status.

This star schema design minimizes the complexity of join operations in Power BI and supports fast query performance for dashboard calculations and drill-down analysis.

4.2.7 Microsoft Power BI — Visualization Layer

Power BI is the final output of the pipeline. It connects directly to Azure Synapse Analytics using DirectQuery and reads the Gold layer data. The dashboard presents interactive visuals that help inventory managers, procurement teams, and financial controllers answer seven key business questions as shown in Table 4.2.

Table 4.2: Business Questions

Dashboard View	Business Question
Overstocked Materials	Which materials have stock on hand exceeding 12 months of consumption (Recoverable Assets)?
Magna Carta Dead Stock	Which materials show no consumption for more than 9 months?

Expired Materials	Which materials have passed their expiry date?
Financial Provision	What is the total write-down amount required under IAS 2?
Stockout Risk	Which materials are at risk of stockout because stock falls below the reorder point?
Affected Customer Orders	Which customer sales orders face delays due to material shortages?
Risk Summary	What is the overall inventory risk picture across all classifications?

Users can filter by material type, supplier, time period, and risk category to explore specific conditions. The dashboard uses KPI cards, bar charts, tables, and drill-through pages to present the information in a clear and actionable format. Figure 2.4 in Chapter 2 shows the illustrative layout of the proposed dashboard.

4.2.8 Security Components

Two Azure services provide security across the entire pipeline:

Azure Key Vault stores all sensitive credentials, including connection strings, database passwords, and access keys. ADF, Databricks, and Synapse retrieve these credentials from Key Vault at runtime instead of hard-coding them into pipeline configurations. This centralizes credential management and reduces the risk of exposed secrets.

Azure Active Directory (Azure AD) manages authentication and access control for all Azure services in the pipeline. It enforces role-based access control (RBAC) so that only authorized users and services can access specific resources such as Data Lake containers, Databricks workspaces, Synapse pools, and Power BI reports.

4.2.9 Technology Integration Summary

Table 4.3 summarizes the role of each technology in the pipeline.

Table 4.3: Technology Stack Summary

Technology	Role	Pipeline Stage
On-Premises SQL Database	Data source storing six operational CSV datasets	Source
Azure Data Factory (ADF)	Data ingestion, workflow orchestration, scheduling, and monitoring	Ingestion
Azure Data Lake Storage Gen2 (ADLS Gen2)	Central storage for Bronze, Silver, and Gold layers	Bronze, Silver, Gold
Azure Databricks	Data cleansing, business rule transformation, and star schema preparation	Silver, Gold
Azure Synapse Analytics	Data warehouse hosting the star schema for fast analytical queries	Gold
Microsoft Power BI	Interactive dashboard and visualization for end users	Visualization
Azure Key Vault	Secure storage and retrieval of credentials and secrets	Security
Azure Active Directory (Azure AD)	Authentication, authorization, and role-based access control	Security

All technologies operate within the Microsoft Azure ecosystem, which ensures native integration between services and eliminates the need for third-party connectors. The team chose this unified approach to reduce configuration complexity and maintain a single-platform deployment.

4.3 Requirement Analysis

This section lists what the system must do (functional requirements) and how well it must do (non-functional requirements) for the PPG Recoverable Assets and Inventory Risk Management pipeline.

4.3.1 Functional Requirements

Table 4.4 shows the functional requirements which describe the specific tasks the system must be able to perform.

Table 4.4: Functional Requirement

ID	Requirement	Description
FR-01	Ingest 6 CSV Files	The system must load all six CSV files (materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, sales_orders.csv) into the Bronze layer of Azure Data Lake. Basic checks for missing values, wrong data types, and duplicate records must also be applied.
FR-02	Apply Recoverable Asset (RA) Flag	The system must calculate how much of each material was consumed in the past 12 months. If the current stock is more than that amount, the material must be flagged as a Recoverable Asset (RA).
FR-03	Apply Magna Carta (MC) Dead Stock Flag	The system must flag a material as Magna Carta Dead Stock if it has had zero consumption for 9 or more months in a row and has no expiry date.
FR-04	Apply Magna Carta (MC) Expired Flag	The system must flag a material as Magna Carta Expired if its expiry date recorded in the inventory snapshot has already passed.
FR-05	Calculate MC Provision Amount	The system must calculate the total financial provision for all MC-flagged materials. The provision is 100% of the material's current stock value. The total must be available in the dashboard.
FR-06	Detect Stockout Risk	The system must identify materials that are running too low to cover demand during the supplier's lead time. These materials must be flagged at risk of stockout.
FR-07	Trace Affected	For each material at stockout risk, the system must

	Customer Orders	find all open customer sales orders that need that material. The output must show the order ID, customer name, quantity needed and the shortfall amount.
FR-08	Load Data into Star Schema (Gold Layer)	The system must load all processed data into a star schema in Azure Synapse Analytics. The main fact table (fact_inventory_status) must store RA/MC flags, provision values and stockout flags per material per month.
FR-09	Build Power BI Dashboard	The system must provide a Power BI dashboard that answers all seven business questions (Q1–Q7), including materials below reorder level, RA and MC materials, total provision amount and customer orders at risk.

4.3.2 Non-functional Requirements

Figure 4.5 shows the non-functional requirements which describe the quality standards the system must meet in terms of speed, growth, stability and safety.

Table 4.5: Non-functional Requirements

ID	Category	Description
NFR-01	Performance	The full pipeline from CSV ingestion to the Gold layer must finish within a reasonable time for monthly reporting. The Power BI dashboard must load quickly so users are not kept waiting.
NFR-02	Scalability	The pipeline must be able to handle more data in the future such as more materials or more monthly records without needing to be rebuilt. Adding new CSV source files should only require a configuration change not a code to rewrite.
NFR-03	Reliability	If a pipeline step fails, it must retry automatically before raising an error. Re-running a failed pipeline

		must not create duplicate records. All pipeline runs must be logged so the team can check what happened and when.
NFR-04	Security	All data stored in Azure must be encrypted. All data moving between services must use a secure connection. Access to Azure resources must follow the least privileged principle which means each person or service only gets the access they need. No passwords or keys should be written directly in the code. They must be stored in Azure Key Vault.

4.4 Project Design

This section presents the design artefacts developed for the Recoverable Assets and Inventory Risk Management system for PPG. Three artefacts are included which are a use case daigram showing how the three PPG user groups interact with the seven business question views, an activity diagram showing the data flow from the six source CSV files through the Bronze, Silver, Gold, and Curated layers to the Power BI dashboard, and a sequence diagram showing the interaction between system components during a pipeline run.

4.4.1 Use Case Diagram

Figure 4.2 shows the dashboard use case diagram for the Recoverable Assets and Inventory Risk Management system. Rather than showing internal system processes, the diagram focuses on how the three main user groups at PPG interact with the Power BI dashboard to access the inventory risk outputs produced by the pipeline. The three actors are the Inventory Manager, Financial Controller, and Procurement Team, each of whom accesses specific dashboard views based on their operational responsibility within PPG.

The Inventory Manager is the primary user of the system and is responsible for monitoring the overall health of PPG's raw material stock across all twenty materials. They have access to Q1 through Q5, covering materials below reorder level, Recoverable Assets, Magna Carta Dead Stock, Magna Carta Expired Materials, and the total MC provision amount. The Financial Controller focuses on the financial reporting

obligations of the inventory risk outcomes, specifically the Magna Carta classifications and total provision amount in Q5, as these outputs directly affect PPG’s balance sheet reporting under IAS 2. The Procurement Team uses the system to manage supply chain risk, checking reorder alerts in Q1, identifying which production orders could not be fulfilled in Q6 because of insufficient raw material stock, and determining which open customer sales orders are at risk of delay in Q7.

Each of the seven use cases maps directly to one of the seven business questions defined in the project brief, ensuring every pipeline output has a specific business user who depends on it for decision-making within PPG’s inventory operations.

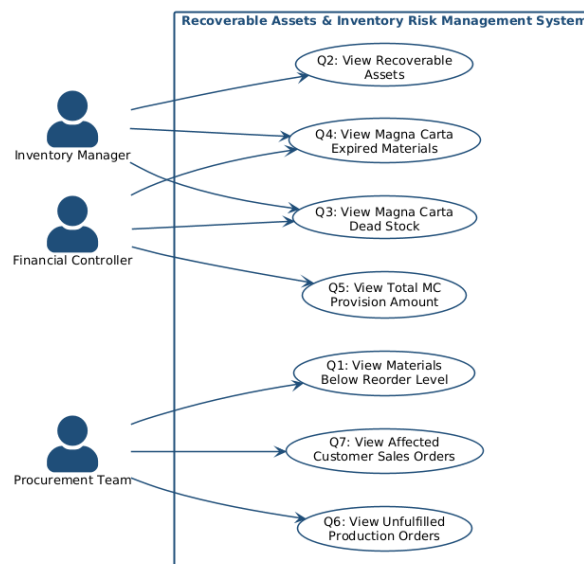


Figure 4.2 Dashboard Use Case Diagram: Recoverable Assets & Inventory Risk Management System

4.4.1.1 Use Case Description

The following tables describe each use case in detail. Each use case corresponds to one of the seven business questions answered by the Power BI dashboard. For all use cases, the precondition is that the full pipeline has been executed, all Curated layer SQL views are registered in Azure Synapse Analytics curated_db, and the Power BI dataset has been refreshed.

4.4.1.1.1 <<UC-01: View Materials Below Reorder Level>>

Table 4.6 shows The Inventory Manager and Procurement Team use this view

to identify which of PPG’s twenty raw materials have fallen below their defined reorder point, enabling immediate replenishment decisions before production is disrupted. The Curated Synapse views must be registered, and the Power BI dataset must be refreshed before this use case can be accessed.

Table 4.6: Use Case Description for View Materials Below Reorder Level

Field	Description
Use Case ID	UC-01
Use Case Name	View Materials Below Reorder Level
Actor	Inventory Manager, Procurement Team
Description	The actor views a list of raw materials whose current stock level has fallen below the defined reorder point, indicating an immediate replenishment need.
Pre-condition	Gold layer data has been successfully loaded into Azure Synapse Analytics and Power BI is connected.
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q1 view. 3. System displays all materials where current stock is below the reorder point, showing material name, category, current stock, and reorder level.
Post-condition	Actor identifies materials requiring urgent replenishment of action.
Exception	If no materials are below the reorder level, the view displays an empty table with a “No materials at risk” message.

4.4.1.1.2 <<UC-02: View Recoverable Assets>>

Table 4.7 shows The Inventory Manager uses this view to identify PPG materials where the current stock on hand exceeds twelve months of total consumption,

flagging them as Recoverable Assets that carry a surplus risk. The `is_recoverable_asset` flag must be generated in the Gold layer and exposed through the Curated layer before the dashboard view is available.

Table 4.7: Use Case Description for View Recoverable Assets

Field	Description
Use Case ID	UC-02
Use Case Name	View Recoverable Assets
Actor	Inventory Manager
Description	The actor views a list of raw materials classified as Recoverable Assets, where the quantity on hand exceeds twelve months of total consumption.
Pre-condition	Silver layer RA flag logic has been applied and results are loaded into the Gold layer.
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q2 view. 3. System displays all materials where <code>RA_Flag = TRUE</code>, showing material name, stock quantity, 12-month consumption, and excess quantity.
Post-condition	Actor identifies surplus materials for management review and potential recovery action.
Exception	If no materials qualify as Recoverable Assets, the view displays an empty table.

4.4.1.1.3 <<UC-03: View Magna Carta Dead Stock>>

Table 4.8 shows The Inventory Manager and Financial Controller use this view to identify PPG materials that have recorded no consumption for nine or more consecutive months with no associated lot expiry date, classifying them as Magna Carta

Dead Stock requiring financial provision. The `is_mc_dead_stock` flag must be applied in the Gold layer, and the results must be available through the Curated Synapse view before this page can be accessed.

Table 4.8: Use Case Description for View Magna Carta Dead Stock

Field	Description
Use Case ID	UC-03
Use Case Name	View Magna Carta Dead Stock
Actor	Inventory Manager, Financial Controller
Description	The actor views materials that have recorded no consumption for nine or more consecutive months and have no associated lot expiry date, qualifying them as Magna Carta Dead Stock.
Pre-condition	Silver layer MC Dead Stock flag logic has been applied and results are loaded into the Gold layer.
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q3 view. 3. System displays all materials where <code>MC_Dead_Stock = TRUE</code>, showing material name, months without consumption, and stock value.
Post-condition	Actor identifies Dead Stock materials requiring financial provision and management escalation.
Exception	If no materials qualify, the view displays an empty table.

4.4.1.1.4 <<UC-04: View Magna Carta Expired Materials>>

Table 4.9 shows The Inventory Manager and Financial Controller use this view to identify PPG materials whose lot expiry date has already passed as of the inventory snapshot date of 2025-12-31, classifying them as Magna Carta Expired and triggering a 100% financial provision. The `is_mc_expired` flag must be applied in the Gold layer

and exposed through the Curated layer before this view is accessible.

Table 4.9: Use Case Description for View Magna Carta Expired Materials

Field	Description
Use Case ID	UC-04
Use Case Name	View Magna Carta Expired Materials
Actor	Inventory Manager, Financial Controller
Description	The actor views materials whose recorded lot expiry date has already passed, classifying them as Magna Carta Expired and requiring immediate write-down.
Pre-condition	Silver layer MC Expired flag logic has been applied and results are loaded into the Gold layer.
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q4 view. 3. System displays all materials where MC_Expired = TRUE, showing material name, lot expiry date, and stock value.
Post-condition	Actor identifies Expired materials requiring a 100% financial provision to be raised.
Exception	If no materials have passed their expiry date, the view displays an empty table.

4.4.1.1.5 <<UC-05: View Total MC Provision Amount>>

Table 4.10 shows The Financial Controller uses this view to obtain the total financial provision amount required across all Magna Carta classified materials at PPG, which directly supports balance sheet adjustments in the current reporting period. The mc_provision_summary output must be generated in the Gold layer and registered as a Curated Synapse view before this use case can be executed.

Table 4.10: Use Case Description for View Total MC Provision Amount

Field	Description
Use Case ID	UC-05
Use Case Name	View Total MC Provision Amount
Actor	Financial Controller
Description	The actor views the total financial provision required across all Magna Carta materials, representing the total value that must be written down in the current reporting period.
Pre-condition	Provision amounts have been calculated in the Silver layer and loaded into the Gold layer
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q5 view. 3. System displays the total provision amount as a KPI card, with a breakdown table showing provision per material, category, and MC classification type.
Post-condition	Financial Controller uses the provision total to support balance sheet adjustments and financial reporting.
Exception	If no MC materials exist, the provision total displays as RM0.00.

4.4.1.1.6 <<UC-06: View Unfulfilled Production Orders>>

Table 4.11 shows The Procurement Team uses this view to identify which of PPG's production orders could not be fulfilled because the raw material stock available was less than the quantity required at the time of scheduled production. The production_fulfilment output must be available as a Curated Synapse view before the dashboard can display these results.

Table 4.11: Use Case Description for View Unfulfilled Production Orders

Field	Description
Use Case ID	UC-06
Use Case Name	View Unfulfilled Production Orders
Actor	Procurement Team
Description	The actor views orders that could not be fulfilled because the required raw material stock was insufficient at the time of scheduled production.
Pre-condition	Stockout risk flags and production order data have been processed in the Silver layer and loaded into the Gold layer.
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q6 view. 3. System displays all production orders where fulfillment failed due to insufficient stock, showing order ID, material name, required quantity, available quantity, and shortfall.
Post-condition	Procurement Team escalates with suppliers or adjusts production schedules accordingly.
Exception	If all production orders were fulfilled, the view displays an empty table with a confirmation message.

4.4.1.1.7 <<UC-07: View Affected Customer Sales Orders>>

Table 4.12 shows The Procurement Team uses this view to identify which open customer sales orders at PPG are at risk of delayed fulfilment because the raw materials required for the linked production order are flagged as stockout risks. The sales_order_impact output must be available as a Curated Synapse view before this use can be accessed.

Table 4.12: Use Case Description for View Affected Customer Sales Orders

Field	Description
Use Case ID	UC-07
Use Case Name	View Affected Customer Sales Orders
Actor	Procurement Team
Description	The actor views open customer sales orders that are at risk of non-fulfilment because the raw materials required for the associated production order is flagged as a stockout risk.
Pre-condition	Stockout risk tracing logic has been applied in the Silver layer, linking at-risk materials to open sales orders, and results are loaded into the Gold layer.
Flow of Events	1. Actor opens the Power BI dashboard. 2. Actor navigates to the Q7 view. 3. System displays all affected sales orders, showing order ID, customer name, material at risk, expected delivery date, and risk status.
Post-condition	Procurement Team proactively contacts affected customers and initiates supplier escalation to minimize service disruption.
Exception	If no sales orders are affected by stockout risk, the view displays an empty table.

4.4.2 Activity Diagram

Figure 4.3 shows the activity diagram for the end-to-end data flow of the pipeline, starting from the point where raw source files are received to the Power BI dashboard being rendered for the end users to use. The diagram is divided into three swim lanes mirroring three main processing environments involved, which are Azure

Data Factory which handles the ingestion, Azure Databricks which handles transformations and business logics, and Azure Synapse Analytics and Power BI which then handle storage and visualization accordingly.

The process begins when Azure Data Factory is triggered and starts loading all six CSV source files into the Bronze layer of Azure Data Lake Storage Gen2. Before the data moves forward, basic quality checks are run on each file to detect nulls, remove duplicate rows, and fix incorrect data types. Any record that fails these checks are logged separately and do not proceed any further. Records that passed are then stored as clean Bronze data and passed to Databricks for Silver layer processing.

In the Silver layer, the data goes through a sequential transformation step. Firstly, twelve-month consumption is calculated for every material. Based on that, the Recoverable Assets flag is applied to any material whose current stock exceeds the consumption figure. Moreover, the pipeline checks every material for Magna Carta Dead Stock conditions, where no consumption recorded for nine or more consecutive months and no expiry dates have already passed to identify Magna Carta Expired materials. For any material that qualifies under either Magna Carta category, a 100% financial provision is calculated by multiplying the stock quantity by the unit cost. The pipeline calculates each material's reorder point based on average daily consumption and supplier lead time and flags any material whose stock falls below that threshold as a stockout risk simultaneously. For those flagged materials, the pipeline traces through the open sales orders data to identify which customer orders are directly affected. Once all flags, provision values, and risk indicators have been computed, the full dataset is loaded into the Gold layer star schema in Azure Synapse Analytics and Power BI connects to then render the seven dashboard views.

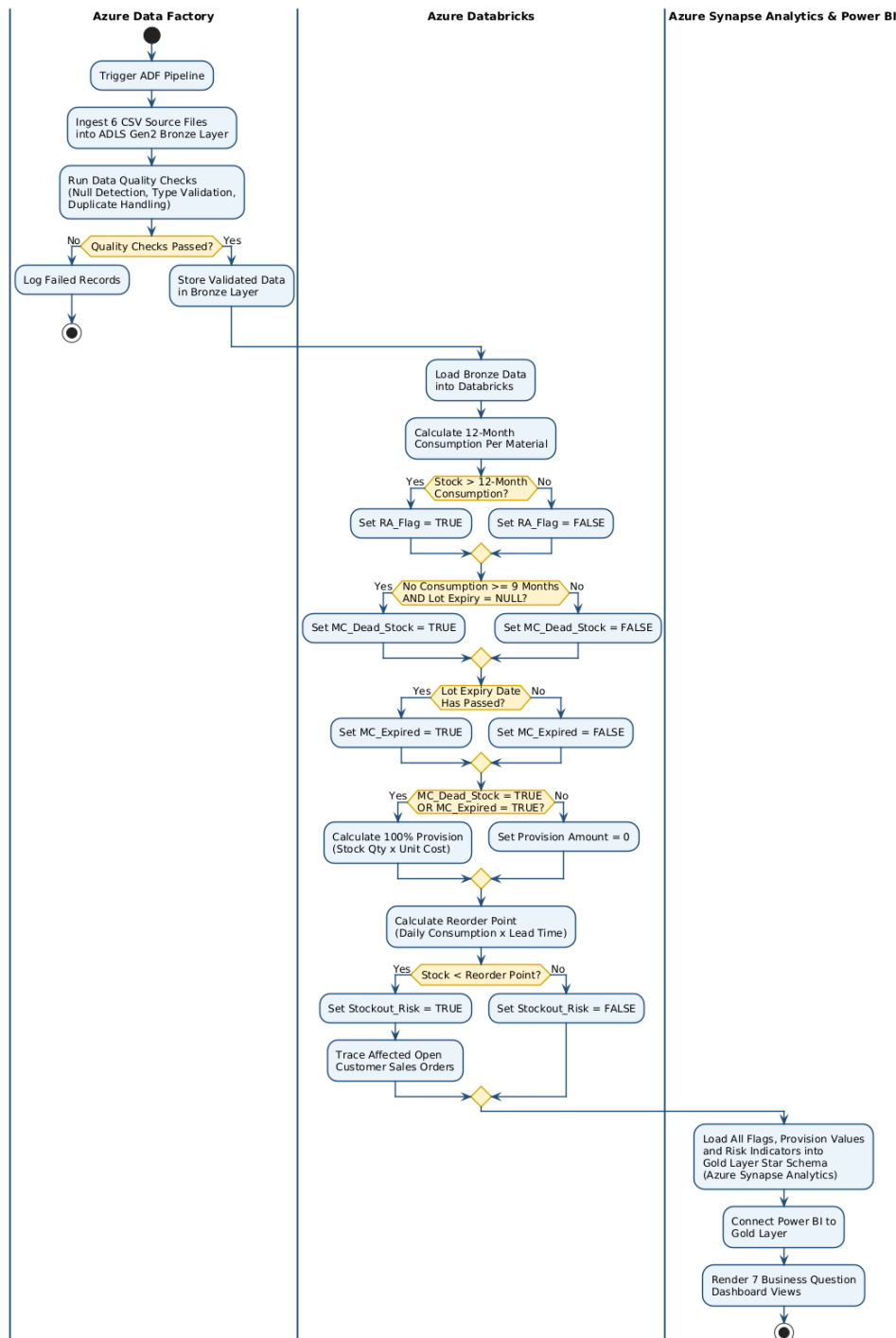


Figure 4.3 Activity Diagram: Recoverable Assets & Inventory Risk Management System

4.4.3 Sequence Diagram

The sequence diagram for the Recoverable Assets and Inventory Risk Management system is shown in Figure 4.4. The sequence diagram is concentrates on the activities within the system as it runs, instead of the overall flow. The diagram illustrates a data

flow from the source files, through the Azure services and finally to the users through the Power BI dashboard.

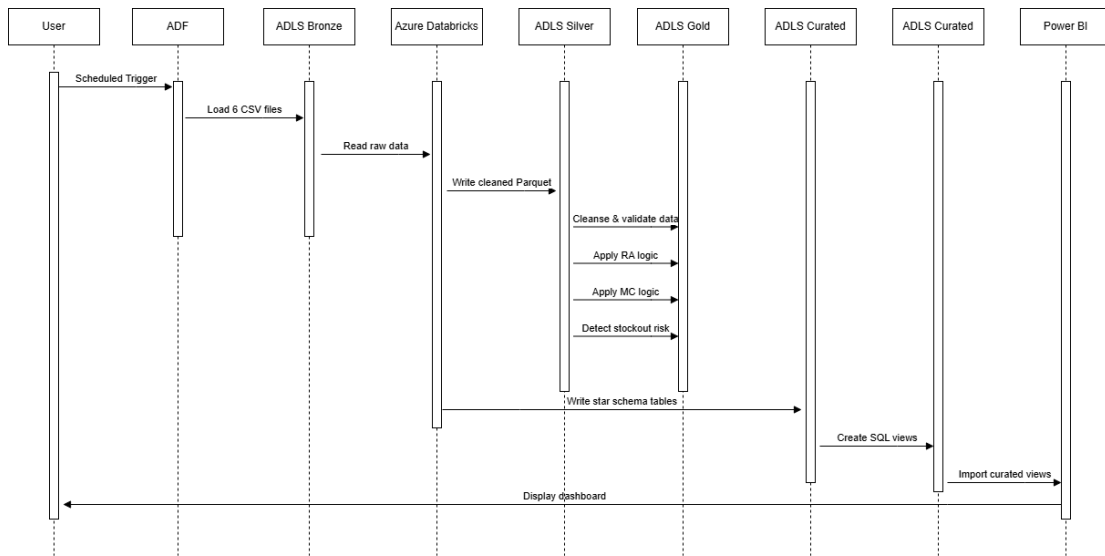


Figure 4.4 Sequence Diagram: Recoverable Assets & Inventory Risk Management System

When the user initiates a scheduled run in Azure Data Factory (ADF), the procedure begins. This loads six source CSV files into the Azure Data Lake Storage (ADLS) Bronze container and initiates the data pipeline. To guarantee complete traceability of the original source data, the data is currently stored exactly as it was received, without any modifications.

Azure Databricks reads the files from the Bronze layer and starts processing the Silver layer after the raw data has been imported. The data is standardized and cleansed in this step. This involves removing duplication, addressing missing values, fixing data types, and guaranteeing uniform formatting throughout all datasets. For improved efficiency and organization, the cleaned data is subsequently stored as Parquet files in the Silver container.

The cleansed Silver data is then used by Databricks to carry out the Gold layer operations. A number of business rules are implemented sequentially. The data is first verified and made ready for analysis. Based on consumption trends, materials with excess stock are then identified using Recoverable Asset logic. Magna Carta logic is then used to determine the financial provision values of dead stock and expired

materials. Lastly, a comparison between current stock levels and reorder thresholds is used to analyze stockout risk. The Gold container contains structured star schema tables that contain the results.

The curated star schema tables are transferred into the Curated layer after the Gold layer is finished. In order to make the data easier to query and accessible through a conventional SQL endpoint without directly accessing the storage files, Azure Synapse Analytics is used in this layer to construct SQL views on top of the tables.

The carefully chosen views from Synapse are then imported into Power BI's data model. The dashboards, which include the Executive Summary page and the seven business question pages, are then created using the data. This indicates the pipeline's final output, where users can engage with the finished analytics solution.

4.5 Database Design

Galaxy Schema is used to implement the database design in the Curated layer. The Gold layer provides business ready outputs, and the Curated layer transforms these outputs into several fact tables and common dimension tables for analytical reporting.

The star schema design supports the project's main business questions, including identifying materials below reorder level, classifying Recoverable Assets (RA), identifying Magna Carta Dead Stock and Expired materials, calculating financial provision amounts, detecting stockout risks, and tracing affected customer orders. This design also aligns with the project's need to provide dashboard-ready data for reporting and decision-making. The project documentation already specifies that a Gold layer star schema, together with a central inventory fact table with material, supplier, time, and sales order dimensions, will be used. The use case document also states that the final deliverable will be a star schema design, which includes a *fact_inventory_status* table and dimension tables.

4.5.1 Database Design Approach

The database uses a dimensional modelling approach. The first one is storing numerical and measurable values into a fact table and descriptive business information into dimension tables. The fact table contains inventory status, consumption, RA classification, and MC classification flags, stockout indicators, and value of financial provision. The dimension tables include context, including details about the material, information about suppliers, date attributes, production order information, and sales order information.

This is an appropriate strategy for the project because the Power BI dashboard must be able to filter and analyze inventory risk by material category, supplier, month, order from customer, and stock risk status. A star schema also eliminates many of the complicated joins that have to be performed when reporting, making dashboard queries easier to comprehend and quicker to run.

4.5.2 Source Tables

The database design is based on six source CSV files used in the project pipeline. Table 4.13 shows the source datasets and their purpose.

Table 4.13 Source Dataset Description

Source File	Purpose
<i>materials.csv</i>	Stores master data for raw materials, including material name, category, unit of measure, reorder level, lead time, and unit cost.
<i>inventory_snapshot.csv</i>	Store monthly inventory quantity on hand for each material and warehouse location.
<i>production_orders.csv</i>	Stores material consumption records based on production orders and is used to calculate 12-month material consumption.
<i>purchase_orders.csv</i>	Stores supplier purchase order records, including ordered quantity, received quantity, expected delivery date, and actual delivery date.
<i>suppliers.csv</i>	Stores supplier reference information, including supplier name, country, and reliability rating.
<i>sales_orders.csv</i>	Stores customer sales order records, including customer name, finished product, quantity ordered, required delivery date, and order status.

4.5.3 Star Schema Design

Figure 4.5 shows the suggested database structure is one single fact table and six-dimension tables. The name of the central fact table is fact_inventory_status. It contains the monthly inventory risk status of each material. The dim tables are dim_material, dim_supplier, dim_time, dim_purchase_order, dim_production_order and dim_sales_order.

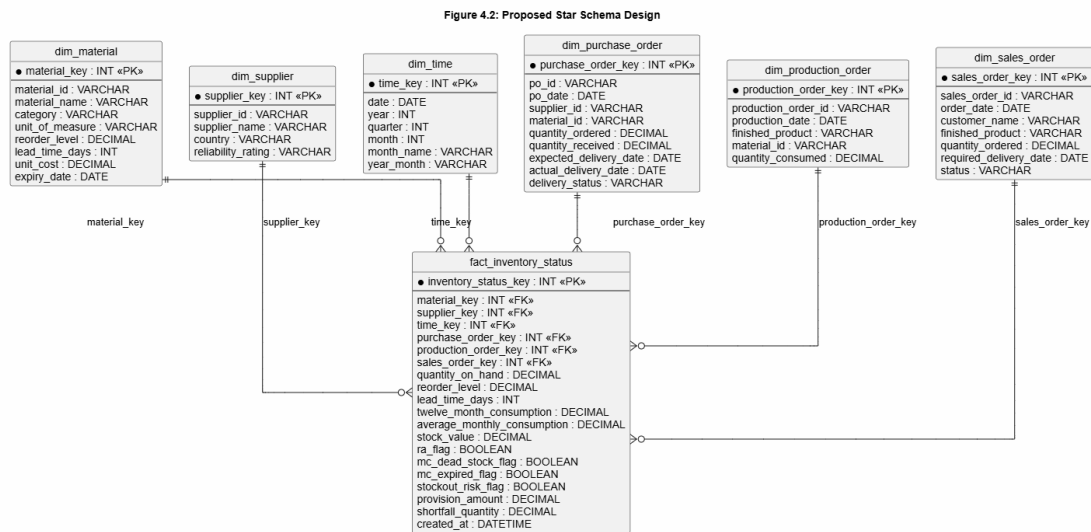


Figure 4.5 Proposed Star Schema Design

The fact table is placed at the centre because it contains the measurable values required for analysis. Each dimension table connects to the fact table through primary and foreign key relationships.

4.5.4 Fact Table Design

The main fact table for the proposed database is `fact_inventory_status`. The final inventory risk results in this table follow the Recoverable Assets (RA), Magna Carta (MC), financial provision, and stockout risk rules. It is the main table in the star schema and has foreign keys to all dimension tables.

The aim of table fact table is to hold the measurable inventory values and classification outputs that are needed for dashboard reporting. The rows in the fact table are for every reporting period of a material inventory.

Table 4.14 is significant because it provides the end result of the inventory risk classification process. The RA flag is created to compare the current stock level with the consumption for the previous 12 months. The MC Dead Stock flag is created by seeing if a material has no use for 9+ months. The material expiry date is used to generate the MC Expired flag by comparing it with the reporting date. The stockout risk flag is based on the current stock level, reorder level, and demand.

Table 4.14: Fact Inventory Status

Column Name	Description
<i>inventory_status_key</i>	Surrogate primary key for each inventory status record.
<i>material_key</i>	Foreign key linked to <code>dim_material</code> .
<i>supplier_key</i>	Foreign key linked to <code>dim_supplier</code> .
<i>time_key</i>	Foreign key linked to <code>dim_time</code> .
<i>purchase_order_key</i>	Foreign key linked to <code>dim_purchase_order</code> , where replenishment information is available.
<i>production_order_key</i>	Foreign key linked to <code>dim_production_order</code> , where consumption information is available.
<i>sales_order_key</i>	Foreign key linked to <code>dim_sales_order</code> , where customer order impact exists.
<i>quantity_on_hand</i>	Current stock quantity available for the material.

<i>reorder_level</i>	Minimum stock level required before replenishment is needed.
<i>lead_time_days</i>	Number of days required to receive replenishment from the supplier.
<i>twelve_month_consumption</i>	Total quantity consumed over the past 12 months.
<i>average_monthly_consumption</i>	Average monthly consumption of the material.
<i>stock_value</i>	Current inventory value, calculated as quantity on hand multiplied by unit cost.
<i>ra_flag</i>	Indicates whether the material is classified as a Recoverable Asset.
<i>mc_dead_stock_flag</i>	Indicates whether the material is classified as Magna Carta Dead Stock.
<i>mc_expired_flag</i>	Indicates whether the material is classified as Magna Carta Expired.
<i>stockout_risk_flag</i>	Indicates whether the material is at risk of stockout.
<i>provision_amount</i>	Financial provision amount for materials classified under Magna Carta.
<i>shortfall_quantity</i>	Quantity shortage where available stock is insufficient to meet demand.
<i>created_at</i>	Timestamp showing when the record was loaded into the Gold layer.

4.5.5 Dimension Table Design

The dimension tables contain descriptive data that will be used for filtering, grouping and interpretation on the Power BI dashboard. The following tables give context to the measures that are stored in the fact table.

Dim Material

Table 4.15 is used to store master data about raw materials. It holds the material details

like Material Name, Category, Unit of Measure, Reorder Level, Lead time, Unit Cost, and Expiry Date. The expiry date has been included because the project requires that materials be identified as Magna Carta Expired.

Table 4.15: Dim Material

Column Name	Description
<i>material_key</i>	Surrogate primary key for the material dimension.
<i>material_id</i>	Original material ID from the source file.
<i>material_name</i>	Name of the raw material.
<i>category</i>	Material category, such as pigment, resin, solvent, or packaging.
<i>unit_of_measure</i>	Unit used to measure the material.
<i>unit_of_measure</i>	Minimum stock level required for the material.
<i>lead_time_days</i>	Number of days required for replenishment.
<i>unit_cost</i>	Cost per unit of material.
<i>expiry_date</i>	Date when the material expires. This field is used to classify Magna Carta Expired materials.

Dim Supplier

Table 4.16 stores supplier reference information. It allows inventory and purchase order performance to be analyzed by suppliers.

Table 4.16: Dim Supplier

Column Name	Description
<i>supplier_key</i>	Surrogate primary key for the supplier dimension.
<i>supplier_id</i>	Original supplier ID from the source file.
<i>supplier_name</i>	Name of the supplier.
<i>country</i>	Supplier country.
<i>reliability_rating</i>	Supplier reliability rating.

Dim Time

Table 4.17 supports monthly, quarterly, and yearly reporting in Power BI.

Table 4.17: Dim Time

Column Name	Description
<i>time_key</i>	Surrogate primary key.
<i>date</i>	Calendar date.
<i>year</i>	Year value.
<i>quarter</i>	Quarter value.
<i>month</i>	Month number.
<i>month_name</i>	Name of the month.
<i>year_month</i>	Year and month format used for monthly reporting.

Dim Purchase Order

Table 4.18 stores purchase order details from *purchase_orders.csv*.

Table 4.18: Dim Purchase Order

Column Name	Description
<i>purchase_order_key</i>	Surrogate primary key.
<i>po_id</i>	Original purchase order ID.
<i>po_date</i>	Date the purchase order was created.
<i>supplier_id</i>	Supplier linked to the purchase order.
<i>material_id</i>	Material ordered.
<i>quantity_ordered</i>	Quantity ordered from supplier.
<i>quantity_received</i>	Quantity received from supplier.
<i>expected_delivery_date</i>	Expected delivery date.
<i>actual_delivery_date</i>	Actual delivery date.
<i>delivery_status</i>	Indicates whether the purchase order was on time, late, partial, or not received.

Dim Production Order

Table 4.19 stores production consumption records from *production_orders.csv*. This table is important because it is used to calculate 12-month consumption and identify inactive materials.

Table 4.19: Dim Production Order

Column Name	Description
<i>production_order_key</i>	Surrogate primary key.
<i>production_order_id</i>	Original production order ID.

<i>production_date</i>	Date of production.
<i>finished_product</i>	Finished paint product produced.
<i>material_id</i>	Raw material consumed.
<i>quantity_consumed</i>	Quantity of raw material consumed.

Dim Sales Order

Table 4.20 stores customer order records from sales_orders.csv. This dimension supports the analysis of customer orders affected by material shortages.

Table 4.20: Dim Sales Order

Column Name	Description
<i>supplier_key sales_order_key</i>	Surrogate primary key.
<i>sales_order_id</i>	Original sales order ID.
<i>order_date</i>	Date the sales order was created.
<i>customer_name</i>	Name of the customer.
<i>finished_product</i>	Finished product ordered by the customer.
<i>quantity_ordered</i>	Quantity ordered by the customer.
<i>required_delivery_date</i>	Required delivery date requested by the customer.
<i>status</i>	Sales order status, such as delivered or open.

4.5.6 Relationships Between Tables

Table 4.21 designed using one-to-many relationships. Each dimension record can be linked to many fact records, but each fact record links to one related dimension record.

Table 4.21: Database Relationship

Parent Table	Child Table	Relationship
<i>dim_material</i>	fact_inventory_status	One material can appear in many monthly inventory status records.
<i>dim_supplier</i>	fact_inventory_status	One supplier can supply many inventory records through purchase orders.
<i>dim_time</i>	fact_inventory_status	One date or month can relate to many inventory status records.

<i>dim_purchase_order</i>	fact_inventory_status	One purchase order can support material replenishment analysis.
<i>dim_production_order</i>	fact_inventory_status	One production order can contribute to material consumption analysis.
<i>dim_sales_order</i>	fact_inventory_status	One sales order can be linked to material shortage impact analysis.

Primary keys are created as surrogate keys in the dimension tables to improve query performance and maintain consistency in the warehouse. Original business keys such as material_id, supplier_id, po_id, production_order_id, and sales_order_id are still kept in the dimension tables for traceability.

4.5.7 Business Rule Mapping in the Database

The database design also supports the business rules required for RA and MC classification as shown in Table 4.22.

Table 4.22: Business Rule Mapping

Business Rule	Source Data Used	Output Column
<i>Recoverable Assets classification</i>	inventory_snapshot.quantity_on_hand, production_orders.quantity_consumed	ra_flag
<i>Magna Carta Dead Stock classification</i>	production_orders.production_date, production_orders.quantity_consumed	mc_dead_stock_flag
<i>Magna Carta Expired classification</i>	Inventory expiry field, if available	mc_expired_flag
<i>Financial provision calculation</i>	quantity_on_hand, unit_cost, MC flags	provision_amount
<i>Stockout risk detection</i>	quantity_on_hand, reorder_level, lead time days	stockout_risk_flag
<i>Customer order impact tracing</i>	sales_orders, production_orders, inventory snapshot	shortfall_quantity, sales order key

The provision amount is calculated only for materials classified under Magna Carta.

The provision value is calculated as 100% of the current stock value. Therefore, the calculation is:

$$\text{provision_amount} = \text{quantity_on_hand} \times \text{unit_cost}$$

For non-Magna Carta materials, the provision amount is set to zero.

4.5.8 Data Dictionary Summary

The final Gold layer database is designed to support dashboard reporting. Table 4.23 used in Power BI include quantity_on_hand, twelve_month_consumption, ra_flag, mc_dead_stock_flag, mc_expired_flag, stockout_risk_flag, provision_amount, and shortfall_quantity. These fields allow the dashboard to answer all seven business questions required in the project.

Table 4.23: Key Analytical Fields

Field	Purpose
<i>quantity_on_hand</i>	Shows available material stock.
<i>reorder_level</i>	Used to identify materials below minimum stock level.
<i>twelve_month_consumption</i>	Used to compare inventory against historical usage.
<i>ra_flag</i>	Identifies Recoverable Asset materials.
<i>mc_dead_stock_flag</i>	Identifies materials with no consumption for nine or more months.
<i>mc_expired_flag</i>	Identifies expired materials, if expiry data is available.
<i>stock_value</i>	Shows financial value of inventory.
<i>provision_amount</i>	Shows required financial write-down for MC materials.
<i>stockout_risk_flag</i>	Identifies materials at shortage risk.
<i>shortfall_quantity</i>	Shows how much material is missing to fulfil demand.

4.5.9 Justification of Database Design

The selection of the star schema is based on its simplicity, scalability, and appropriateness for business intelligence reporting. It helps to distinguish between the facts of an inventory that can be measured and its descriptive dimensions, thus simplifying the understanding and maintenance of the data model. It also provides Power BI users the ability to filter the dashboard by material, supplier, selected time period, customer order, and risk category.

This design is suitable for the project due to the Medallion Architecture of raw data being kept in Bronze, cleaned data being kept in Silver, and analytics-ready data being kept in Gold. The project utilizes a star schema in the Gold layer to ensure that the data produced is structured, consistent, and suitable for visualization on a dashboard.

4.6 Interface Design

This section presents the interface design of the Power BI dashboard. The dashboard consists of three pages, each addressing a different dimension of inventory risk.

Figure 4.6 covers Recoverable Asset classification, Magna Carta Dead Stock, Magna Carta Expired materials, and total financial provision. The four KPI cards display the Recoverable Asset Count (11), MC Dead Stock Count (5), MC Expired Count (2), and Total Provision (78.89K). A donut chart shows the proportion of materials by risk status, while a horizontal bar chart breaks down the provision amount by material category. A detail table at the bottom lists each material with its risk classification and provision amount.

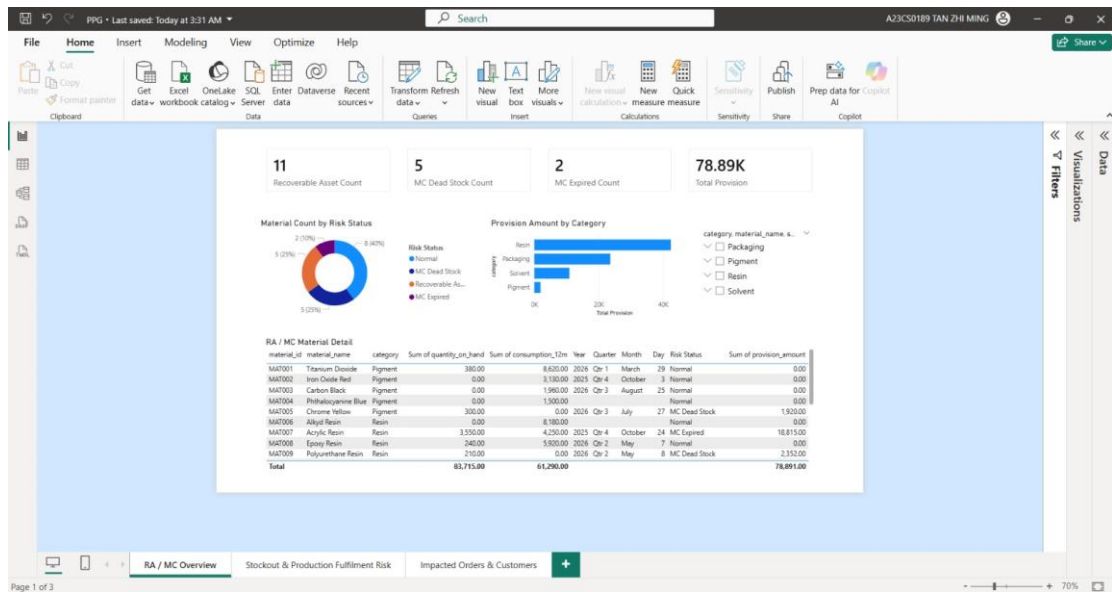


Figure 4.6 MC Overview

Figure 4.7 focuses on materials at stockout risk and unfulfilled production orders. The KPI cards show Stockout Risk Count (8), Below Reorder Count (9), Unfulfilled Production Orders (83), and Total Production Shortfall (43.34K). A horizontal bar chart lists materials flagged for stockout risk ranked by quantity on hand, and a vertical bar chart shows shortfall quantity per production order. Two detail tables at the bottom list the stockout risk materials and the production order fulfilment status respectively.

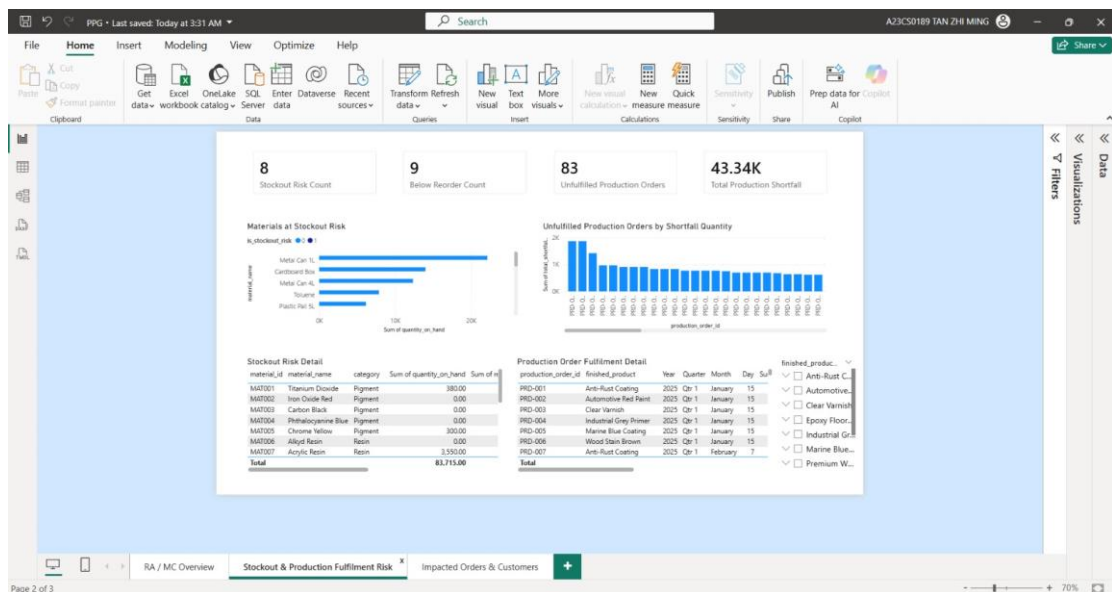


Figure 4.7 Stockout & Production Fulfillment Risk

Figure 4.8 shows the downstream customer impact of material shortages. The KPI cards display Impacted Orders (10), Impacted Customers (10), Total Impacted Quantity (9K), and Open Impacted Orders (23). A horizontal bar chart ranks customers by impacted quantity, and a vertical bar chart plots impacted quantity against required delivery dates. A detail table at the bottom lists all open impacted sales orders with the material, customer name, quantity ordered, required delivery date, and order status.

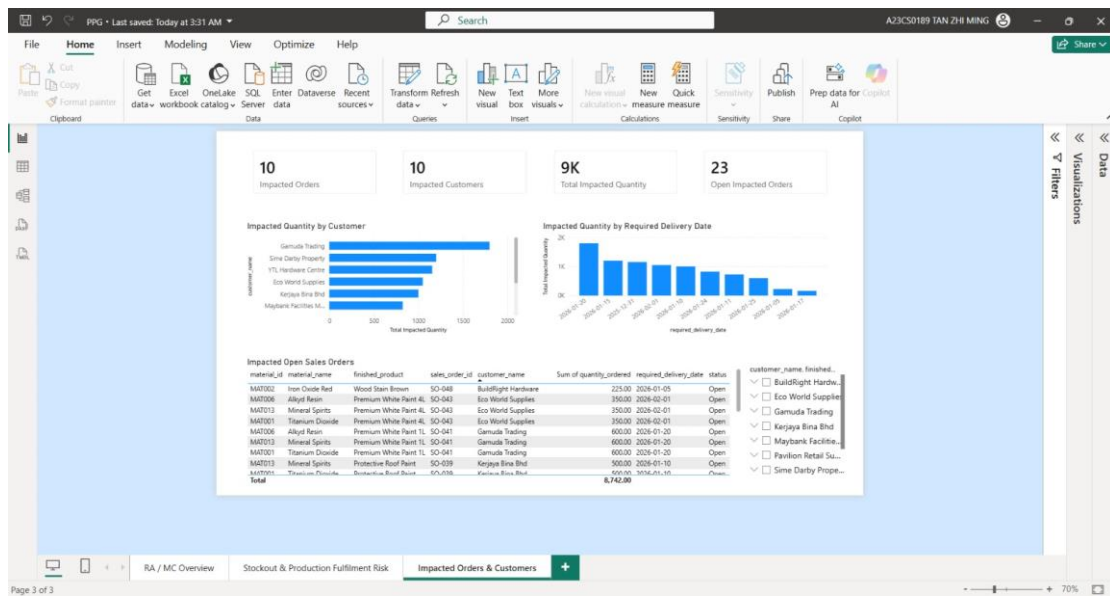


Figure 4.8 *Impacted Orders & Customers*

4.7 Summary

The analysis and design of the proposed Recoverable Assets and Inventory Risk Management system have been explained in this chapter. The overall system architecture was presented, explained the interaction and integration between the various Microsoft Azure services and Power BI.

This chapter also covered functional and non-functional requirements which outline the expected capabilities and quality of the system. Several design artefacts were also provided such as use case diagram, use case descriptions, activity diagram and sequence diagram. These diagrams clearly illustrate the functioning, workflow and interactions of the system's components. In addition, the structure and the interface design of the database were designed to ensure effective data storage, processing and

visualization. The proposed design will also ensure that inventory risks like Recoverable Assets, Magna Carta classification, stockout risk and affected customer orders can be identified and effectively presented in the Power BI dashboard.

In general, the activities of analysis and design carried out in this chapter will give a good basis for the implementation phase as discussed in the next chapter for the development of the system, so the system can be developed in a structured and systematic manner.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.0 Introduction

Chapter 5 presents the full implementation of the end-to-end data pipeline developed for this project. The development process followed a four-layer medallion architecture consisting of the Bronze, Silver, Gold, and Curated layers, each serving a specific purpose in processing the data. Every layer was built and tested using Microsoft Azure cloud services, including Azure Data Factory, Azure Databricks, Azure Data Lake Storage Gen2, and Azure Synapse Analytics. Once all layers were completed, a Power BI dashboard was developed and connected to the Curated layer to visually present the results and answer seven pre-defined business questions related to inventory risk management.

5.1 System Development

The Bronze layer was developed using Azure Data Factory, which acted as the raw ingestion environment for orchestrating and validating all the synthetic datasets required for the project. The development process followed a clear, activity-based pipeline design, where dedicated Data Flow components handled the extraction of the six raw CSV files from the landing zone and directly enforced basic data quality checks. Connectors were established to link the source datasets to the target Bronze container within the storage account to ensure that all data was preserved exactly as-is while simultaneously executing runtime null detection, type validation and duplicate row detection prior to storage.

Next, the Silver layer was developed using Azure Databricks, which acted as the transformation environment for standardizing and cleaning all the datasets that have been ingested by the Bronze layer. The development process followed a sequential notebook-based approach, where every cell handled a specific transformation task, from loading the raw data through to writing the cleaned outputs back to Azure Data Lake Storage Gen2. The notebook was connected to the Bronze container in the ppgspipeline storage account

using a storage access key, with paths defined for both the Bronze ingested folder and the Silver output container before any data processing began.

Subsequently, The Gold layer was developed using Azure Databricks which acted as the business transformation environment for converting the cleaned Silver datasets into business-ready outputs that could directly support analysis and reporting. The development followed a notebook-based approach where each cell applied a specific business rule or calculation including Recoverable Asset classification, Magna Carta expired and dead stock identification, stockout risk detection, production fulfillment checking and sales order impact analysis. The outputs were written as Parquet files into the Gold container in Azure Data Lake Storage Gen2, with each business output stored in its own subfolder for easy access.

Moreover, the Curated layer was developed using Azure Databricks and Azure Synapse Analytics, which together acted as the serving environment for organizing and exposing the Gold outputs in a structured format suitable for dashboard consumption. The development followed a star schema design approach, where the Gold data was restructured into one central fact table and four dimension tables. The curated tables were written as Parquet files into the Curated container in Azure Data Lake Storage Gen2, and then registered as SQL views in Azure Synapse Analytics using a serverless SQL pool. These views were then connected to Power BI for final dashboard reporting.

Lastly, the Power BI dashboard was developed as the final component of the system, serving as the visualization layer that presents the processed inventory data to end users. The dashboard was connected to the Azure Synapse Analytics Curated layer using Import Mode, where all required tables were loaded and relationships were configured following the same star schema structure. A total of eight dashboard pages were created, consisting of a Home Page for key inventory risk indicators and seven dedicated pages each answering one of the pre-defined business questions.

5.2 Bronze Layer

Figure 5.1 shows the Azure Data Lake Service (ADLS) Gen2 bronze container was set up with a raw subdirectory which serves as the manual upload zone for the six synthetic CSV

source files. The six folders visible are the inventory_snapshot, materials, production_orders, purchase_orders, sales_orders and suppliers, each represent one of the uploaded source datasets, confirming that all raw data was successfully placed into the designated landing area before pipeline execution.

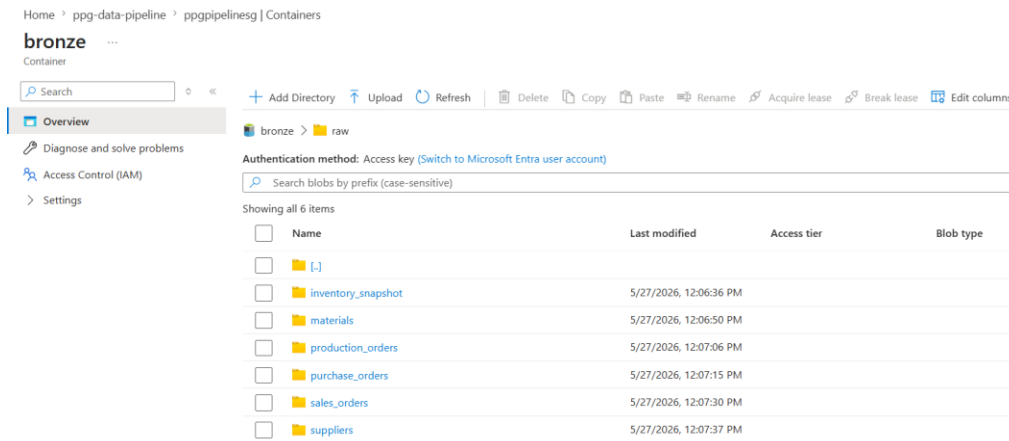


Figure 5.1 Raw data uploaded

Next, A Linked Service named AzureDataLakeStorage1 as shown in Figure 5.2 was configured in Azure Data Factory (ADF) to establish a secure connection between the ADF pipeline and the ADLS Gen2 storage account (ppgpipelinesq.dfs.core.windows.net). The connection was authenticated using an account key and routed through the AutoResolveIntegrationRuntime, enabling ADF to read from the raw container and write pipeline outputs to the bronze layer.

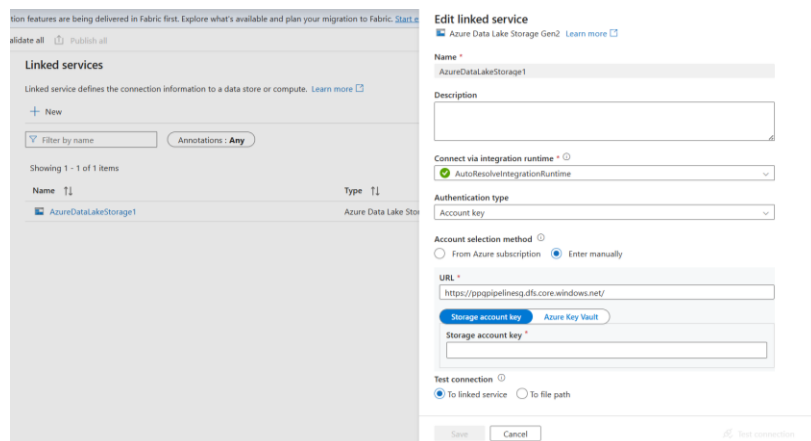


Figure 5.2 Create Linked Service

Subsequently, The Mapping Data Flow (dataflow2) in Figure 5.3 illustrates the data quality logic completed to check the quality, applied to every dataset. Data is first ingested

from the source, then passed through a CountRows aggregate transformation that groups records by all key columns to detect duplicates. A UniqueRows conditional split separates unique from duplicate records, while a subsequent GoodData and CorruptedData split flags rows failing null or type validation. Clean records are routed to sink1, duplicate rows to sink2 and corrupted rows to sink3, ensuring all data quality issues are isolated into dedicated error log folders without interrupting the pipeline.

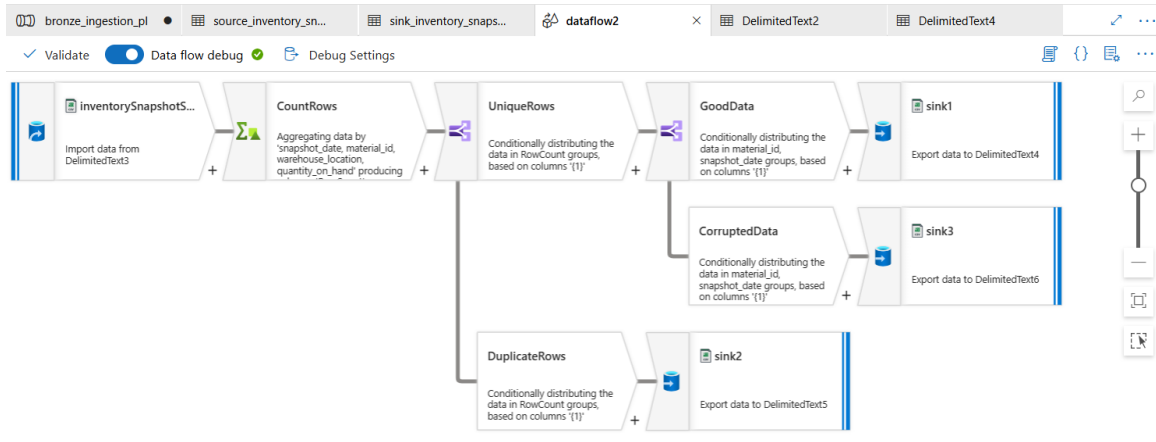


Figure 5.3 Data flow

Additionally, the Azure Data Factory (ADF) pipeline canvas in **Figure 5.4** displays the structural design of the bronze_ingestion_pl pipeline, which was created to ingest the raw data into the bronze container. All six copy activities, namely copy_materials, copy_production_orders, copy_sales_orders, copy_purchase_orders, copy_inventory_snapshot, and copy_suppliers are sequentially connected to their respective data flows. This ensures that each of the six raw CSV files is processed through the data quality logic during the ingestion pipeline run.

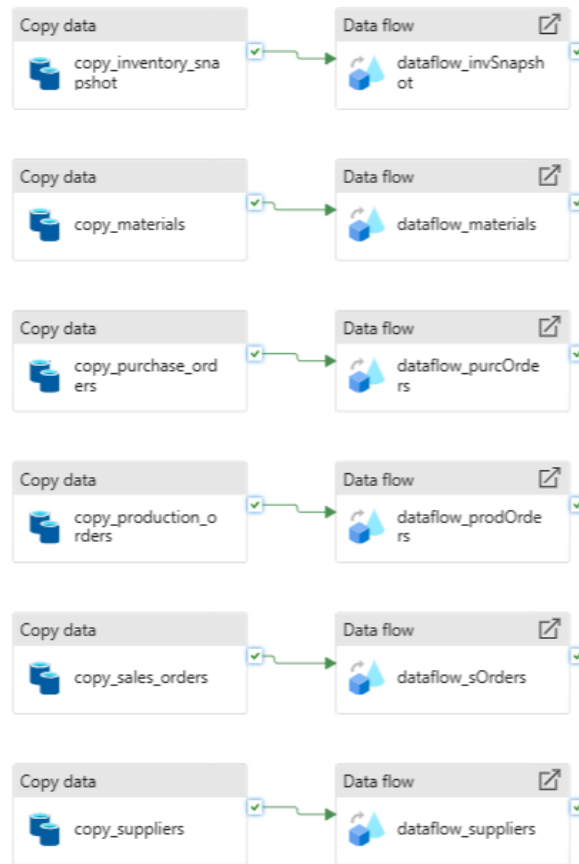


Figure 5.4 Data pipeline ingestion

Following the data flow debug run, the resulting output folders generated by the quality check logic are created inside the bronze container as shown in Figure 5.4. The clean_inventory_snapshot folder contains records that passed all validation rules, while the error_log_corrupted and error_log_duplicate folders capture rows that failed null or type checks and duplicate detection respectively. The presence of these segregated output folders confirms that the data flow quality checks executed correctly with corrupted and duplicate records isolated from the clean data.

Name	Last modified	Access tier	Blob type
[.]			
clean_inventory_snapshot	6/7/2026, 8:57:22 AM		
error_log_corrupted	6/7/2026, 9:00:18 AM		
error_log_duplicate	6/7/2026, 8:59:19 AM		
inventory_snapshot.csv	6/7/2026, 9:16:06 AM	Hot (Inferred)	Block blob

Figure 5.5 Data quality logic result outputs

5.3 Silver Layer

5.3.1 Loading Bronze Data

The first step of the Silver notebook was to load all raw CSV files from the Bronze ingested folder in the ppgpipelinesg storage account into Databricks dataframes using PySpark's `spark.read` method with header detection and schema inference enabled. The six datasets were loaded into individual dataframes including `df_materials` for the materials master data, `df_purchase` for the purchase orders, `df_inventory` for the inventory snapshot, `df_production` for the production orders, `df_suppliers` for the supplier reference data, and `df_sales` for the customer sales orders. The output confirmed that all of the files were successfully loaded with the correct column names and inferred data types, including `material_id`, `snapshot_date`, `po_id`, `production_order_id`, `supplier_id`, and `sales_order_id` as their respective primary identifiers, confirming that the Silver notebook could access the Bronze outputs before any cleaning operations were applied.

```

18 hours ago (6s) 2
# Load all 6 CSV files from Bronze layer
df_materials = spark.read.option("header", True).option("inferSchema", True).csv(bronze_path + "ingested/materials/
materials.csv")
df_inventory = spark.read.option("header", True).option("inferSchema", True).csv(bronze_path + "ingested/
inventory_snapshot/inventory_snapshot.csv")
df_purchase = spark.read.option("header", True).option("inferSchema", True).csv(bronze_path + "ingested/
purchase_orders/purchase_orders.csv")
df_production = spark.read.option("header", True).option("inferSchema", True).csv(bronze_path + "ingested/
production_orders/production_orders.csv")
df_suppliers = spark.read.option("header", True).option("inferSchema", True).csv(bronze_path + "ingested/suppliers/
suppliers.csv")
df_sales = spark.read.option("header", True).option("inferSchema", True).csv(bronze_path + "ingested/sales_orders/
sales_orders.csv")

print("All 6 files loaded successfully")
(12) Spark Jobs

```

Figure 5.6 Bronze Container Ingested Folder.

```

> df_inventory: pyspark.sql.connect.dataframe.DataFrame = [snapshot_date: date, material_id: string ... 2 more fields]
> df_materials: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, material_name: string ... 5 more fields]
> df_production: pyspark.sql.connect.dataframe.DataFrame = [production_order_id: string, production_date: date ... 3 more fields]
> df_purchase: pyspark.sql.connect.dataframe.DataFrame = [po_id: string, po_date: date ... 6 more fields]
> df_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
> df_suppliers: pyspark.sql.connect.dataframe.DataFrame = [supplier_id: string, supplier_name: string ... 2 more fields]
All 6 files loaded successfully

```

Figure 5.7 Output Shows All 6 Files Loaded.

5.3.2 Removing Duplicates

The second step applied `dropDuplicates()` across all dataframes to eliminate any fully repeated rows that may have been introduced during the manual upload of the source files into the Bronze container. The operation completed across all datasets and the output confirmed “Duplicates removed.” Since the six source CSV files were well-structured synthetic datasets, no duplicate rows were detected in this run, which confirmed that the Bronze layer has preserved the original source data without introducing redundancies before passing it to the Silver cleaning stage.

```

18 hours ago (<1s) 4
from pyspark.sql.functions import col, to_date, upper, trim

# Remove duplicates from all 6 datasets
df_materials = df_materials.dropDuplicates()
df_inventory = df_inventory.dropDuplicates()
df_purchase = df_purchase.dropDuplicates()
df_production = df_production.dropDuplicates()
df_suppliers = df_suppliers.dropDuplicates()
df_sales = df_sales.dropDuplicates()

print("Duplicates removed")

```

Figure 5.8 Remove Duplicates from All Datasets.

```

> df_inventory: pyspark.sql.connect.dataframe.DataFrame = [snapshot_date: date, material_id: string ... 2 more fields]
> df_materials: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, material_name: string ... 5 more fields]
> df_production: pyspark.sql.connect.dataframe.DataFrame = [production_order_id: string, production_date: date ... 3 more fields]
> df_purchase: pyspark.sql.connect.dataframe.DataFrame = [po_id: string, po_date: date ... 6 more fields]
> df_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
> df_suppliers: pyspark.sql.connect.dataframe.DataFrame = [supplier_id: string, supplier_name: string ... 2 more fields]
Duplicates removed

```

Figure 5.9 Output shows Duplicates Removed.

5.3.3 Handling Null Values

The third step addressed missing values in the six datasets using `fillna()` with values appropriate to each column’s role in the downstream Gold business logic. Numeric columns including `unit_cost`, `reorder_level`, `quantity_received`, `quantity_on_hand`, `quantity_consumed`, `reliability_rating`, and `quantity_ordered` were filled with 0 or 0.0 to prevent errors in the Gold layer calculations such as the twelve-month consumption total, the excess quantity computation, and the provision amount formula. Categorical columns including `country`, `category`, and `status` were filled with ‘UNKNOWN’ to preserve row

completeness without distorting the Magna Carta classification or stockout risk grouping logic. The validation output confirmed that only one null remained across all datasets, specifically in the `actual_delivery_date` column of the purchase orders. This was as expected behavior because purchase orders that had not yet been delivered at the time of December 2025 inventory snapshot would naturally have no recorded actual delivery date.



```

# Fill nulls with appropriate defaults
# Materials
df_materials = df_materials.fillna({
    "category": "UNKNOWN",
    "unit_cost": 0.0,
    "reorder_level": 0
})

# Inventory snapshot
df_inventory = df_inventory.fillna({
    "quantity_on_hand": 0
})

# Purchase orders
df_purchase = df_purchase.fillna({
    "quantity_received": 0
})

# Production orders
df_production = df_production.fillna({
    "quantity_consumed": 0
})

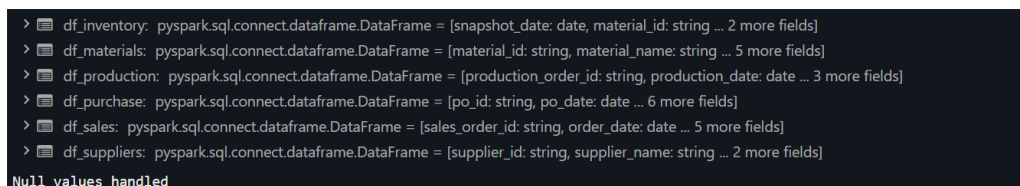
# Suppliers
df_suppliers = df_suppliers.fillna({
    "reliability_rating": 0.0,
    "country": "UNKNOWN"
})

# Sales orders
df_sales = df_sales.fillna({
    "quantity_ordered": 0,
    "status": "UNKNOWN"
})

print("Null values handled")

```

Figure 5.10 Fill Nulls with Appropriate Defaults.



```

> df_inventory: pyspark.sql.connect.dataframe.DataFrame = [snapshot_date: date, material_id: string ... 2 more fields]
> df_materials: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, material_name: string ... 5 more fields]
> df_production: pyspark.sql.connect.dataframe.DataFrame = [production_order_id: string, production_date: date ... 3 more fields]
> df_purchase: pyspark.sql.connect.dataframe.DataFrame = [po_id: string, po_date: date ... 6 more fields]
> df_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
> df_suppliers: pyspark.sql.connect.dataframe.DataFrame = [supplier_id: string, supplier_name: string ... 2 more fields]
Null values handled

```

Figure 5.11 Output Shows Null Values Handled

5.3.4 Fixing Data Types

The fourth function standardized each date column across the involved datasets to the yyyy-

mm-dd format by using the PySpark's `to_date()` function. The affected columns include `snapshot_date` in the inventory snapshot, `po_date`, `expected_delivery_date`, and `actual_delivery_date` in the purchase orders, `production_date` in the production orders, and `order_date` and `required_delivery_date` in the sales orders. This step was very important as date columns which remain as strings cannot be used reliably in the Silver layer's downstream date-based comparisons, particularly the lot expiry checks, and the nine-month consumption gap calculations required for the Magna Carta classification logic.

```

18 hours ago (1s) 6
# Fix date columns to proper date format
df_inventory = df_inventory.withColumn(
    "snapshot_date", to_date(col("snapshot_date"), "yyyy-MM-dd")
)

df_purchase = df_purchase.withColumn(
    "po_date", to_date(col("po_date"), "yyyy-MM-dd")
).withColumn(
    "expected_delivery_date", to_date(col("expected_delivery_date"), "yyyy-MM-dd")
).withColumn(
    "actual_delivery_date", to_date(col("actual_delivery_date"), "yyyy-MM-dd")
)

df_production = df_production.withColumn(
    "production_date", to_date(col("production_date"), "yyyy-MM-dd")
)

df_sales = df_sales.withColumn(
    "order_date", to_date(col("order_date"), "yyyy-MM-dd")
).withColumn(
    "required_delivery_date", to_date(col("required_delivery_date"), "yyyy-MM-dd")
)

print("Data types fixed")

```

Figure 5.12 Fix Data Columns to Proper Date Format.

```

> df_inventory: pyspark.sql.connect.dataframe.DataFrame = [snapshot_date: date, material_id: string ... 2 more fields]
> df_production: pyspark.sql.connect.dataframe.DataFrame = [production_order_id: string, production_date: date ... 3 more fields]
> df_purchase: pyspark.sql.connect.dataframe.DataFrame = [po_id: string, po_date: date ... 6 more fields]
> df_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
Data types fixed

```

Figure 5.13 Output Shows Data Types Fixed.

5.3.5 Standardizing String Formats

The fifth function applied the `upper()` and `trim()` to all key string columns across all three datasets. In `df_materials`, the `category` and `material_name` columns were both converted to uppercase with leading whitespace removed. The same treatment was then applied to the `country` and `supplier_name` in `df_suppliers`, and to `status` and `customer_name` in `df_sales`. This made sure that string-based filtering operations in the Gold layer like grouping by material category or filtering by order status produce consistent results no matter the original data was entered.

```

18 hours ago (1s) 7

# Standardise string casing
df_materials = df_materials.withColumn(
    "category", upper(trim(col("category")))
).withColumn(
    "material_name", upper(trim(col("material_name")))
)

df_suppliers = df_suppliers.withColumn(
    "country", upper(trim(col("country")))
).withColumn(
    "supplier_name", upper(trim(col("supplier_name")))
)

df_sales = df_sales.withColumn(
    "status", upper(trim(col("status")))
).withColumn(
    "customer_name", upper(trim(col("customer_name")))
)

print("Formats standardised")

```

Figure 5.14 Standardize String Casing.

```

> df_materials: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, material_name: string ... 5 more fields]
> df_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
> df_suppliers: pyspark.sql.connect.dataframe.DataFrame = [supplier_id: string, supplier_name: string ... 2 more fields]
Formats standardised

```

Figure 5.15 Output Shows The Formats Are Standardized.

5.3.6 Final Validation and Writing to Silver Layer

The final validation checked row counts and the null values of all cleaned datasets before writing. The results confirmed 20 materials, 240 inventory records, 40 purchase orders, 336 production orders, 10 suppliers, and 48 sales orders, all with zero nulls in critical columns except for the expected `actual_delivery_date` field in purchase orders. All of the 6 cleaned dataframes were written to the Silver container in the ADLS Gen2 as Parquet files using `write.mode("overwrite").parquet()`, with each dataset stored in its own subfolder. The operation completed successfully and the Silver container was then verified in the Azure portal showing every six folders which are `inventory_snapshot`, `materials`, `production_orders`, `purchase_orders`, `sales_orders`, and `suppliers` where each containing a part file, a `_SUCCESS` marker, and the associated Spark metadata files confirming a clean write.

```

18 hours ago (29s) 8

# Final validation - check row counts and null counts
datasets = {
    "materials": df_materials,
    "inventory": df_inventory,
    "purchase_orders": df_purchase,
    "production_orders": df_production,
    "suppliers": df_suppliers,
    "sales_orders": df_sales
}

for name, df in datasets.items():
    print(f"\n--- {name} ---")
    print(f"Row count: {df.count()}")
    null_counts = {c: df.filter(col(c).isNull()).count() for c in df.columns}
    print(f"Null counts: {null_counts}")

```

▶ (62) Spark Jobs

Figure 5.16 Final Validation.

```

> df: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]

--- materials ---
Row count: 20
Null counts: {'material_id': 0, 'material_name': 0, 'category': 0, 'unit_of_measure': 0, 'reorder_level': 0, 'lead_time_days': 0, 'unit_cost': 0}

--- inventory ---
Row count: 240
Null counts: {'snapshot_date': 0, 'material_id': 0, 'warehouse_location': 0, 'quantity_on_hand': 0}

--- purchase_orders ---
Row count: 40
Null counts: {'po_id': 0, 'po_date': 0, 'supplier_id': 0, 'material_id': 0, 'quantity_ordered': 0, 'quantity_received': 0, 'expected_delivery_date': 0, 'actual_delivery_date': 1}

--- production_orders ---
Row count: 336
Null counts: {'production_order_id': 0, 'production_date': 0, 'finished_product': 0, 'material_id': 0, 'quantity_consumed': 0}

--- suppliers ---
Row count: 10
Null counts: {'supplier_id': 0, 'supplier_name': 0, 'country': 0, 'reliability_rating': 0}

--- sales_orders ---
Row count: 48
Null counts: {'sales_order_id': 0, 'order_date': 0, 'customer_name': 0, 'finished_product': 0, 'quantity_ordered': 0, 'required_delivery_date': 0, 'status': 0}

```

Figure 5.17 Output Shows Final Validation Row Counts and Null Counts.

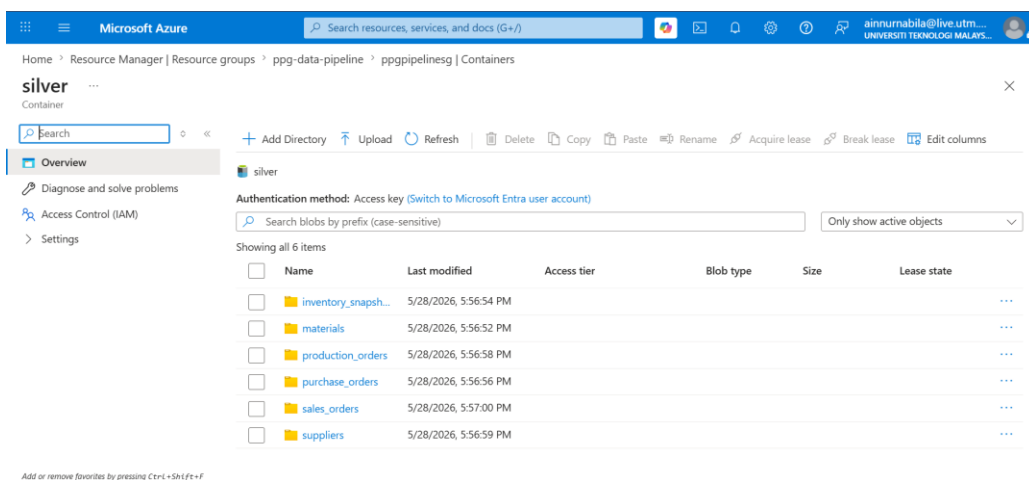


Figure 5.18 Silver Container Showing All 6 Folders.

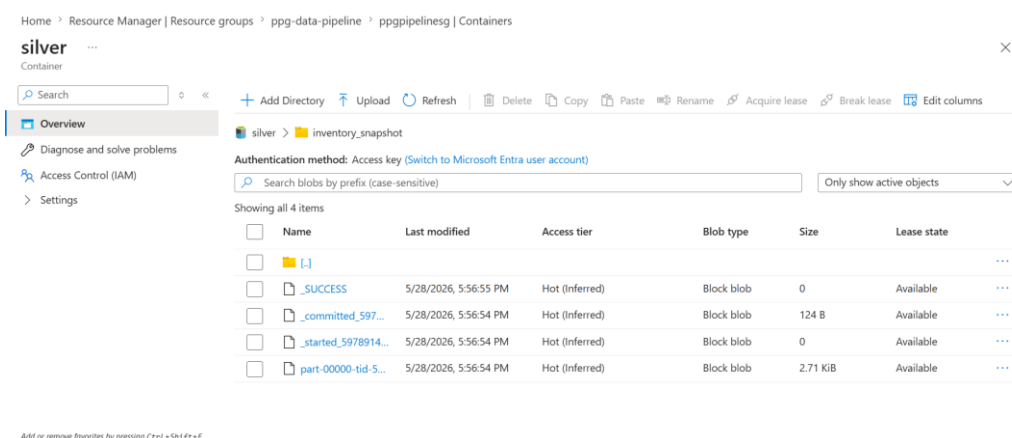


Figure 5.19 Inside One Folder Showing Parquet Files.

5.4 Gold Layer

The Gold layer focuses on transforming the cleaned Silver datasets into business-ready outputs that can directly support analysis and dashboard reporting. In this stage, the data was no longer only cleaned, but was also enhanced with business logic such as Recoverable Asset classification, Magna Carta Expired and Dead Stock identification, stockout risk detection, production fulfillment checking, and sales order impact analysis. The outputs produced in this layer were stored in the Gold container as Parquet files and used as the main foundation for answering the project's inventory risk management questions.

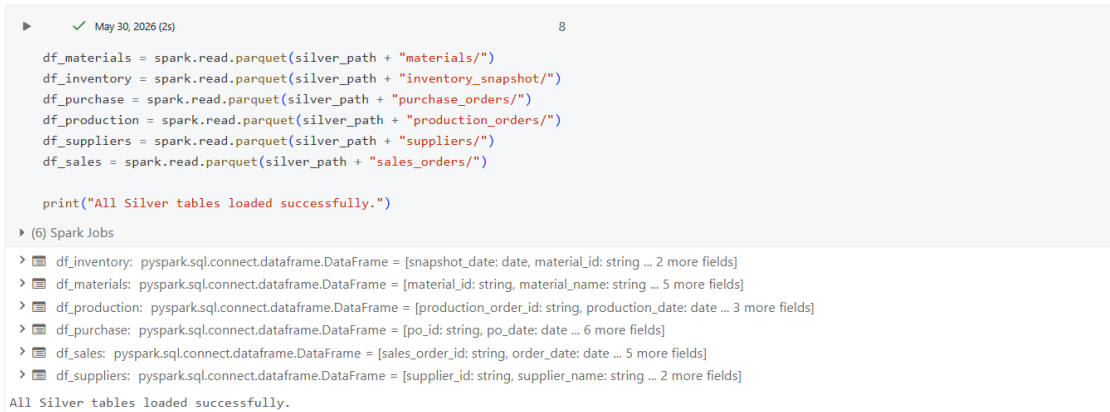
5.4.1 Loading Silver Data

The first task in creating the Gold layer was to load the cleaned data from the Silver container into Databricks data frames. Since the Silver layer had already cleaned and stored the datasets in Azure Data Lake Storage Gen2 as Parquet files, the Gold notebook used PySpark's `spark.read.parquet()` method to read each dataset. The six Silver datasets that were loaded were materials, inventory snapshot, purchase orders, production orders, suppliers, and sales orders. The datasets were loaded into separate dataframes: `df_materials`, `df_inventory`, `df_purchase`, `df_production`, `df_suppliers`, and `df_sales`. This verified that the Gold layer could access the cleaned Silver outputs without any issues before applying the business transformation logic.

Figure 5.20 shows the Gold notebook successfully loading all six cleaned Silver Parquet datasets into separate Databricks dataframes before applying the Gold layer business

transformation logic.

Step 4 — Read the Silver Parquet tables



```
df_materials = spark.read.parquet(silver_path + "materials/")
df_inventory = spark.read.parquet(silver_path + "inventory_snapshot/")
df_purchase = spark.read.parquet(silver_path + "purchase_orders/")
df_production = spark.read.parquet(silver_path + "production_orders/")
df_suppliers = spark.read.parquet(silver_path + "suppliers/")
df_sales = spark.read.parquet(silver_path + "sales_orders/")

print("All Silver tables loaded successfully.")
```

▶ (6) Spark Jobs

- > df_inventory: pyspark.sql.connect.dataframe.DataFrame = [snapshot_date: date, material_id: string ... 2 more fields]
- > df_materials: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, material_name: string ... 5 more fields]
- > df_production: pyspark.sql.connect.dataframe.DataFrame = [production_order_id: string, production_date: date ... 3 more fields]
- > df_purchase: pyspark.sql.connect.dataframe.DataFrame = [po_id: string, po_date: date ... 6 more fields]
- > df_sales: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
- > df_suppliers: pyspark.sql.connect.dataframe.DataFrame = [supplier_id: string, supplier_name: string ... 2 more fields]

All Silver tables loaded successfully.

Figure 5.20 Loading Silver Parquet Files into Databricks

5.4.2 Setting Up Gold Layer Paths

The Silver container path was loaded with the Silver data, and the Gold path was set as the output path for all business-ready datasets. The transformed results from the Silver data were stored in the Gold container in Azure Data Lake Storage Gen2. The results of the transformation performed on the Silver data were stored in the Gold container in Azure Data Lake Storage Gen2. The path that Gold is written to is `abfss://gold@ppgppipelinesg.dfs.core.windows.net`. This methodology enabled the notebook to log each Gold output into its own sub-folder, making it easier to find, check, and utilize for the next Curated layer. The Gold layer was not registered, as it required a Unity Catalogue external location for this operation. For this reason, the Gold datasets were directly validated as Parquet files from the Gold container.

Figure 5.21 shows the Gold container path being configured and tested in Databricks to confirm that the Gold storage location exists before writing the business-ready output files.

Step 3 — Test whether Gold container exists

```
▶ ✓ May 30, 2026 (<1s) 6

gold_path = "abfss://gold@ppgppipelinesg.dfs.core.windows.net/"

files = dbutils.fs.ls(gold_path)
print(files)
```

Figure 5.21 Gold Container Path Configuration

5.4.3 Calculating Material Consumption

The `material_consumption` output was created with the first Gold transformation. This table was created from the cleaned production orders dataset. The aim of this transformation was to determine the total consumption amount for each material over the most recent 12-month period and to find the latest consumption date for each material. The production order data has been aggregated by `material_id`, and the `quantity_consumed` column has been added to obtain the `consumption_12_months`. The latest production date was also retrieved as `last_consumption_date`. This information was needed because the Recoverable Assets logic is based on comparing the stock on hand with the 12-month consumption, while the Magna Carta Dead Stock logic is based on finding materials that have not been consumed in the last 12 months. This resulted in a dataframe that was saved to the Gold container within the `material_consumption` folder.

Figure 5.22 shows the Gold layer transformation that calculates each material's 12-month consumption and latest consumption date from the cleaned production orders data before saving the result into the Gold container.

```
▶ ✓ May 30, 2026 (2s) 16

gold_material_consumption = (
    df_production
    .filter(F.col("production_date") >= F.add_months(F.current_date(), -12))
    .groupBy("material_id")
    .agg(
        F.sum("quantity_consumed").alias("consumption_12_months"),
        F.max("production_date").alias("last_consumption_date")
    )
    .withColumn(
        "months_since_last_consumption",
        F.months_between(F.current_date(), F.col("last_consumption_date"))
    )
)

display(gold_material_consumption)

gold_material_consumption.write.mode("overwrite").parquet(
    gold_path + "material_consumption/"
)

▶ (4) Spark Jobs

> gold_material_consumption: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, consumption_12_months: long ... 2 more fields]
```

Figure 5.22 *Material Consumption Calculation*

5.4.4 Creating Inventory Status Table

The output `inventory_status` is a result of the main Gold transformation. This table took the latest snapshot of the inventory, material master data, and material consumption data and turned them into a business-ready table. A window function was used to choose the latest inventory record for each material, based on the most recent `snapshot_date`. The dataframe was then merged with the materials dataset using `material_id` to add material information such as material name, category, unit cost, reorder level, etc. It was then merged with the material consumption table to add `consumption_12_months`, `last_consumption_date`, and `months_since_last_consumption`. This table became the foundation of the Gold output as it had the business flags required to answer the primary Gold inventory risk questions, such as below reorder level, Recoverable Assets, Magna Carta Expired, Magna Carta Dead Stock, excess quantity, excess value, and provision amount.

Figure 5.23 shows the creation of the `inventory_status` Gold output by selecting the latest inventory snapshot for each material and joining it with material master data and 12-month consumption results.



```
▶ ✓ May 30, 2026 (5s) 20

w = Window.partitionBy("material_id").orderBy(F.col("snapshot_date").desc())

latest_inventory = (
    df_inventory
    .withColumn("rn", F.row_number().over(w))
    .filter(F.col("rn") == 1)
    .drop("rn")
)

gold_inventory_status = (
    latest_inventory
    .join(df_materials, on="material_id", how="left")
    .join(gold_material_consumption, on="material_id", how="left")
)
```

Figure 5.23 *Inventory Status Gold Transformation*

5.4.5 Applying Recoverable Assets and Reorder Risk Logic

The Gold layer then used business rules to categorize materials according to inventory risk. If the `quantity_on_hand` for a material was less than the `reorder_level`, the material was marked as below reorder level. This resulted in the `is_below_reorder_level` flag, which can

be used in the dashboard question to check whether there are materials at stockout risk. The amount of Recoverable Assets was determined by comparing the current stock on hand with the material consumption over a 12-month period. The `is_recoverable_asset` column was set to True if `quantity_on_hand` is more than `consumption_12_months`, indicating that the material is a Recoverable Asset. The surplus was then the difference between the stock on hand and consumption for the past 12 months. The `excess_value` was calculated by multiplying the excess quantity by the material unit cost. The calculations enabled the Gold layer to identify materials that could have an overstock risk and estimate the monetary value of the overstock.

Figure 5.24 shows the Gold layer business logic used to flag materials below reorder level, identify Recoverable Assets based on 12-month consumption, and calculate the related excess quantity and excess value.

```
gold_inventory_status = (
    gold_inventory_status
    .fillna({
        "consumption_12_months": 0,
        "quantity_on_hand": 0,
        "unit_cost": 0,
        "reorder_level": 0
    })
    .withColumn(
        "is_below_reorder_level",
        F.when(F.col("quantity_on_hand") < F.col("reorder_level"), 1).otherwise(0)
    )
    .withColumn(
        "is_recoverable_asset",
        F.when(F.col("quantity_on_hand") > F.col("consumption_12_months"), 1).otherwise(0)
    )
    .withColumn(
        "excess_quantity",
        F.when(
            F.col("quantity_on_hand") > F.col("consumption_12_months"),
            F.col("quantity_on_hand") - F.col("consumption_12_months")
        ).otherwise(0)
    )
    .withColumn(
        "excess_value",
        F.col("excess_quantity") * F.col("unit_cost")
    )
)
```

Figure 5.24 Recoverable Asset and Reorder Risk Logic

5.4.6 Applying Magna Carta Classification Logic

In addition, the `inventory_status` table was also implemented using the Magna Carta classification logic. The original dataset did not have usable material expiry dates, so some dummy expiry dates were added for selected materials according to the supervisor's advice. The following procedures were used to assign expired or future dates to some materials and null expiry dates to the others: all materials with expired dates were assigned dates to be checked at that time and later, and all materials with future dates were assigned dates to be checked in the future and later, while the remaining materials had null expiry dates to

maintain the Dead Stock checking logic. The lot_expiry_date was found to be before the snapshot_date, which identified Magna Carta Expired. Materials such as Toluene, Butanol, and Acetone were correctly identified as expired, as the lot expiry date was set to 2025-06-30 and the inventory snapshot date was set to 2025-12-31. Materials that had not been consumed in nine months or more and had no lot expiry date were defined as Magna Carta Dead Stock. The validation result showed that none of the materials met the Dead Stock condition and returned the is_mc_dead_stock result with zero rows. This was considered an acceptable result from the available data. The provision amount was calculated for materials classified as Magna Carta Expired and Magna Carta Dead Stock by using quantity_on_hand times unit_cost.

Figure 5.25 shows the Magna Carta classification logic used to flag expired materials and dead stock materials, then calculate the provision amount for materials that meet either Magna Carta condition.

```

.withColumn(
  "is_mc_expired",
  F.when(
    F.col("lot_expiry_date").isNotNull() &
    (F.col("lot_expiry_date") < F.current_date()),
    1
  ).otherwise(0)
)
.withColumn(
  "is_mc_dead_stock",
  F.when(
    (F.col("months_since_last_consumption") >= 9) &
    (F.col("lot_expiry_date").isNull()),
    1
  ).otherwise(0)
)

.withColumn(
  "mc_provision_amount",
  F.when(
    (F.col("is_mc_expired") == 1) | (F.col("is_mc_dead_stock") == 1),
    F.col("quantity_on_hand") * F.col("unit_cost")
  ).otherwise(0)
)

display(gold_inventory_status)

gold_inventory_status.write.mode("overwrite").parquet(
  gold_path + "inventory_status/"
)

```

Figure 5.25 Magna Carta Expired and Dead Stock Classification

5.4.7 Creating Stockout Risk Output

The stockout_risk Gold output was derived by filtering the inventory_status table to create a set of materials that had is_below_reorder_level set to 1. This table only reported materials that were below the reorder level. The shortage quantity was determined by subtracting quantity_on_hand from reorder_level. The end result provided significant columns such as Material ID, Material Name, Category, Quantity on Hand, Reorder Level,

Shortage Quantity, and Unit Cost. This is the output data needed to answer the dashboard question: What materials are currently below their reorder level?

Figure 5.26 shows the creation of the `stockout_risk` Gold output by filtering materials below the reorder level and calculating the shortage quantity for each affected material.



```
gold_stockout_risk = (
    gold_inventory_status
    .filter(F.col("is_below_reorder_level") == 1)
    .withColumn(
        "shortage_quantity",
        F.col("reorder_level") - F.col("quantity_on_hand")
    )
    .select(
        "material_id",
        "material_name",
        "category",
        "quantity_on_hand",
        "reorder_level",
        "shortage_quantity",
        "unit_cost"
    )
)

display(gold_stockout_risk)

gold_stockout_risk.write.mode("overwrite").parquet(
    gold_path + "stockout_risk/"
)
```

Figure 5.26 Stockout Risk Gold Output

5.4.8 Creating Magna Carta Provision Summary

The `mc_provision_summary` Gold output was created by aggregating the Magna Carta provision values from the `inventory_status` table. This summary calculates the total MC provision amount, expired material provision, dead stock provision, RA excess value, and the number of materials in each risk category. The output is used to support the dashboard question related to the total financial impact of Magna Carta materials and provides a high-level summary of inventory risk exposure.

Figure 5.27 shows the creation of the `mc_provision_summary` Gold output by aggregating provision values and material counts from the `inventory_status` table for Magna Carta and inventory risk reporting.

```

May 30, 2026 (4s) 24

gold_mc_provision_summary = (
  gold_inventory_status
    .agg(
      F.sum("mc_provision_amount").alias("total_mc_provision"),
      F.sum(
        F.when(F.col("is_mc_dead_stock") == 1, F.col("mc_provision_amount")).otherwise(0)
      ).alias("mc_dead_stock_provision"),
      F.sum(
        F.when(F.col("is_mc_expired") == 1, F.col("mc_provision_amount")).otherwise(0)
      ).alias("mc_expired_provision"),
      F.countDistinct(
        F.when(F.col("is_recoverable_asset") == 1, F.col("material_id"))
      ).alias("recoverable_asset_material_count"),
      F.countDistinct(
        F.when(F.col("is_mc_dead_stock") == 1, F.col("material_id"))
      ).alias("mc_dead_stock_material_count"),
      F.countDistinct(
        F.when(F.col("is_mc_expired") == 1, F.col("material_id"))
      ).alias("mc_expired_material_count"),
      F.countDistinct(
        F.when(F.col("is_below_reorder_level") == 1, F.col("material_id"))
      ).alias("stockout_risk_material_count")
    )
)

display(gold_mc_provision_summary)

gold_mc_provision_summary.write.mode("overwrite").parquet(
  gold_path + "mc_provision_summary/"
)

```

▶ (14) Spark Jobs

▶ gold_mc_provision_summary: pyspark.sql.connect.dataframe.DataFrame = [total_mc_provision: double, mc_dead_stock_provision: double ... 5 more fields]

Figure 5.27 *Magna Carta Provision Summary Output*

5.4.9 Creating Production Fulfilment Output

The production_fulfillment output is generated to find production orders that cannot be fulfilled because of a lack of material stock. Material_id was used to join the production orders dataset to the most recent inventory data. The needed quantity of material was compared to the quantity on hand. If the required quantity or consumed quantity was more than the available quantity, a shortage quantity was calculated, and the production order was marked as "INSUFFICIENT STOCK". If not, the order was considered FULFILLABLE. This output will assist in answering the business inquiry on whether there were any production orders that were unable to be completed because of inadequate stock. In the validation results, the quantity on hand for certain materials in several production orders was either zero or less than the quantity consumed, which led to a shortage situation.

Figure 5.28 shows the creation of the 'production_fulfillment' Gold output by joining production orders with the latest inventory data, calculating shortage quantity, and assigning each order a fulfillment status based on material availability.


```

gold_production_fulfilment = (
  df_production
    .join(
      latest_inventory.select("material_id", "quantity_on_hand"),
      on="material_id",
      how="left"
    )
    .withColumn(
      "shortage_quantity",
      F.when(
        F.col("required_col") > F.col("quantity_on_hand"),
        F.col("required_col") - F.col("quantity_on_hand")
      ).otherwise(0)
    )
    .withColumn(
      "fulfilment_status",
      F.when(F.col("shortage_quantity") > 0, "INSUFFICIENT STOCK")
      .otherwise("FULFILLABLE")
    )
)

display(gold_production_fulfilment)

gold_production_fulfilment.write.mode("overwrite").parquet(
  gold_path + "production_fulfilment/"
)

```

▶ (6) Spark Jobs

▶ gold_production_fulfilment: rucspark.col.manant.DataFrame - finished_col: string, production_order_id: string, amount: float

Figure 5.28 *Production Fulfilment Gold Output*

5.4.10 Creating Sales Order Impact Output

The sales_order_impact output is designed to track material issues that may affect open customer sales orders. The dataset was found to be missing product_id in the database during development. On the contrary, both the sales orders and the production orders datasets had the finished_product column. So the join relationship was modified to use finished_product. The sales orders dataset was initially filtered to include only sales orders with an OPEN status. These open sales orders were then merged with production orders with the help of finished_product. The output was then merged with the stockout_risk output based on material_id. This enabled the Gold layer to map customer orders to completed products and then to stockout-prone raw materials. A new column, is_customer_affected, was added to show whether the sales order was affected by material shortage or not. This output directly contributes to the most crucial business question: Which sales orders are open, and which customers will be impacted by stockout risk?

Figure 5.29 shows the creation of the `sales\_order\_impact` Gold output by linking open sales orders to related production orders and stockout-risk materials to identify which customer orders may be affected by material shortages.

```

open_sales_orders = df_sales.filter(F.col("status") == "OPEN")

if "product_id" in df_sales.columns and "product_id" in df_production.columns:
    gold_sales_order_impact = (
        open_sales_orders
        .join(
            df_production.select(
                "production_order_id",
                "product_id",
                "material_id"
            ).dropDuplicates(),
            on="product_id",
            how="left"
        )
        .join(
            gold_stockout_risk.select(
                "material_id",
                "material_name",
                "shortage_quantity"
            ),
            on="material_id",
            how="inner"
        )
        .withColumn(
            "is_customer_affected",
            F.when(F.col("shortage_quantity") > 0, 1).otherwise(0)
        )
    )

    display(gold_sales_order_impact)

    gold_sales_order_impact.write.mode("overwrite").parquet(
        gold_path + "sales_order_impact/"
    )

else:
    print("Cannot build sales_order_impact because product_id is missing from sales_orders or production_orders.")

> open_sales_orders: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, order_date: date ... 5 more fields]
Cannot build sales_order_impact because product_id is missing from sales_orders or production_orders.

```

Figure 5.29 Sales Order Impact Gold Output

5.4.11 Final Validation and Writing to Gold Layer

The final validation step checked that all required Gold outputs were successfully written to the Gold container. The Gold container contained six business-ready folders: material_consumption, inventory_status, stockout_risk, mc_provision_summary, production_fulfillment, and sales_order_impact. Each Gold output was validated by reading the Parquet files directly from the Gold container using `spark.read.parquet()`. The validation queries confirmed that the Gold layer could answer the required business questions. The inventory_status table returned materials below the reorder level and Recoverable Assets. The Magna Carta Expired query returned three expired materials based on the dummy expiry dates. The Magna Carta Dead Stock query returned zero rows, which was consistent with the current dataset conditions. The mc_provision_summary output provided the total provision values, while the production_fulfillment and sales_order_impact outputs showed production shortages and affected open sales orders. Overall, the Gold layer successfully transformed the cleaned Silver data into business-ready outputs for inventory risk analysis, Magna Carta classification, financial provision calculation, production fulfillment checking, and customer impact tracking.

Figure 5.30 shows that all six Gold business-ready output folders were successfully written to the Gold container in Azure Data Lake Storage Gen2 for downstream validation and Curated layer processing.

Name	Last modified	Access tier	Blob type	Size	Lease state
inventory_status	30/05/2026, 18:09:01				...
material_consumption	30/05/2026, 18:08:47				...
mc_provision_summary	30/05/2026, 18:09:45				...
production_fulfilment	30/05/2026, 18:09:52				...
sales_order_impact	30/05/2026, 18:18:09				...
stockout_risk	30/05/2026, 18:09:23				...

Figure 5.30 Gold Container Output Folders

5.5 Curated Layer

The Curated layer represents the final and most refined stage of the data pipeline, designed to organize the Gold layer outputs into a structured star schema that is optimized for analytical querying and dashboard consumption. This layer was implemented using Azure Databricks for schema construction and Azure Synapse Analytics as the serving layer, where the curated tables were exposed as SQL views for downstream access by Power BI. The star schema consisted of one central fact table, `fact_inventory_status`, and four-dimension tables, namely `dim_material`, `dim_supplier`, `dim_time`, and `dim_sales_order`. All seven business questions were validated against this schema to confirm that the curated layer could support the full scope of the inventory risk analysis required by the project.

5.5.1 Loading Gold Layer Data into Databricks

The first step in building the Curated layer was to load all six business-ready output tables from the Gold container into Azure Databricks. Since the Gold layer had already stored the transformed datasets as Parquet files in Azure Data Lake Storage Gen2, the Curated notebook used PySpark's `spark.read.parquet()` method to read each dataset directly. The Gold container was then listed using `dbutils.fs.ls()` to confirm the available folders before reading. The six Gold datasets loaded were `inventory_status`, `material_consumption`, `mc_provision_summary`, `production_fulfilment`, `sales_order_impact`, and `stockout_risk`. Each dataset was assigned to a separate data frame and verified using a row count check.

The output confirmed that all six tables were successfully loaded, with `inventory_status` returning 20 records, `material_consumption` returning 15 records, `mc_provision_summary` returning 1 record, `production_fulfilment` returning 336 records, `sales_order_impact` returning 192 records, and `stockout_risk` returning 9 records, confirming that the Curated layer could access all Gold outputs before proceeding with the star schema construction. As shown in Figure 5.31, all six Gold Parquet files were successfully read into Databricks data frames with the correct row counts.

```

# CELL 3 - Read all Gold Parquet files
gold_path = "abfss://gold@ppgpipelineeg.dfs.core.windows.net/"

df_inventory_status = spark.read.parquet(gold_path + "inventory_status/")
df_material_consumption = spark.read.parquet(gold_path + "material_consumption/")
df_mc_provision = spark.read.parquet(gold_path + "mc_provision_summary/")
df_production = spark.read.parquet(gold_path + "production_fulfilment/")
df_sales_impact = spark.read.parquet(gold_path + "sales_order_impact/")
df_stockout = spark.read.parquet(gold_path + "stockout_risk/")

# Check row counts
print("inventory_status:", df_inventory_status.count())
print("material_consumption:", df_material_consumption.count())
print("mc_provision_summary:", df_mc_provision.count())
print("production_fulfilment:", df_production.count())
print("sales_order_impact:", df_sales_impact.count())
print("stockout_risk:", df_stockout.count())

(18) Spark Jobs
> df_inventory_status: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, snapshot_date: date ... 19 more fields]
> df_material_consumption: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, consumption_12_months: long ... 2 more fields]
> df_mc_provision: pyspark.sql.connect.dataframe.DataFrame = [total_mc_provision: double, mc_dead_stock_provision: double ... 5 more fields]
> df_production: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, production_order_id: string ... 6 more fields]
> df_sales_impact: pyspark.sql.connect.dataframe.DataFrame = [sales_order_id: string, customer_name: string ... 9 more fields]
> df_stockout: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, material_name: string ... 5 more fields]

inventory_status: 20
material_consumption: 15
mc_provision_summary: 1
production_fulfilment: 336
sales_order_impact: 192
stockout_risk: 9

```

Figure 5.31 Loading Gold Parquet Files into Databricks

5.5.2 Designing and Building the Dimension Table

Four dimension tables were constructed in Azure Databricks to support the star schema design of the Curated layer. Each dimension table was derived from the relevant Gold layer data frames using PySpark's `select()` method to extract only the necessary columns, followed by `dropDuplicates()` to ensure each record was unique.

The first dimension table, `dim_material`, was extracted from the `inventory_status` dataframe and contained eight attributes: `material_id`, `material_name`, `category`, `unit_of_measure`, `unit_cost`, `reorder_level`, `lead_time_days`, and `warehouse_location`. Deduplication was applied on `material_id` as the primary key, resulting in 20 unique material records. The second dimension table, `dim_sales_order`, was derived from the `sales_order_impact` dataframe and included six attributes: `sales_order_id`, `customer_name`, `finished_product`, `quantity_ordered`, `required_delivery_date`, and `status`.

Deduplication was applied to sales_order_id, producing 10 unique sales order records. The third dimension table, dim_time, was built from the snapshot_date column of the inventory_status dataframe. Additional date attributes were derived using PySpark date functions, including year, month, day, quarter, and month_name, to support time-based filtering in the dashboard. The table returned one unique date record corresponding to the inventory snapshot date of 2025-12-31. The fourth dimension table, dim_supplier, was created as a placeholder using a manually defined row, as supplier information was not available in the Gold layer outputs. The table contained four attributes: supplier_id, supplier_name, country, and lead_time_days, and was included to complete the star schema structure. As shown in Figure 5.32 and Figure 5.33, the construction code and resulting outputs for all four dimension tables were verified successfully in Databricks.

```
# === DIM_MATERIAL ===
dim_material = df_inventory_status.select(
    col("material_id"),
    col("material_name"),
    col("category"),
    col("unit_of_measure"),
    col("unit_cost"),
    col("reorder_level"),
    col("lead_time_days"),
    col("warehouse_location")
).dropDuplicates(["material_id"])

print("dim_material:", dim_material.count())
dim_material.show(3)

# === DIM_SALES_ORDER ===
dim_sales_order = df_sales_impact.select(
    col("sales_order_id"),
    col("customer_name"),
    col("finished_product"),
    col("quantity_ordered"),
    col("required_delivery_date"),
    col("status")
).dropDuplicates(["sales_order_id"])

print("dim_sales_order:", dim_sales_order.count())
dim_sales_order.show(3)

# === DIM_TIME ===
# Collect all dates from inventory snapshot
dim_time = df_inventory_status.select(
    col("snapshot_date").alias("date")
).dropDuplicates(["date"]).withColumn(
    "year", year("date")
).withColumn(
    "month", month("date")
).withColumn(
    "day", dayofmonth("date")
).withColumn(
    "quarter", quarter("date")
).withColumn(
    "month_name", date_format("date", "MMMM")
)

print("dim_time:", dim_time.count())
dim_time.show(3)

# === DIM_SUPPLIER (placeholder - not in gold data) ===
from pyspark.sql import Row
supplier_data = [
    Row(supplier_id="S001", supplier_name="Unknown", country="Unknown", lead_time_days=0)
]
dim_supplier = spark.createDataFrame(supplier_data)

print("dim_supplier:", dim_supplier.count())
dim_supplier.show()
```

Figure 5.32 Dimension Tables Construction Code

```
dim_material: 20
+-----+-----+-----+-----+-----+-----+-----+
|material_id|material_name|category|unit_of_measure|unit_cost|reorder_level|lead_time_days|warehouse_location|
+-----+-----+-----+-----+-----+-----+-----+
|MAT004|PHTHALOCYANINE BLUE|PIGMENT|KG|12.6|150|21|WH-D|
|MAT014|ACETONE|SOLVENT|Litre|2.1|400|7|WH-D|
|MAT012|TOLUENE|SOLVENT|Litre|1.6|800|7|WH-B|
+-----+-----+-----+-----+-----+-----+-----+
only showing top 3 rows

dim_sales_order: 10
+-----+-----+-----+-----+-----+-----+
|sales_order_id|customer_name|finished_product|quantity_ordered|required_delivery_date|status|
+-----+-----+-----+-----+-----+-----+
|SO-045|MAYBANK FACILITIE...|Marine Blue Coating|275|2026-01-24|OPEN|
|SO-047|YTL HARDWARE CENTRE|Epoxy Floor Coating|385|2025-12-31|OPEN|
|SO-043|ECO WORLD SUPPLIES|Premium White Pai...|350|2026-02-01|OPEN|
+-----+-----+-----+-----+-----+-----+
only showing top 3 rows

dim_time: 1
+-----+-----+-----+-----+-----+-----+
|date|year|month|day|quarter|month_name|
+-----+-----+-----+-----+-----+-----+
|2025-12-31|2025|12|31|4|December|
+-----+-----+-----+-----+-----+-----+

dim_supplier: 1
+-----+-----+-----+-----+
|supplier_id|supplier_name|country|lead_time_days|
+-----+-----+-----+-----+
|S001|Unknown|Unknown|0|
+-----+-----+-----+-----+
```

Figure 5.33 Dimension Tables Output

5.5.3 Building The Fact Table

The central fact table of the star schema, `fact_inventory_status`, was constructed by selecting fourteen key columns from the `inventory_status` Gold dataframe. These columns included `material_id` and `snapshot_date` as the primary identifiers, along with `quantity_on_hand`, `lot_expiry_date`, `consumption_12_months`, `last_consumption_date`, and `months_since_last_consumption` as the core inventory measures. The business flag columns `is_below_reorder_level`, `is_recoverable_asset`, `is_mc_expired`, and `is_mc_dead_stock` were also included to capture the inventory risk classifications applied during the Gold transformation. Additionally, `excess_quantity`, `excess_value`, and `mc_provision_amount` were retained to support financial provision calculations in the dashboard. The `material_id` column serves as the foreign key, linking the fact table to the `dim_material` dimension table. The output confirmed that the fact table contained 20 records, each representing a unique material's latest inventory position as of the snapshot date 2025-12-31, as shown in Figure 5.34.

```
# CELL 6 - Build Fact Table
fact_inventory_status = df_inventory_status.select(
    col("material_id"),
    col("snapshot_date"),
    col("quantity_on_hand"),
    col("lot_expiry_date"),
    col("consumption_12_months"),
    col("last_consumption_date"),
    col("months_since_last_consumption"),
    col("is_below_reorder_level"),
    col("is_recoverable_asset"),
    col("is_mc_expired"),
    col("is_mc_dead_stock"),
    col("excess_quantity"),
    col("excess_value"),
    col("mc_provision_amount")
)

print("fact_inventory_status:", fact_inventory_status.count())
fact_inventory_status.show(3)
```

(3) Spark Jobs

fact_inventory_status: pyspark.sql.connect.dataframe.DataFrame = [material_id: string, snapshot_date: date ... 12 more fields]

fact_inventory_status: 20

material_id	snapshot_date	quantity_on_hand	lot_expiry_date	consumption_12_months	last_consumption_date	months_since_last_consumption	is_below_reorder_level	is_recoverable_asset	is_mc_expired	is_mc_dead_stock	excess_quantity	excess_value	mc_provision_amount
MAT0001	2025-12-31	380	2026-12-31	7620	2025-12-05	5.80645161	1	0					
MAT0002	2025-12-31	0	NULL	2510	2025-11-12	6.58064516	1	0					
MAT0003	2025-12-31	0	NULL	1960	2025-11-12	6.58064516	1	0					

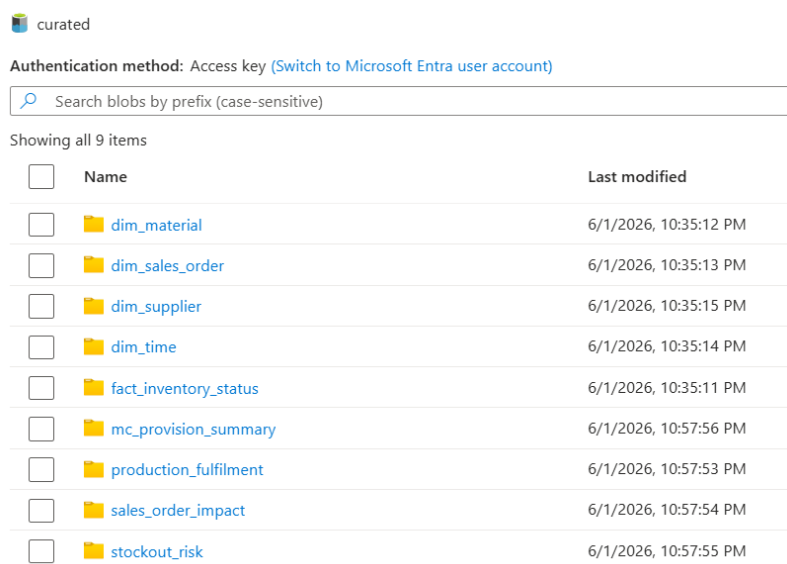
only showing top 3 rows

Figure 5.34 Fact Table Construction and Output

5.5.4 Saving the Curated Tables to ADLS Gen2

Once all dimension and fact tables were constructed in Databricks, they were written to the curated container in Azure Data Lake Storage Gen2 as Parquet files using PySpark's `write.mode("overwrite").parquet()` method. The five star schema tables, `fact_inventory_status`, `dim_material`, `dim_sales_order`, `dim_time`, and `dim_supplier`,

were saved to their respective subfolders within the curated container. An additional four Gold output tables, `production_fulfilment`, `sales_order_impact`, `stockout_risk`, and `mc_provision_summary`, were also saved to the curated container to ensure that all data required to answer the seven business questions was available in a single location for downstream access by Azure Synapse Analytics. As shown in Figure 5.35, the curated container in the `ppgpipelinesg` storage account confirmed that all nine folders were successfully created, with each folder containing the corresponding Parquet files written on 1 June 2026.



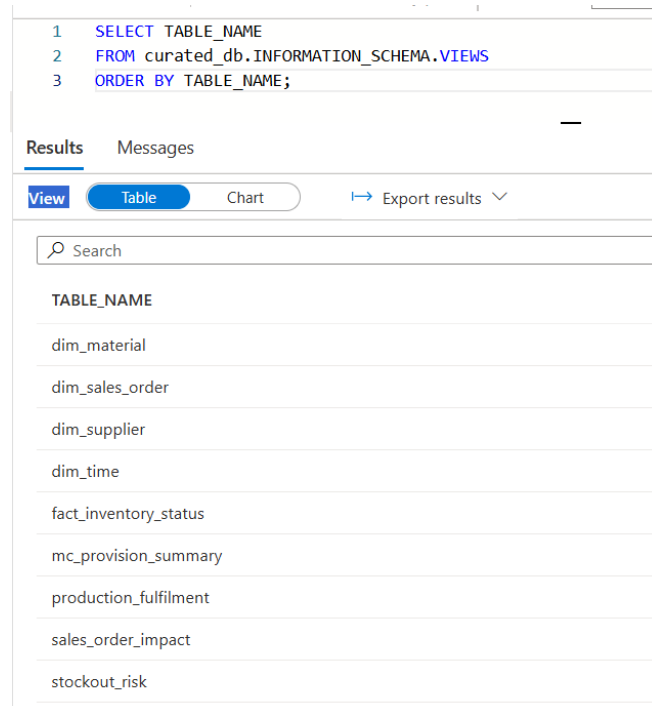
☐	Name	Last modified
☐	dim_material	6/1/2026, 10:35:12 PM
☐	dim_sales_order	6/1/2026, 10:35:13 PM
☐	dim_supplier	6/1/2026, 10:35:15 PM
☐	dim_time	6/1/2026, 10:35:14 PM
☐	fact_inventory_status	6/1/2026, 10:35:11 PM
☐	mc_provision_summary	6/1/2026, 10:57:56 PM
☐	production_fulfilment	6/1/2026, 10:57:53 PM
☐	sales_order_impact	6/1/2026, 10:57:54 PM
☐	stockout_risk	6/1/2026, 10:57:55 PM

Figure 5.35 Curated Container Showing All Nine Output Folders

5.5.5 Creating SQL Views in Azure Synapse Analytics

Following the successful storage of all curated tables in ADLS Gen2, the next step was to expose the data through Azure Synapse Analytics for downstream consumption by Power BI. Since the workspace operates on a serverless SQL pool, the curated tables were registered as SQL views using the `OPENROWSET` function, which reads the Parquet files directly from the curated container without requiring a dedicated SQL pool. A new database named `curated_db` was first created in the serverless pool. A master key and a database scoped credential were then configured using a Shared Access Signature token to authenticate access to the `ppgpipelinesg` storage account. An external data source named `curated_source` was created pointing to the curated container, and a Parquet file format was defined. Nine SQL views were subsequently created, covering all star schema tables and supporting Gold output tables, namely `dim_material`, `dim_sales_order`, `dim_supplier`, `dim_time`, `fact_inventory_status`,

mc_provision_summary, production_fulfilment, sales_order_impact, and stockout_risk. As shown in Figure 5.36, a query against the curated_db information schema confirmed that all nine views were successfully registered and available for querying in Azure Synapse Analytics.



The screenshot shows a SQL query executed in Azure Synapse Analytics. The query is: `SELECT TABLE_NAME FROM curated_db.INFORMATION_SCHEMA.VIEWS ORDER BY TABLE_NAME;`. The results are displayed in a table with one column, `TABLE_NAME`, listing nine views: `dim_material`, `dim_sales_order`, `dim_supplier`, `dim_time`, `fact_inventory_status`, `mc_provision_summary`, `production_fulfilment`, `sales_order_impact`, and `stockout_risk`.

TABLE_NAME
dim_material
dim_sales_order
dim_supplier
dim_time
fact_inventory_status
mc_provision_summary
production_fulfilment
sales_order_impact
stockout_risk

Figure 5.36 SQL Views Registered in Azure Synapse Analytics curated_db

5.5.6 Validating the Fact Table in Azure Synapse Analytics

To confirm that the curated layer was functioning correctly, the `fact_inventory_status` view was queried directly in Azure Synapse Analytics using a `SELECT` statement. The query was executed against the `curated_db` database using the serverless SQL pool. As shown in Figure 5.37, the results returned 20 records, each representing a unique material inventory position as of the snapshot date 2025-12-31. The table displayed all key columns including `material_id`, `snapshot_date`, `quantity_on_hand`, `lot_expiry_date`, `consumption_12_months`, `last_consumption_date`, `months_since_last_consumption`, and the business flag columns `is_below_reorder_level`, `is_recoverable_asset`, `is_mc_expired`, `is_mc_dead_stock`, and `excess_quantity`, confirming that the fact table was correctly structured and accessible through Synapse Analytics for downstream Power BI consumption.

material_id	snapshot_date	quantity_on_h...	lot_expiry_date	consumption_1...	last_consumpti...	months_since_l...	is_below_reord...	is_recoverable...	is_mc_expired	is_mc_dead_st...	excess_quan...
MAT001	2025-12-31	380	2026-12-31	7620	2025-12-05	5.80645161	1	0	0	0	0
MAT002	2025-12-31	0	(NULL)	2510	2025-11-12	6.58064516	1	0	0	0	0
MAT003	2025-12-31	0	(NULL)	1960	2025-11-12	6.58064516	1	0	0	0	0
MAT004	2025-12-31	0	(NULL)	1500	2025-11-12	6.58064516	1	0	0	0	0
MAT006	2025-12-31	0	(NULL)	7160	2025-11-12	6.58064516	1	0	0	0	0
MAT007	2025-12-31	3550	(NULL)	3790	2025-12-05	5.80645161	0	0	0	0	0
MAT008	2025-12-31	240	2026-12-31	5620	2025-11-12	6.58064516	1	0	0	0	0
MAT011	2025-12-31	5895	2026-12-31	4485	2025-11-12	6.58064516	0	1	0	0	1410
MAT012	2025-12-31	7880	2025-06-30	680	2025-10-19	7.35483871	0	1	1	0	7200
MAT013	2025-12-31	0	(NULL)	5040	2025-12-05	5.80645161	1	0	0	0	0
MAT015	2025-12-31	2580	2025-06-30	570	2025-11-12	6.58064516	0	1	1	0	2010

Figure 5.37 *fact_inventory_status*

5.5.7 Validating the Seven Business Questions

The final step of the Curated layer implementation was to validate that all seven business questions defined in the project could be answered directly from the curated star schema views in Azure Synapse Analytics as shown in Table 5.1. Each business question was tested by executing a dedicated SQL query against the relevant view in `curated_db` using the serverless SQL pool.

For Question 1, a query filtering `fact_inventory_status` by `is_below_reorder_level` equal to 1, joined with `dim_material` on `material_id`, returned 9 materials that were currently below their reorder level, confirming that the schema could identify stockout risk materials.

For Question 2, filtering by `is_recoverable_asset` equal to 1 returned 11 materials classified as Recoverable Assets, where the quantity on hand exceeded the 12-month consumption figure.

For Question 3, filtering by `is_mc_dead_stock` equal to 1 returned zero records, which was consistent with the current dataset conditions as no materials met the nine-month no-consumption threshold with a null lot expiry date.

For Question 4, filtering by `is_mc_expired` equal to 1 returned 3 materials, namely Toluene, Acetone, and Butanol, all of which had a lot of expiry date of 2025-06-30, which preceded the inventory snapshot date of 2025-12-31.

For Question 5, aggregating the mc_provision_amount column from fact_inventory_status returned a total Magna Carta provision amount of RM29,720, comprising entirely of expired material provisions as no dead stock materials were identified.

For Question 6, querying the production_fulfilment view and filtering by fulfilment_status equal to INSUFFICIENT STOCK returned 183 production orders that could not be fulfilled due to inadequate material stock levels.

For Question 7, querying the sales_order_impact view and filtering by is_customer_affected equal to 1 returned 192 records of affected customer sales orders, representing open orders linked to materials with stockout risk. The complete validation results are summarised in Table 5.1 below.

Table 5.1: Business Questions Validation Results

Business Question	View Used	Result
Q1: Materials below reorder level	fact_inventory_status, dim_material	9 materials
Q2: Recoverable Assets	fact_inventory_status, dim_material	11 materials
Q3: MC Dead Stock materials	fact_inventory_status	0 materials
Q4: MC Expired materials	fact_inventory_status, dim_material	3 materials
Q5: Total MC provision amount	fact_inventory_status	RM 29720
Q6: Unfulfilled production orders	production_fulfilment	183 orders
Q7: Affected customer sales orders	sales_order_impact	192 orders

5.5.8 Star Schema Design

The Curated layer was designed following a star schema structure to optimize the data for analytical querying in Power BI. The central fact table, fact_inventory_status, holds all measurable inventory attributes and business flags, with material_id and snapshot_date serving as the composite key linking to the surrounding dimension tables.

The dim_material table connects via material_id and provides descriptive attributes for each material such as category, unit cost, reorder level, and warehouse location. The dim_time table connects via snapshot_date and supports time-based filtering with derived attributes including year, month, quarter, and month_name. The dim_sales_order table captures open customer sales order details and links to the fact table through material_id to support stockout impact analysis. The dim_supplier table was included as a structural placeholder to complete the schema design, as supplier data was not available in the Gold layer outputs. As shown in Figure 5.38, all relationships follow a many-to-one direction from the fact table to each dimension table, consistent with standard star schema design principles.

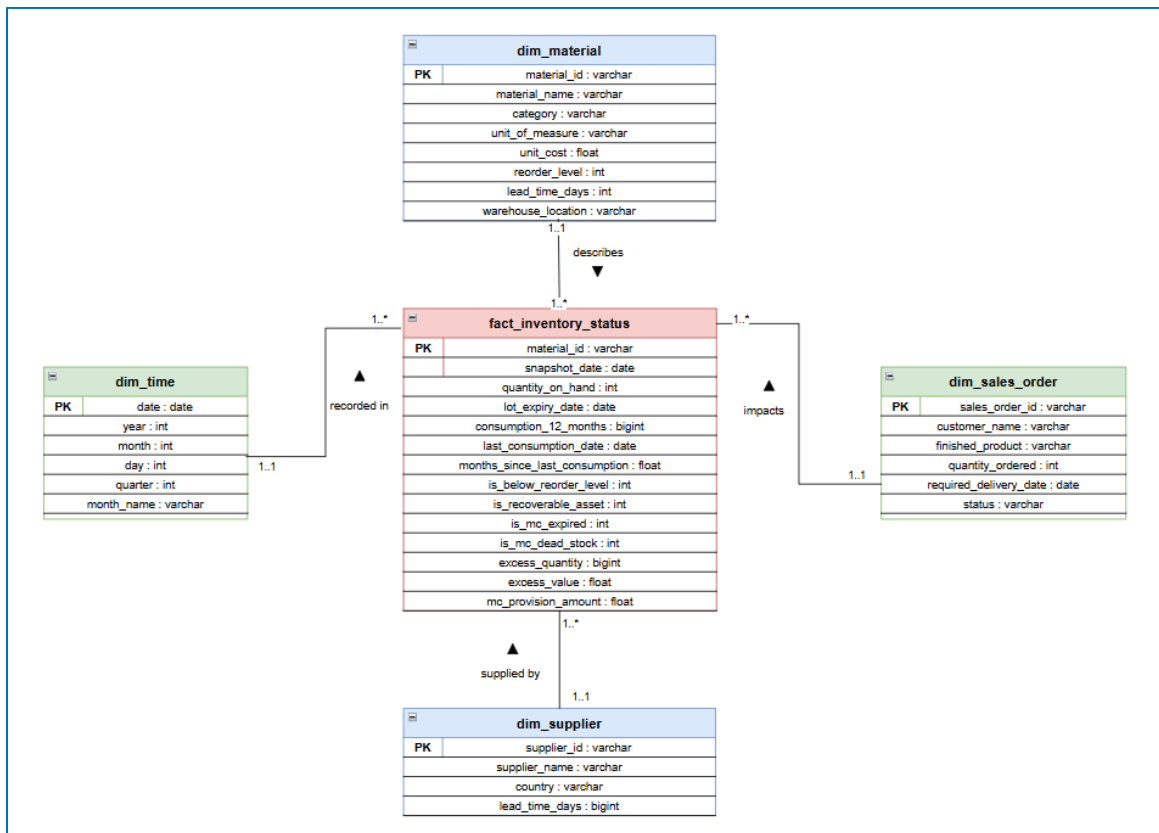


Figure 5.38 Star Schema

5.6 Power BI Dashboard Interface

This section explains how the Microsoft Power BI dashboard was implemented as the end-to-end data pipeline created in this project was finished. The dashboard is directly linked to the Azure Synapse Analytics Curated layer that holds the processed inventory data and visualizes that data to answer seven pre-defined business questions about Recoverable

Assets (RA), Magna Carta (MC) classifications, financial provisions, and stock-out risk. The following subsections will outline the connection setup, data model configuration and design of each dashboard page.

5.6.1 Dashboard Connection Setup

Before creating the dashboard visualizations, Power BI Desktop was connected to the Curated layer database stored in Azure Synapse Analytics. This mapping enables Power BI to get access to a processed data and create meaningful reports.

Step 1 – Connecting to Azure Synapse Analytics

In Power BI Desktop, the connection was made by choosing "Get Data", selecting Azure Synapse Analytics SQL. To load the data into Power BI, the Synapse server address and database name were supplied, and Import Mode was selected. Because the data is kept directly within the Power BI model, this method enhances dashboard performance.

Step 2 – User Authentication

Authentication for the database was done with a Microsoft organizational account that was associated to the Azure environment. Appropriate read permissions were set for the Azure Synapse Analytics account to provide access to the tables in the Curated layer that were needed.

Step 3 – Importing Tables

Once the connection was successfully made, a few tables were loaded into Power BI, the fact tables and dimension tables that were created in the Curated layer. All information needed to respond to the seven business questions defined in the project requirements is available in these tables.

5.6.2 Data Model Configuration

The data model was then configured in Power BI, using the Model View, after the tables were imported. The fact tables and dimension tables were joined together to allow for the correct filtering and analysis of the data on various pages of the dashboard as shown in Figure 5.39. The inventory, stockout risk, production fulfilment and sales order impact

tables were mapped to the material dimension using the material_id field, for instance. The time dimension was also joined to the inventory fact table to enable time based analysis. A many-to-one relationship was used in all relationships with one direction of the relationship filter, in order to keep it efficient and to prevent data ambiguity. Two tables in the model are maintained without active relationships. The dim_supplier table is disconnected because no supplier identifier exists in the fact or dimension table to create a relationship. The mc_provision_summary table is also kept separate because it contains pre-calculated summary values generated in the Gold layer. Several measures were developed to enable calculations on the dashboard and KPI indicators. These measures provide information on key metrics like the number of Recoverable Assets (RA), Magna Carta Dead Stock, Expired Materials, Unfulfilled Production Orders, and Affected Customer Orders. Such measures enable Power BI to dynamically compute and present the most current inventory risk data, aiding users in more effectively monitoring and making informed choices about inventory conditions.

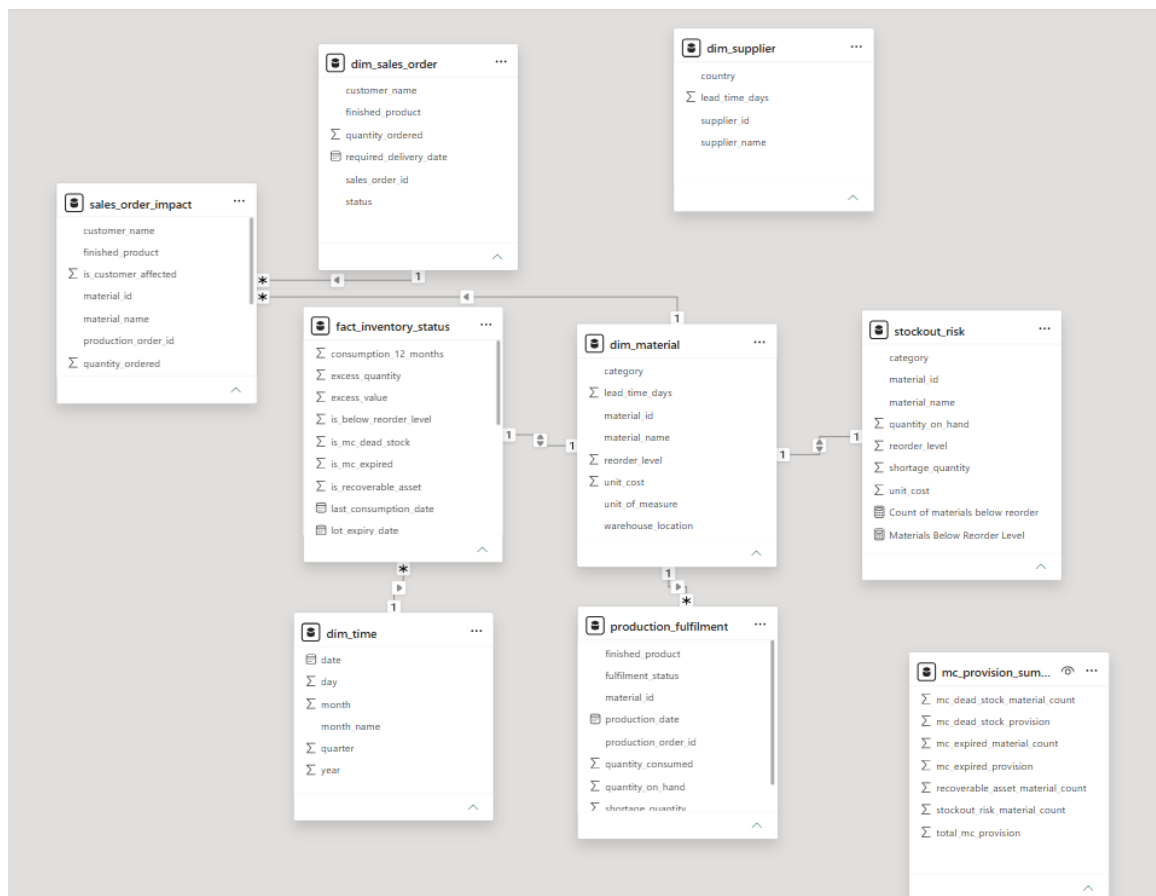


Figure 5.39 Model View Relationship

5.6.3 Dashboard Overview – Executive Summary Home Page

The PPG Inventory Risk Management dashboard's main page is the Executive Summary page. It doesn't just support a navigation page, it's a detailed overview of all seven business questions on one screen. With this, managers or decision makers can rapidly see the total inventory risk situation without having to open any pages of the dashboard. Overall, the page features a professional and consistent corporate look, with the PPG company logo prominently appearing at the top. The page also includes a dark blue header with the title "PPG Inventory Risk Management — Executive Summary," giving the page a professional and consistent corporate appearance. As shown in Figure 5.40, the dashboard consists of seven sections. One business question is reflected in each section, which also includes a KPI card and a small supporting visual. Each section is uniquely coloured and can be clicked to go directly to matching detailed dashboard page. The executive summary dashboard provides an overview of key metrics, visual summaries and navigational elements. This allows management and operational staff to efficiently manage and make better business decisions regarding inventory risks.



Figure 5.40 Executive Summary Home Page — All 7 Business Questions Overview

5.6.4 Question 1: Materials Below Reorder Level

The Q1 page is designed to help identify materials that are currently below the reorder level. Materials that are at risk of stock shortage and will need to be replenished immediately. As shown in Figure 5.41, the page contains a matrix table, a bar chart, and a KPI card. The

matrix table provides information like material ID, material name, category, current stock, reorder level, shortage quantity. Critical items are highlighted in darker red using conditional formatting to make it easier to see that there are larger shortages. The bar chart is used to visually compare the shortage quantities of all the affected materials, and the KPI card displays the total number of materials in shortage at the present moment that are below the reorder level. The results show that ALKYD RESIN records the highest shortage quantity, followed by MINERAL SPIRITS and PLASTIC DRUM 20L. Moreover, five materials are fully depleted, with a high demand to replenish them. The Materials Below Reorder Level dashboard page is used to monitor the inventory balance of materials.

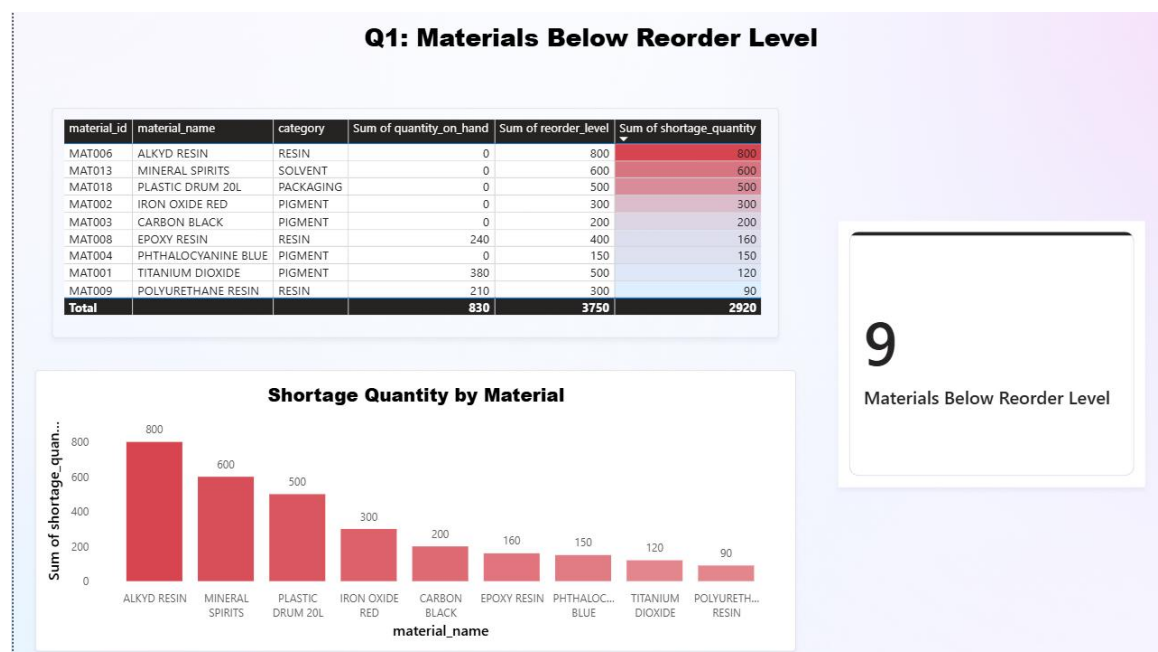


Figure 5.41 Q1 Page – Materials Below Reorder Level Dashboard Page

5.6.5 Question 2: Recoverable Assets (RA)

Materials that are identified as Recoverable Assets (RA) are on the Q2 page as shown in Figure 5.42. These are materials that have an on-hand balance greater than their one-year annual requirements, so that there is inventory left over that may not be used for one year. There is a matrix table, a bar chart and two KPI cards. All the RA materials are listed on the matrix table along with their current stock, consumption of materials in the past year, excess quantity, and value. The bar chart displays the excess value of each material, enabling the user to instantly see which materials they have the greatest money at risk in. The KPI cards will show the total amount of RA materials and Excess stock value. The

dataset includes 11 materials that are identified as Recoverable Assets and have an excess value of 92.77K. The results indicate that the stock value for VINYL RESIN (MAT010) is the highest surplus at 21.3K, followed by METAL CAN 1L (MAT016) and METAL CAN 4L (MAT017). Interestingly, 6 of the flagged materials did not consume anything during the measurement period, suggesting that they can be dead stock if not consumed again in future.

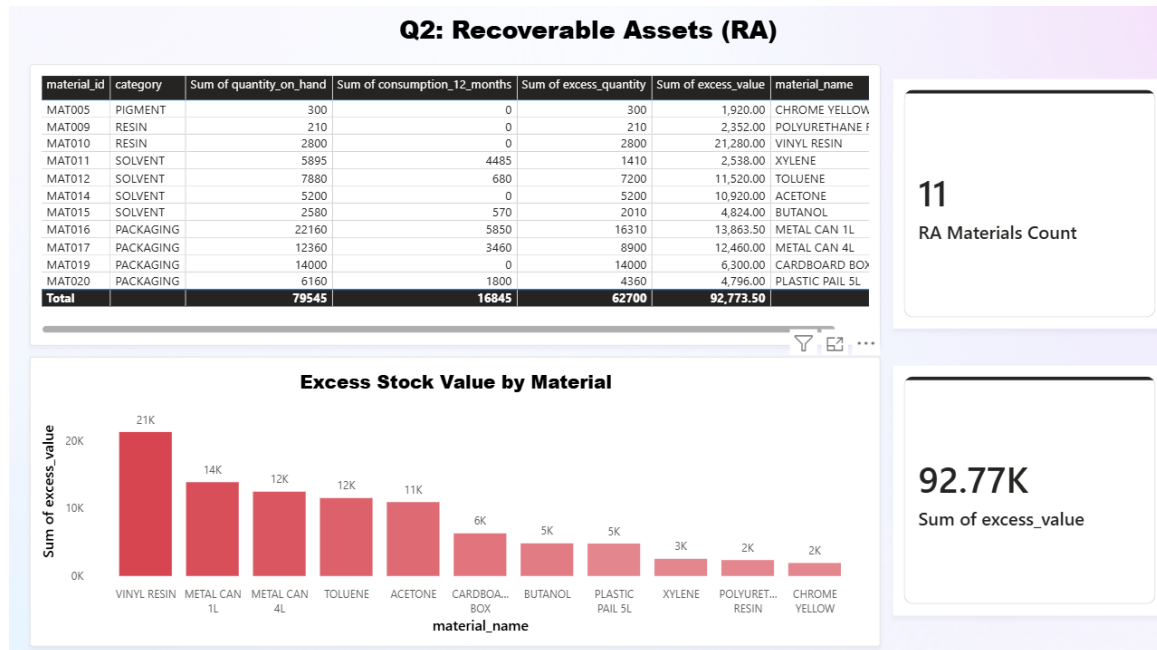


Figure 5.42 Q2 Page – Recoverable Assets Dashboard Page

5.6.6 Question 3 – MC Dead Stock Materials

The Q3 page is dedicated to resources for Magna Carta Dead Stock as shown in Figure 5.43. These include materials which have not been used for a long time and may need financial support in the future if they are not in use. This page features a Matrix table, a bar chart and cards with KPIs. The table shows the material category, the quantity of the material on hand, the last date the material was consumed, and the number of months since the last consumption. The bar chart enables users to make comparisons of inactivity times on different materials. Several materials were not used for a long time, but the present dataset has no materials which completely satisfy the dead stock criterion. The Dead Stock Count KPI card and Dead Stock Provision KPI card both show a value of 0. A note is added to the dashboard to alert users to potential risks when materials are above the set dead stock limit.

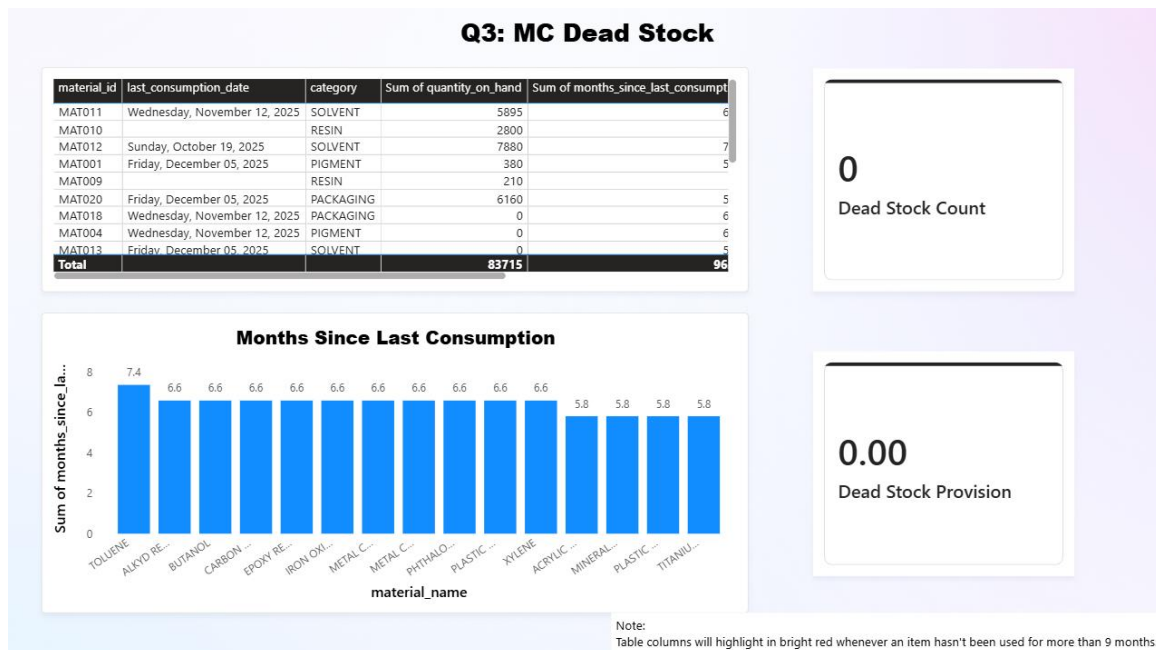


Figure 5.43 Q3 Page – MC Dead Stock Materials Dashboard

5.6.7 Question 4: MC Expired Materials

The Q4 page specifies materials that have expired and are deemed to be Magna Carta Expired materials as shown in Figure 5.44. These materials are subject to complete financial provision per Company policy, and can't be used in operations. Two KPI cards are found at the top of the page that show the total amount of expired materials and the amount of provision available. A bar chart displays the provision value for each expired material, with extra information in the matrix table including expiry date, stock and provision value. There's also a date slicer to provide a way for users to filter records by expiry periods. Based on this analysis, 29.72K is the total provision amount, with three materials being expired. TOLUENE contributes the highest provision value (12.6K), followed by ACETONE and BUTANOL. The 3 materials have all expired on 30 June 2025. The results listed above reflect the direct financial consequences of the expired inventory, and offer valuable data for inventory and financial management.

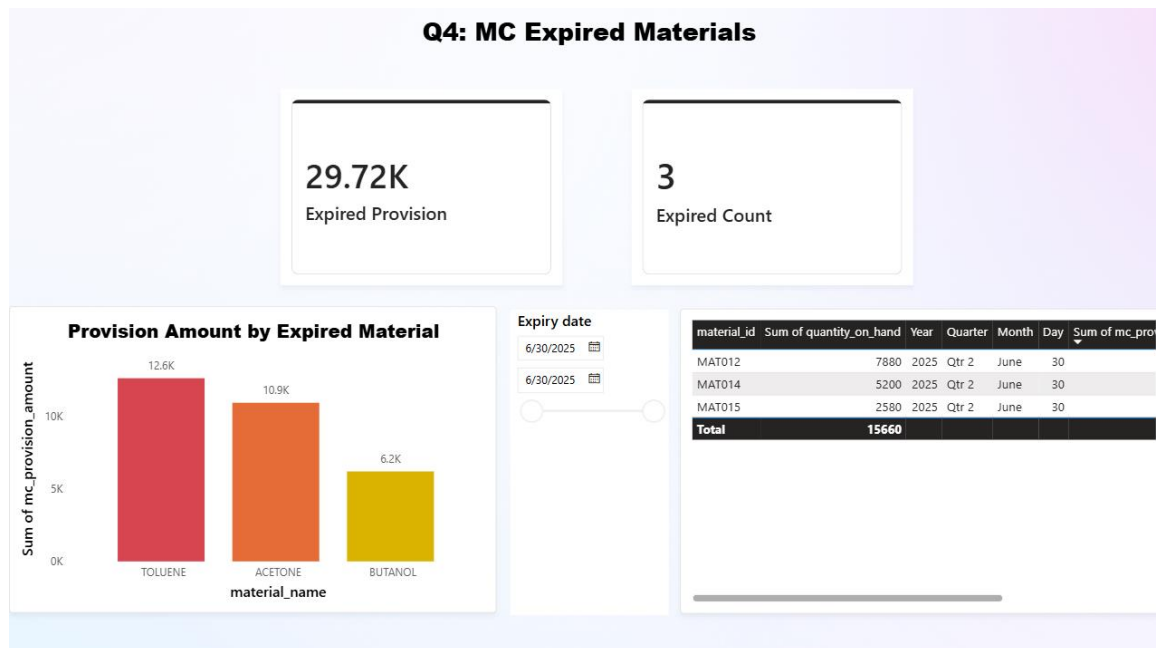


Figure 5.44 Q4 Page – MC Expired Materials Dashboard

5.6.8 Question 5: Financial Impact of Magna Carta (MC)

The Q5 page gives a summary of the overall financial risks to the Magna Carta (MC) inventory. This page uses the pre-calculated table “mc_provision_summary” from the Gold layer which already has aggregated results. So, no further filtering is needed. As shown in Figure 5.45, at the top of the page, there are five KPI cards displayed. Some of these KPIs are Dead Stock Provision, Total MC Provision, Expired Provision, Dead Stock Material Count, and Expired Material Count. With the information currently available, the Total MC Provision is 29.72K, representing 100% of which expired materials make up. No dead stock provision is recognised as no materials are classified as MC Dead Stock. A donut chart shows the distribution of provision by MC category to give a more detailed picture of provision distribution. All provisions are linked to old materials and therefore there is only one provision category, for 100% provision. There's also a bar chart on the page that displays the number of materials in each inventory risk category. The results indicate that there are 11 Recoverable Asset materials, 3 Expired materials, and no Dead Stock materials. This visual enables stakeholders to get a quick overview of the general inventory risk profile. Considering all in all, the analysis reveals that the company now has an MC provision liability of 29.72K for expired inventories. In the event that the material has expired, it cannot be used and the value is considered to be fully impaired and should be recognised as a financial adjustment in the current reporting period. This information can

be used for both inventory management and financial planning.

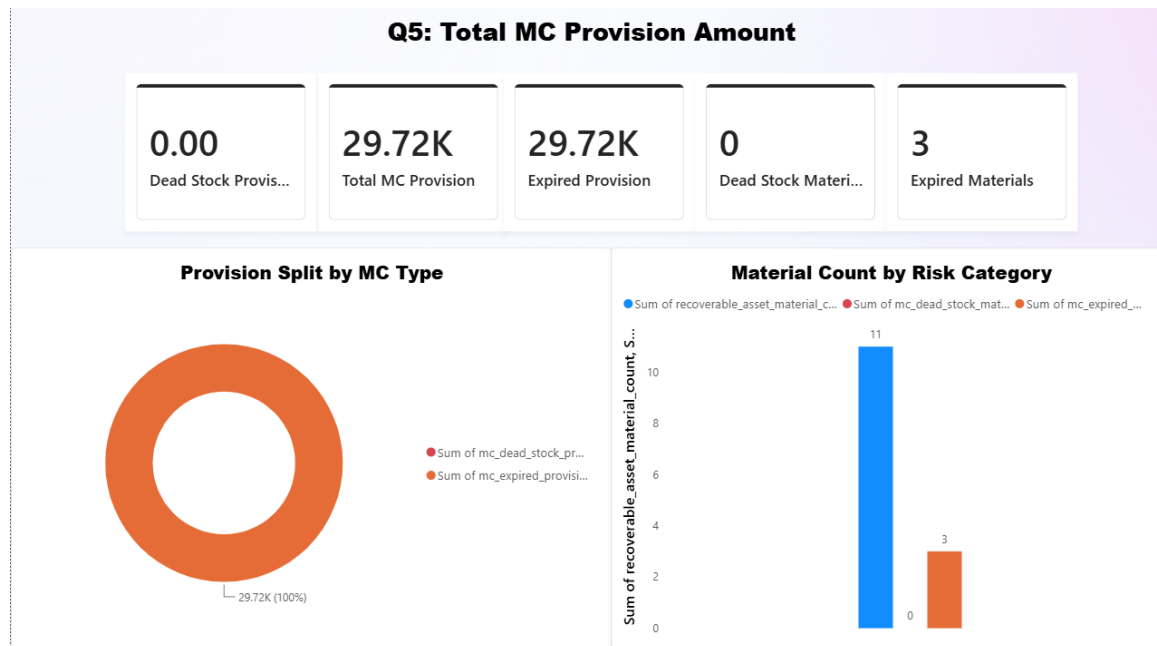


Figure 5.45 Q5 Page – Total MC Provision Amount Dashboard

5.6.9 Question 6: Unfulfilled Production Orders

The Q6 page deals with the effects of shortage of materials on production. It detects orders that can't be processed because the materials needed are unavailable in requisite amounts. As shown in Figure 5.46, there is a bar chart, KPI card and a matrix table on the page. The bar chart shows the quantity of the shortfall for each finished product and is useful in determining that which finished products are most impacted by inventory shortfalls. The results reveal that Premium White Paint 4L has the highest amount of shortage, followed by Marine Blue Coating, Clear Varnish and Industrial Grey Primer. There are ten finished products affected, which suggests the problem is not only present in one product line, but also multiple product lines. As you can see in the KPI card, there are currently 183 production orders that have not been fulfilled. Each production order that is affected is displayed in the matrix table along with the finished product and materials that are causing the shortage. When materials are in short supply, for instance, one production order could be delayed because of a shortage in several materials at once. The results provide a general idea of the direct effect of the adverse impact of the inventory shortage on the production operations. The nine materials that fall below their reorder levels in Q1 are responsible for the delay of 183 production orders, which is a clear indication of the need for timely

replenishment and procurement planning.

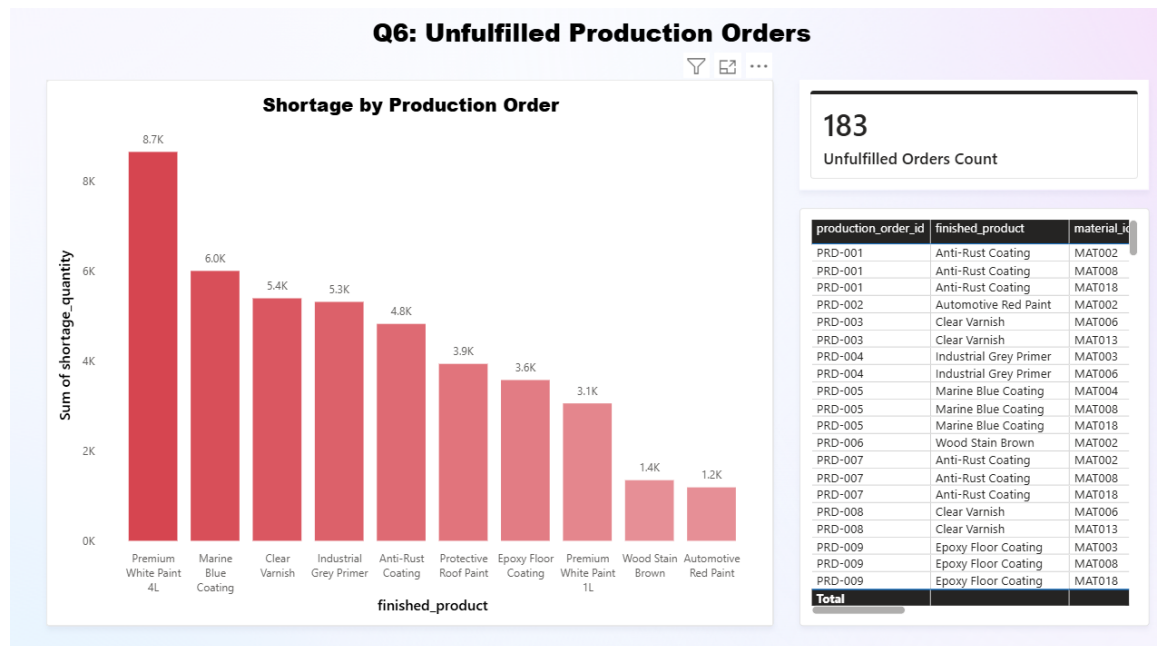


Figure 5.46 Q6 Page – Unfulfilled Production Orders Dashboard

5.6.10 Question 7: Affected Customer Sales Orders

The Q7 page will provide an extended analysis of the impact of inventory shortfalls on customer sales orders. This page is designed to remind people that while Q6 is about disruption of production, when materials are not available the customer may also be impacted with delays in order fulfilment. As indicated in figure 5.47, the page contains a matrix table, bar chart, a material filter and two KPI cards. The matrix table offers detailed information on sales order, such as: Customer name; Finished product; Material used; Order quantity; Shortage quantity; Order status. Currently the orders are still open for all affected orders in the dataset that it is studying. The affected data for each sales order material is displayed in the bar chart. The results show that MINERAL SPIRITS and TITANIUM DIOXIDE have the highest impact of 41 customer orders. The next important material is ALKYD RESIN and several other materials have influence on fewer orders. There is a slicer provided to help users narrow down to a particular material and see which customers and sales orders are impacted by that shortage. Affected sales orders on the KPI cards totals 192 and the overall shortage quantity 78K. The last page completes the inventory risk analysis, connecting shortages of materials to their business implications. The dashboard shows the breakdown of the impact from the materials below the reorder

level (Q1) through to production orders (Q6) and customer sales orders (Q7). This enables both the operations and selling team to be aware of the consequences of stock-outs and plan accordingly to prevent negative impacts and manage customer expectations.

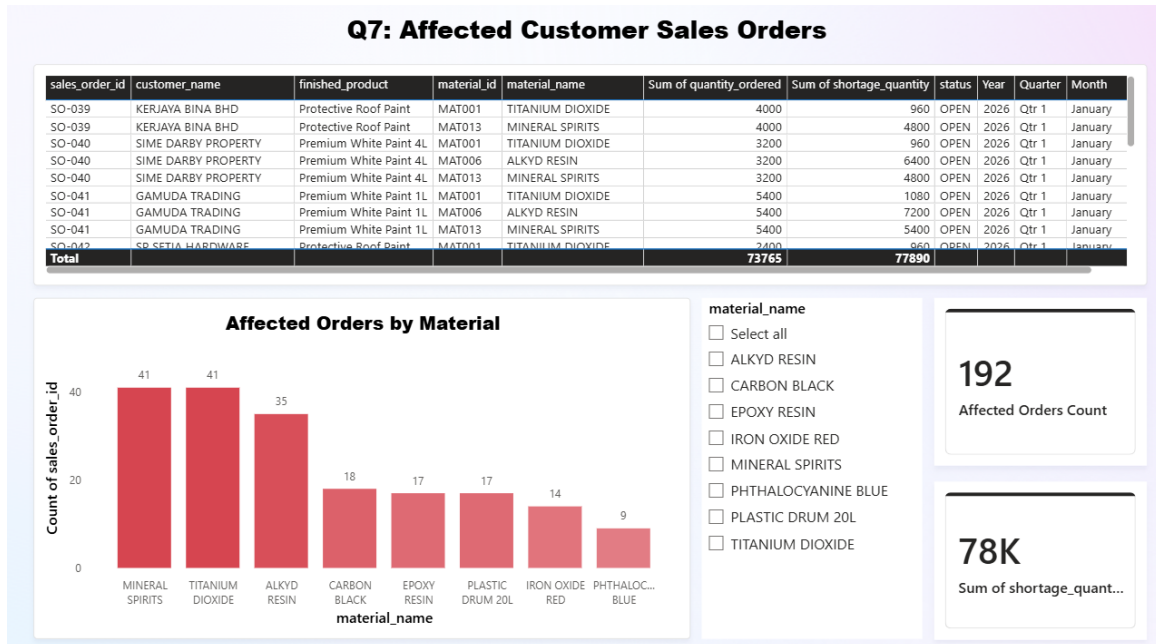


Figure 5.47 Q7 Page – Affected Customer Sales Orders Dashboard

5.7 Summary

Chapter 5 has presented the complete implementation of the end-to-end data pipeline for this project. Each layer was successfully built and validated, from raw data ingestion in the Bronze layer, data cleaning in the Silver layer, business logic transformation in the Gold layer, star schema design in the Curated layer, and finally the Power BI dashboard that visually answered all seven business questions. Overall, the implementation confirmed that the pipeline was functioning as intended and could support inventory risk management decisions based on the processed data.

REFERENCES

- Allen, L., Atkinson, J., Jayasundara, D., Cordiner, J., & Moghadam, P. Z. (2021). Data visualization for Industry 4.0: A stepping-stone toward a digital future, bridging the gap between academia and industry. *Patterns*, 2(6), 100266. <https://doi.org/10.1016/j.patter.2021.100266>
- Armbrust, M., Ghodsi, A., Xin, R., & Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR)*. https://www.cidrdb.org/cidr2021/papers/cidr2021_paper17.pdf
- Bandara, F., Jayawickrama, U., Subasinghage, M., Olan, F., Alamoudi, H., & Alharthi, M. (n.d.). Enhancing ERP responsiveness through Big Data Technologies: An Empirical investigation. *Information Systems Frontiers*, 251–275.
- Behera, L., & Chilukoori, V. V. R. (2024). End-to-End Data Pipelines: Redefining the architecture of data engineering in cloud environments. In *ESP International Journal of Advancements in Science & Technology*. <https://doi.org/10.56472/25839233/IJAST-V2I4P104>
- Harun, S., Dorasamy, M., Ahmad, A. a. B., Yap, C. S., & Harguem, S. (2023). Enterprise resource planning implementation within science and technology park (STP) organisations: an avenue for future research. A systematic review. *F1000Research*, 10, 1148. <https://doi.org/10.12688/f1000research.73347.3>
- Issah, O., Yeboah, H. O., Agboyi, R. M., Asare, C., & Baah, S. A. (2025). Inventory Management Techniques and Supply Chain Efficiency: the Moderating Effect of Technology Readiness. *Journal of Business and Strategic Management*, 10(2), 1–24. <https://doi.org/10.47941/jbsm.2514>
- Kavala, Y. (2022). Smart ERP: scalable data engineering frameworks using artificial Intelligence. *International Journal of Computational Mathematical Ideas (IJCMI)*, 15(1), 1248–1262. <https://doi.org/10.70153/IJCMI/2023.15302>
- Kimball, R., & Ross, M. (2013). *The data warehouse toolkit: The definitive guide to dimensional modeling* (3rd ed.). Wiley.
- Mentzer, J. T., DeWitt, W., Keebler, J. S., Min, S., Nix, N. W., Smith, C. D., & Zacharia, Z. G. (2001). DEFINING SUPPLY CHAIN MANAGEMENT. *Journal of Business Logistics*, 22(2), 1–25. <https://doi.org/10.1002/j.2158-1592.2001.tb00001.x>
- Mishra, A., & Alzoubi, Y. I. (2023). Structured software development versus agile software development: A comparative analysis. *International Journal of System Assurance*

- Engineering and Management, 14(4), 1504–1522. <https://doi.org/10.1007/s13198-023-01958-5>
- Nitsche, B. (2018). Unravelling the complexity of supply chain volatility management. *Logistics*, 2, 14. <https://doi.org/10.3390/logistics2030014>
- Saravanos, A., & Curinga, M. X. (2023). Simulating the software development lifecycle: The waterfall model. *Applied System Innovation*, 6(6), 108. <https://doi.org/10.3390/asi6060108>
- Zimmermann, R., & Brandtner, P. (2024). From data to decisions: Optimizing supply chain management with machine learning-infused dashboards. *Procedia Computer Science*, 237, 955–964. <https://doi.org/10.1016/j.procs.2024.05.184>